

ΕΘΝΙΚΟ ΚΑΙ ΚΑΠΟΔΙΣΤΡΙΑΚΟ  
ΠΑΝΕΠΙΣΤΗΜΙΟ ΑΘΗΝΩΝ  
ΤΜΗΜΑ ΠΛΗΡΟΦΟΡΙΚΗΣ  
ΚΑΙ ΤΗΛΕΠΙΚΟΙΝΩΝΙΩΝ

ΣΗΜΕΙΩΣΕΙΣ  
ΑΝΤΙΚΕΙΜΕΝΟΣΤΡΑΦΟΥΣ  
ΠΡΟΓΡΑΜΜΑΤΙΣΜΟΥ

ΙΖΑΜΠΩ ΚΑΡΑΛΗ  
ΑΘΗΝΑ – 2025

## Περιεχόμενο του μαθήματος

- Γενικά για τον αντικειμενοστραφή προγραμματισμό και τις κλάσεις
- Εμβέλεια ονομάτων
- Αφαίρεση στα δεδομένα (σε C++)
- Δημιουργία / καταστροφή αντικειμένων (σε C++)
- Σύνθεση κλάσεων (σε C++)
- Κληρονομικότητα (σε C++)
- Πολυμορφισμός (σε C++)
- Άλλες γλωσσικές δομές της C++, κρίσιμες για τις κλάσεις
- Μετάβαση στη γλώσσα Java

## Γενικά Στοιχεία για τον Αντικειμενοστραφή Προγραμματισμό

Θα μπορούσατε να μας διευκρινήσετε τί ακριβώς εξυπηρετεί ο αντικειμενοστραφής προγραμματισμός; Όχι πώς υλοποιείται (με κλάσεις, αντικείμενα κλπ), αλλά ποια είναι η πρακτική χρησιμότητα και η ουσιαστική διαφορά στις δυνατότητές του από τον διαδικαστικό. Γιατί δηλαδή κάποιος θα χρησιμοποιήσει πχ τη C++ για να φτιάξει ένα πρόγραμμα, αντί για τη C.

- Ο αντικειμενοστραφής προγραμματισμός (object oriented programming - OOP) επιτρέπει στον προγραμματιστή να οργανώνει τον κώδικά του σε “κλάσεις”(classes) οι οποίες αντιστοιχούν σε οντότητες ή, γενικότερα, έννοιες της εφαρμογής του, όπως οι κατηγορίες δεδομένων της εφαρμογής, π.χ. οι “αίθουσες”, οι “διδάσκοντες” κ.ά. σε μια εφαρμογή κατασκευής σχολικού ωρολογίου προγράμματος.
- Οι κλάσεις μπορεί να συνδέονται μεταξύ τους με μια σχέση εξειδίκευσης. Π.χ. μια “αίθουσα εργαστηρίου” είναι μια ειδική κατηγορία “αίθουσας”. Αν μεταξύ δύο κλάσεων δηλώσουμε σχέση εξειδίκευσης στο πρόγραμμά μας, δεν χρειάζεται να επαναλάβουμε στον ορισμό της ειδικότερης κλάσης ό,τι έχει οριστεί στη γενικότερη, διότι “κληρονομείται” αυτόματα. Η ιδιότητα αυτή αναφέρεται ως “κληρονομικότητα” (inheritance) μεταξύ των κλάσεων.

- Αφού οριστούν κλάσεις, ο προγραμματιστής μπορεί να δημιουργεί συγκεκριμένα στιγμιότυπα (instances) των κλάσεων αυτών μέσα στον κώδικά του, π.χ. την αίθουσα “A35” ως στιγμιότυπο ‘αίθουσας’. Στο πλαίσιο του OOP, τα στιγμιότυπα αυτά ονομάζονται “αντικείμενα” (objects). Ο τρόπος με τον οποίο ο προγραμματιστής χειρίζεται τα αντικείμενα μιας κλάσης στο πρόγραμμά του καθορίζεται από τον ορισμό της κλάσης. Συγκεκριμένα, μέσα στον ορισμό μιας κλάσης καθορίζεται η “συμπεριφορά” (behavior) των αντικειμένων της, η οποία προκύπτει από ένα σύνολο “ενεργειών” που μπορούν να εφαρμοστούν στα αντικείμενα αυτά, π.χ. η ενέργεια “ανάθεση μαθήματος” για τα αντικείμενα της “αίθουσας”.

## Χρήση του Αντικειμενοστραφούς Προγραμματισμού

- Η ανάγκη για τη δυνατότητα ορισμού κλάσεων από μια γλώσσα προγραμματισμού προέκυψε από τις απαιτήσεις εφαρμογών με περίπλοκες οντότητες -συγκεκριμένα, εφαρμογές προσομοίωσης.
- Η τμηματοποίηση του κώδικα που επιτρέπουν οι κλάσεις διέυρυναν τη χρήση του OOP σε κάθε περίπτωση που απαιτείται ανάπτυξη τμηματοποιημένου ή/και εκτενούς κώδικα.
- Καταλήξαμε στις μέρες μας σχεδόν όλες οι γλώσσες προγραμματισμού να προσφέρουν γλωσσικά στοιχεία OOP.

## Πλεονεκτήματα του Αντικειμενοστραφούς Προγραμματισμού

Συνοπτικά, οι κλάσεις επιτρέπουν:

- απευθείας αναπαράσταση των οντοτήτων ή/και των εννοιών του προβλήματος προς επίλυση στην εφαρμογή και
- την κατάτμηση του κώδικα της εφαρμογής σε τμήματα, τις κλάσεις.

Κατά συνέπεια, τόσο η ανάπτυξη όσο και η συντήρηση της εφαρμογής γίνονται ευκολότερα. Η τμηματοποίηση επιτρέπει διαμοιρασμό του έργου της ανάπτυξης, καθώς και επαναχρησιμοποίηση τμημάτων κώδικα. Και τα δύο παραπάνω χαρακτηριστικά δε, οδηγούν στην παραγωγή πιο ευανάγνωστου κώδικα με ό,τι αυτό συνεπάγεται.

## Διαδικαστικός και Αντικειμενοστραφής Προγραμματισμός

**Διαδικαστικός Προγραμματισμός** Στον παραδοσιακό διαδικαστικό προγραμματισμό:

- Η έμφαση δίδεται στον σχεδιασμό των διαδικασιών που επιλύουν ένα πρόβλημα.
- Προσδιορίζουμε την ακολουθία των βημάτων/ενεργειών (actions) που οδηγούν στην επίλυσή του. Οι διαδικασίες επιδρούν σε κάποια δεδομένα (εισόδου) και οι ενέργειες αποσκοπούν στον μετασχηματισμό των δεδομένων αυτών, ώστε να καταλήξουν στο επιθυμητό αποτέλεσμα.
- Η προσπάθεια εστιάζει στον σχεδιασμό και την υλοποίηση των διαδικασιών αυτών.

**ΟΟΡ** Στον αντικειμενοστραφή προγραμματισμό, η εστίαση αλλάζει:

- Σχεδιασμός με έμφαση στις οντότητες της εφαρμογής, συνήθως τα δεδομένα.
- Αυτές τις συσχετίζουμε με μια συμπεριφορά που οφείλουν να ακολουθούν όλα τα στιγμιότυπά τους.
- Η συμπεριφορά αυτή υλοποιείται μέσω διαδικασιών.

## Διαδικαστικός vs Αντικειμενοστραφής Προγραμματισμός – Συνοπτικά:

Στον διαδικαστικό προγραμματισμό, σχεδιάζουμε και υλοποιούμε διαδικασίες (που επιδρούν σε δεδομένα), στον (διαδικαστικό) OOP σχεδιάζουμε και υλοποιούμε κλάσεις (που συσχετίζονται με διαδικασίες). Το τι είναι κατάλληλο να επιλέξουμε, γλώσσα διαδικαστικού προγραμματισμού (π.χ. C) ή γλώσσα με αντικειμενοστραφή χαρακτηριστικά (π.χ. C++), εξαρτάται από τη φύση του προβλήματος που έχουμε να αντιμετωπίσουμε.



## Λογικά Βήματα κατά την Εξέλιξη του (Διαδικαστικού) Αντικειμενοστραφούς Προγραμματισμού

### *Διαδικαστικός Προγραμματισμός* (procedural programming)

- προβλήματα που απαιτούν εύρεση διαδικασιών για την επίλυσή τους
- εστιάζει στη σχεδίαση των διαδικασιών
- “αποφάσισε ποιες είναι οι διαδικασίες που χρειάζονται για την υλοποίηση και χρησιμοποίησε τους καλύτερους αλγορίθμους που μπορείς να βρείς”
- Παραδείγματα διαδικασιών:
  - εύρεση της τετραγωνικής ρίζας
  - εύρεση Μ.Κ.Δ.
  - ταξινόμηση αριθμών
- Γλώσσες: Algol60, Algol68, FORTRAN, Pascal, C κ.ά.

## Προγραμματισμός με Στοιχεία (modular programming)

*Σύγκρουση ονομάτων: δύο διαφορετικοί ορισμοί για τη συνάρτηση void foo()*

```
void bla1() {  
    ...  
    foo();  
    ...  
}  
  
void foo() { ... }
```

```
void bla2() {  
    ...  
    bla1();  
    ...  
    foo();  
    ...  
}  
  
void foo() { ... }
```

*Λύση (;): Οι συναρτήσεις void foo() μετονομάστηκαν με διαφορετικά ονόματα*

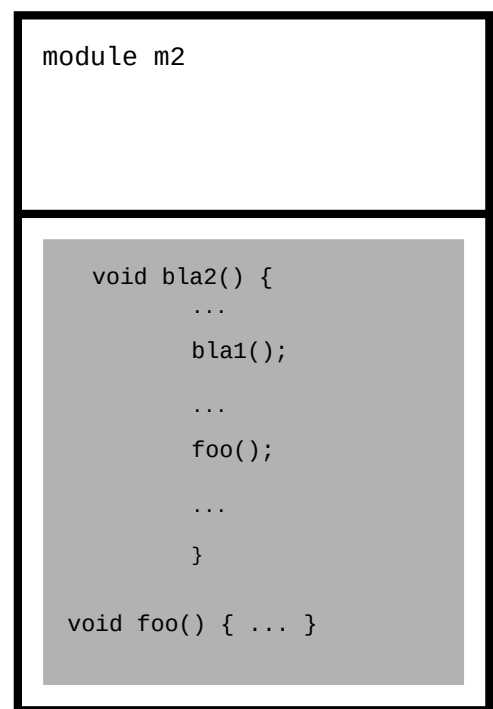
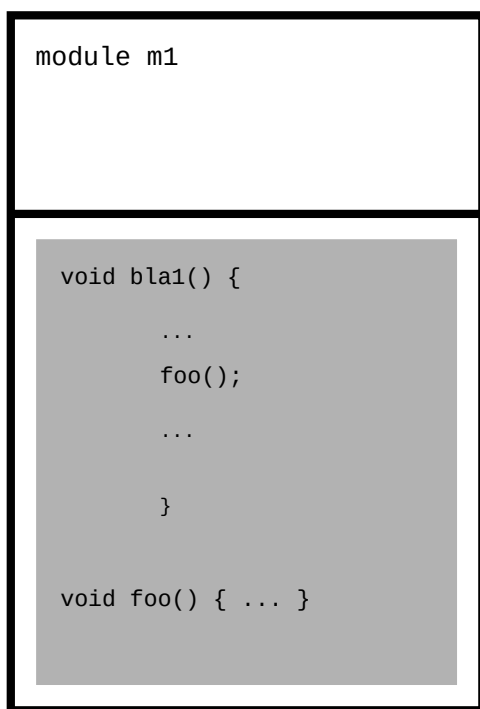
```
void bla1() {  
    ...  
    foo_Sep20();  
    ...  
}
```

```
void foo_Sep20() { ... }
```

```
void bla2() {  
    ...  
    bla1();  
    ...  
    foo_Oct20();  
    ...  
}
```

```
void foo_Oct20() { ... }
```

*Λύση: Ο κώδικάς μας μέσα σε στοιχεία (modules)*



*Ολοκληρωμένη Λύση: Ο κώδικάς μας μέσα σε στοιχεία με τις διεπαφές τους*

```
module m1
export void bla1()
```

```
void bla1() {
    ...
    foo();
    ...
}

void foo() { ... }
```

```
module m2
export void bla2()
import void bla1() from m1
```

```
void bla2() {
    ...
    bla1();
    ...
    foo();
    ...
}

void foo() { ... }
```

## *Προγραμματισμός με Στοιχεία*

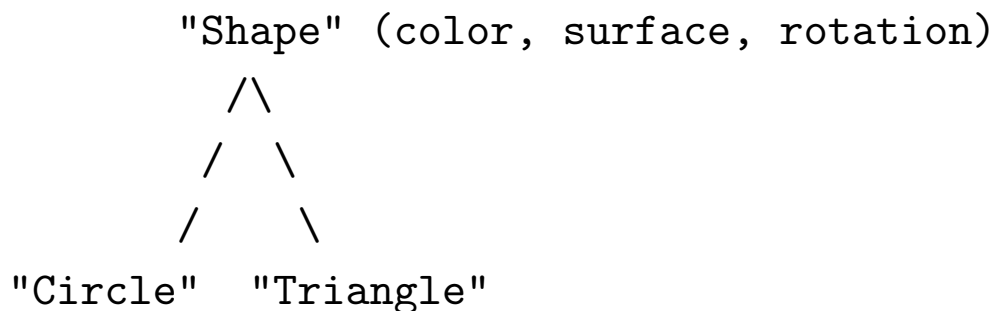
- προβλήματα που απαιτούν ανάπτυξη εκτενούς κώδικα
- εστιάζει στην οργάνωση των δεδομένων και των διαδικασιών
- “αποφάσισε ποια είναι τα στοιχεία (modules) που σου χρειάζονται και χώρισε το πρόγραμμά σου έτσι ώστε να κρύψεις δεδομένα και διαδικασίες στα κατάλληλα στοιχεία”
- υπάρχουν κριτήρια για το χωρισμό (βλ. [Meyer])
- Παραδείγματα στοιχείων:
  - στοιχείο διαδικασιών επεξεργασίας προσωπικού
  - στοιχείο διαδικασιών επεξεργασίας εγγράφων
  - Γλώσσες: Modula -2, Modula -3 κ.ά.

## *Αφαίρεση Δεδομένων* (data abstraction)

- περισσότερη έμφαση στα δεδομένα
- συνδέει τύπους με λειτουργίες μέσω των οποίων χειριζόμαστε τα στοιχεία τους
- “αποφάσισε ποιους τύπους χρειάζεσαι, δώσε ένα πλήρες σύνολο λειτουργιών για τον κάθε τύπο”
- Παραδείγματα τύπων: stack (top, pop, push)
- Γλώσσες: Ada, C++, Java κ.ά.

## *Αντικειμενοστραφής Προγραμματισμός* (Object oriented programming)

- προσδιορισμός ιεραρχιών τύπων δεδομένων
- έκφραση “κοινών” χαρακτηριστικών μεταξύ τύπων μέσω κληρονομικότητας
- “αποφάσισε ποιες κλάσεις θέλεις, δώσε ένα πλήρες σύνολο από λειτουργίες για την κάθε κλάση, έκφρασε ρητά τις κοινές λειτουργίες μέσω κληρονομικότητας”
- Παραδείγματα κλάσεων:



- Γλώσσες: Simula, C++, Java κ.ά.



## Γλώσσες με Χαρακτηριστικά Αντικειμενοστραφούς Προγραμματισμού (παραδείγματα)

- ADA
- Smalltalk (1972)
- Modula -3
- C++ (1980)
- Java
- C#
- SIMULA (1967): προσφέρει τη γλωσσική δομή “class” για τον ορισμό κλάσεων, κατόπιν μπορούν να οριστούν αντικείμενα της κλάσης, *ΟΜΩΣ* δεν υπάρχει μηχανισμός προστασίας για τα ονόματα μέσα σε μια κλάση κι έτσι ο χρήστης μπορεί να προσπελάσει οποιοδήποτε μέρος της υλοποίησης
- Πλέον, σχεδόν όλες οι γλώσσες προγραμματισμού γενικής χρήσης υποστηρίζουν χαρακτηριστικά αντικειμενοστραφούς προγραμματισμού

...κι ακόμα

- Συναρτησιακός Προγραμματισμός (Functional Programming) με αντικειμενοστραφή χαρακτηριστικά
- Λογικός Προγραμματισμός (Logic Programming) με αντικειμενοστραφή χαρακτηριστικά

### Σημαντική Αναφορά

- Κλάσεις Εγγραφών (Record Classes) C. A. R. Hoare (1965): Σύνθετα αντικείμενα, ενταγμένα στην γλώσσα ALGOL 60.

## **Πού Αλλού Συναντάμε το Αντικειμενοστραφές Μοντέλο**

- Ανάλυση και Σχεδίαση Λογισμικού (π.χ. γλώσσα UML)
- Αναπαράσταση Γνώσης
- Βάσεις Δεδομένων

## Τύποι Ορισμένοι από τον Χρήστη

- Έστω εφαρμογή χρονοπρογραμματισμού προσωπικού για τη στελέχωση αεροπορικών πτήσεων.
- Ας θεωρήσουμε μια αεροπορική πτήση, εννοιολογικά, σαν μια δόμηση δεδομένων ως εξής:

### Αεροπορική Πτήση

|                |              |
|----------------|--------------|
| Αριθμός Πτήσης | ακέραιος     |
| Πόλη Αφετηρίας | συμβολοσειρά |
| Πόλη Αφίξης    | συμβολοσειρά |
| Ώρα Αναχώρησης | Χρόνος (;)   |
| Διάρκεια       | Χρόνος (;)   |

Οι Προδιαγραφές του Νέου Τύπου “Χρόνος”,  
έστω *Time*

*read* :  $\rightarrow \underline{Time}$

*print* :  $Time \rightarrow$

*set* :  $integer\ integer \rightarrow \underline{Time}$

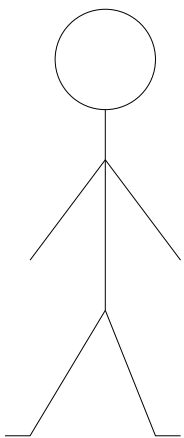
$+$  :  $Time\ Time \rightarrow Time$

*get* :  $Time \rightarrow (\underline{integer}, \underline{integer})$

Υλοποίηση της Έννοιας “Αεροπορική Πτήση”  
(Flight) σε Ψευδογλώσσα

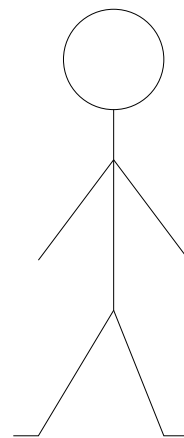
```
struct Flight {  
    int FlightNo;  
    string DepartureCity;  
    string ArrivalCity;  
    Time DepartureTime;  
    Time Duration;  
}
```

Οι δύο προγραμματιστές της εφαρμογής και η  
“συμφωνία” τους για τις προδιαγραφές του τύπου  
Time



Programmer A:  
the type user

```
read(?Time)
print(Time)
set(?Time,int,int)
get(Time,?int,?int)
Time +(Time,Time)
```



Programmer B:  
the type implementor

## Η Χρήση του Νέου Τύπου Time - Παράδειγμα

```
Flight AthensParis;
```

```
AthensParis.DepartureCity = "Athens";
```

```
AthensParis.ArrivalCity = "Paris";
```

```
AthensParis.FlightNo = 314;
```

```
set(AthensParis.DepartureTime, 10, 30);
```

```
set(AthensParis.Duration, 3, 15);
```

```
Time ArrivalTime;
```

```
ArrivalTime = AthensParis.DepartureTime +  
              AthensParis.Duration;
```

```
print(ArrivalTime);
```



## Η Αναπαράσταση (των Μεταβλητών) του Νέου Τύπου Time - Εναλλακτικά

**Time t;**

*1st alternative*

|            |                |
|------------|----------------|
| <i>int</i> | <b>t.hours</b> |
| <i>int</i> | <b>t.mins</b>  |

*2nd alternative*

|            |                    |
|------------|--------------------|
| <i>int</i> | <b>t.totalMins</b> |
|------------|--------------------|

## Ορισμός Νέου Τύπου

Ορισμός τύπων σε ψευδογλώσσα:

```
my_type  ⟨ TypeName ⟩ {  
  
    ⟨ hidden names declarations ⟩  
  
    use_by:  
  
    ⟨ specs names declarations ⟩  
}
```

Ο ορισμός του τύπου Time μπορεί να είναι:

```
my_type Time {  
    int hours;  
    int mins;  
use_by:  
    read(?Time);  
    print(Time);  
    set(?Time,int,int);  
    get(Time,?int,?int);  
    Time +(Time,Time);  
}
```

## Οι ορισμοί των συναρτήσεων

```
read(?Time t) {  
    read(t.hours); read(t.mins);  
}
```

```
print(Time t) {  
    print(t.hours); print("hours");  
    print(t.mins); print("mins");  
}
```

```
set(?Time t, int h, int m) {  
    t.hours = h; t.mins = m;  
}
```

```
get(Time t, ?int h, ?int m) {  
    h = t.hours; m = t.mins;  
}
```

```
Time +(Time t1, Time t2) {  
    Time result;  
    result.hours = 0;  
    results.mins = t1.mins + t2.mins;  
    if (result.mins >= 60) {  
        result.mins = result.mins - 60;  
        result.hours = results.hours + 1;  
    }  
    result.hours = result.hours + t1.hours + t2.hours;  
    return result;  
}
```

Κακή χρήση του τύπου

```
AthensParis.Duration.hours++;
```

αντί για:

```
set(AthensParis.Duration, 4, 15);
```

Αλλαγή του ορισμού του τύπου Time, π.χ. λόγω αναγκών χρήσης:

```
my_type Time {  
    int totalMins; // this is the change  
                  // in the type definition  
use_by:  
    read(?Time);  
    print(Time);  
    set(?Time,int,int);  
    get(Time,?int,?int);  
    Time +(Time,Time);  
}
```

## Ορισμοί συναρτήσεων με το νέο ορισμό του τύπου

```
read(?Time t) {  
    int h, m;  
    read(h); read(m);  
    t.totalMins = h * 60 + m;  
}
```

```
print(Time t) {  
    print(t.totalMins / 60); print("hours");  
    print(t.totalMins % 60); print("mins");  
}
```

```
set(?Time t, int h, int m) {  
    t.totalMins = h * 60 + m;  
}
```

```
get(Time t, ?int h, ?int m) {  
    h = t.totalMins / 60; m = t.totalMins % 60;  
}
```

```
Time +(Time t1, Time t2) {  
    Time result;  
    result.totalMins = t1.totalMins + t2.totalMins;  
    return result;  
}
```

## Υποστήριξη Αντικειμενοστραφούς Προγραμματισμού

- Ορισμός κλάσεων, αντικειμένων, κληρονομικότητας
- Δυνατότητα ελέγχου τύπων
  - λειτουργίες τύπου εφαρμόζονται μόνο πάνω στα αντικείμενα του τύπου
  - αντικείμενα του τύπου προσπελούνται μόνο μέσω λειτουργιών του τύπου
- Δυνατότητα υποστήριξης εγκλωβισμού (encapsulation)
  - επιτυγχάνει απόκρυψη πληροφορίας (information hiding)
  - συνδέει τον τύπο με τις λειτουργίες του
  - μόνο τα χαρακτηριστικά και οι λειτουργίες που καθορίζουν τον τύπο γίνονται ορατά (χρησιμοποιήσιμα) έξω από την κλάση
- Δυνατότητα δημιουργίας αντικειμένων (στιγμιотύπων) και αρχικοποίησης τους
- Ενδεχόμενα, υποστήριξη κατάστασης (state) αντικειμένου



## Ιδιότητες

- Στοιχειοποίηση (“διαίρει και βασίλευε”)
- Εγκλωβισμός (απόκρυψη πληροφορίας)
- Επαναχρησιμοποίηση (reusability)
- Πολυμορφισμός
  - Υπερφόρτωση συμβόλων (Αριθμός + Αριθμός, Λίστα + Λίστα) (overloading)
  - Γενικευμένοι τύποι (`stack_of(X)`), γενικευμένες διαδικασίες (`length_of_list_of(X)`) (Generic Programming)
  - Υποτύποι (subtyping)

## Γενικευμένοι Τύποι

Παράδειγμα: Στοιίβες και Ουρές

- Στοιίβες: από τα πιάτα έως τις κλήσεις συναρτήσεων
- Ουρές: από τους ανθρώπους έως τις διεργασίες

## Απόκρυψη Πληροφορίας

- Αλλαγή ταχυτήτων κατά την οδήγηση αυτοκινήτου -απλά μετακινούμε τον μοχλό, δεν χρειάζεται και δεν επιτρέπεται να βάλουμε τα χέρια μας μέσα στο σασμάν,
- Το τηλεκοντρόλ της τηλεόρασής μας -απλά πατάμε το κουμπί που μας ενδιαφέρει, δεν ασχολούμαστε με το τι γίνεται με το πάτημά του και ούτε μπορούμε να παρέμβουμε.

Ακόμα κι αν μετά από κάποια επισκευή είτε του ενός είτε του άλλου, κάτι αλλάξει εσωτερικά, δεν θα αλλάξει κάτι για εμάς τους χρήστες

## Ονοματολογία για τις Κλάσεις

### Δομική (structural)

Δομικοί λίθοι: attributes, data members, fields

### Συμπεριφοράς (behavioral)

Διαδικασίες μέσω των οποίων προσπελάζουμε τα αντικείμενα: member functions, procedures, methods/messages

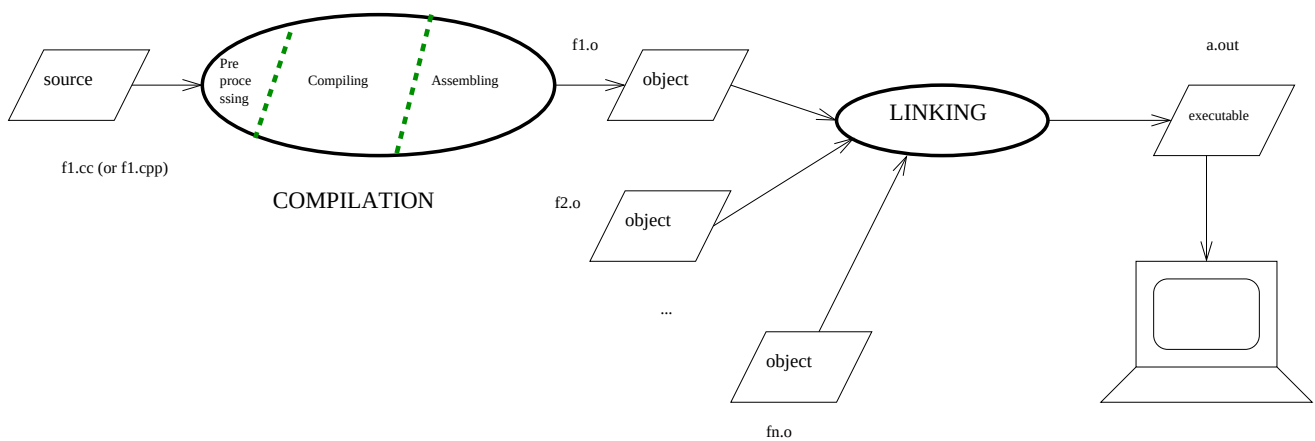
## Κλάσεις και Αφηρημένοι Τύποι Δεδομένων

- “Object Oriented Design is the construction of software systems as structured collections of abstract data type implementations” [Meyer]
- Παράδειγμα: “stack”: υλοποιημένο σαν συνδεδεμένη λίστα ή array
- Αφηρημένοι Τύποι Δεδομένων (ΑΤΔ) : Αφαιρετική περιγραφή τύπων μέσω των ιδιοτήτων των λειτουργιών τους κι όχι λεπτομερειών της αναπαράστασής τους
- Κλάσεις : Συγκεκριμένη μοντελοποίηση και υλοποίηση

## C++ Γενικά

- C++ (=increment of C)
- Σχεδιάστηκε να
  - είναι μια καλύτερη C
  - υποστηρίζει αφαίρεση δεδομένων
  - υποστηρίζει αντικειμενοστραφή προγραμματισμό
- Bjarne Stroustrup, 1980, Bell Labs
- “C με κλάσεις”  $\longrightarrow$  C++ το 1983
- Για μεγαλύτερα προγράμματα
- ΟΟ χαρακτηριστικά εμπνευσμένα από τη Simula67 (C++ =? C + Simula67)
- Εμβέλεια (scope) ονομάτων-νοημάτων
- Κλάσεις, αντικείμενα, κληρονομικότητα, αφηρημένες κλάσεις  
...

## Μεταγλώττιση και Αρχεία Επικεφαλίδας -σε περιβάλλον Unix μορφής



## Το Ελάχιστο C++ Πρόγραμμα

```
int main()
{
    return 0;
}
```

Για να έχουμε και κάποια έξοδο

```
#include <iostream>

using namespace std; // more on this later

int main()
{
    cout << "Hello world" << endl;
    return 0;
}
```

Η δήλωση `#include <iostream>` δηλώνει στον προεπεξεργαστή να περιλάβει τις δηλώσεις που βρίσκονται στο αρχείο-επικεφαλίδα `iostream`

## ΔΙΑΔΙΚΑΣΤΙΚΟΣ ΠΡΟΓΡΑΜΜΑΤΙΣΜΟΣ με/σε C++

- Δεδομένα
  - Ενσωματωμένοι τύποι
  - Τύποι που ορίζονται από το χρήστη
  - Τύποι της καθιερωμένης (standard) βιβλιοθήκης
- Διαδικασίες πάνω στα δεδομένα
  - Δομικά στοιχεία προκειμένου για διατύπωση εκφράσεων προς υπολογισμό
  - Ένα σύνολο εντολών
  - Τη δυνατότητα ορισμού “πακεταρίσματος” κώδικα με τη μορφή συναρτήσεων
  - Συναρτήσεις της καθιερωμένης βιβλιοθήκης

## Τύποι

- Τύπος `boolean` (`bool`) - τιμές: `true`, `false` -
- Τύποι χαρακτήρων (π.χ. `char`)
- Τύποι ακεραίων (π.χ. `int`)
- Τύποι πραγματικών (π.χ. `double`)
- Τύποι απαρίθμησης (π.χ. `enum`)
- απουσία πληροφορίας : `void` (να δηλώσει ότι μια συνάρτηση δεν επιστρέφει τιμή, βασικός τύπος για δείκτες σε αντικείμενα που δεν ξέρουμε τον τύπο τους)
- Παραγόμενοι Τύποι:
  - Τύποι δεικτών (π.χ. `int*` )
  - Arrays (π.χ. `char[]` )
  - Τύποι αναφορών (π.χ. `int&` )
  - `()` δηλώνει συνάρτηση (postfix)
- Αριθμητική σε δείκτες όπως στη C
- Δομές δεδομένων και κλάσεις



## Άλλα θέματα σχετικά με τους τύπους

- Μετατροπές τύπων

- `struct`

```
struct address {
    char* street;
    long number;
    char* city;
    int postcode;
};
```

- Για μετονομασία τύπων, `typedef`

```
typedef char* Pchar;
Pchar p1;
```

Προσοχή: Η χρήση του `typedef` στη C++ είναι πολύ περιορισμένη. Χρησιμοποιείται κυρίως:

- για ονομασία παραγόμενων τύπων, π.χ. `char*`
- σε περιπτώσεις που δεν έχουμε ακόμα αποφασίσει τον τύπο κάποιων μεταβλητών και ενδέχεται να τον αλλάξουμε αργότερα

- `union`

- Στους τύπους θα επανέλθουμε συζητώντας για τις κλάσεις

## Τελεστές

- π.χ.  $+$ ,  $-$ ,  $*$ ,  $/$ ,  $\%$ , ...
- Είναι πλούσια σε τελεστές, όπως η C
- Μερικοί έχουν και νέα σημασία, π.χ.  $<<$

## Μερικές Παρατηρήσεις

- Δεν υπάρχουν εντολές ανάθεσης και εντολές κλήσης συνάρτησης: η ανάθεση και η κλήση συνάρτησης υπάγονται στην κατηγορία των εκφράσεων.
- Όπου γίνονται έλεγχοι τιμών αλήθειας είναι πιο ευανάγνωστο να γράφουμε ρητά τις λογικές εκφράσεις, με χρήση των λογικών τελεστών, παρά να υπονοείται ο έλεγχος με βάση τις μη μηδενικές τιμές (μην ακολουθούμε την προσέγγιση της C να βασιζόμαστε στην μηδενική ή μη μηδενική τιμή της έκφρασης).
- Καλό είναι να χρησιμοποιούμε παρενθέσεις, ( και ), για να κάνουμε σαφή την “ομαδοποίηση” της αποτίμησης μιας έκφρασης.
- Γενικά η σειρά αποτίμησης δεν είναι εγγυημένη.

## Εντολές (à la C)

### Εντολές Επιλογής:

Η γενική μορφή τους είναι

- `if ( expression ) statement`
- `if ( expression ) statement else statement`
- `switch ( expression ) statement`

Στην κατηγορία αυτή, ελέγχεται η τιμή που έχει το *expression* για να μεταφερθεί, ανάλογα, ο έλεγχος της εκτέλεσης του προγράμματος.

### Εντολές Επανάληψης:

Η γενική μορφή τους είναι

- `while ( expression ) statement`
- `do statement while ( expression );`
- `for ( init-statement ; expression ; expression ) statement`

Η κατηγορία αυτή εκφράζει την επανάληψη του *statement*.

## Εντολές Μεταφοράς Ελέγχου:

Η γενική μορφή τους είναι

- *case constant-expression : statement*
- *default: statement*
- *break;*
- *continue;*
- *return expression ;*
- *goto identifier ;*

Η κατηγορία αυτή εκφράζει μεταφορά ελέγχου. Εκτός από την περίπτωση χρήσης *case*, *default* και *break* σε εντολή *switch*, καθώς και του *return* για την επιστροφή από συνάρτηση, κάθε άλλη ρητή μεταφορά ελέγχου καλό είναι να αποφεύγεται.

**Ονομασία Εντολής:** Η εντολή *identifier : statement* δίνει το όνομα *identifier* στο *statement*

## Στοιχεία της C++ με Ιδιαίτερο Ενδιαφέρον για τις Κλάσεις

### Σταθερές

- Η λέξη-κλειδί `const` μπορεί να συμπεριληφθεί σε μια δήλωση προσδιοριστή (identifier) για να τον μετατρέψει σε σταθερά αντί για μεταβλητή. Π.χ.

```
const int a = 5;
```

το περιεχόμενο του `a` δεν μπορεί να αλλάξει

- Ακριβώς επειδή το περιεχόμενο δεν μπορεί να αλλάξει, ένας προσδιοριστής που δηλώνεται με `const` πρέπει να αρχικοποιείται
- Ο μεταγλωττιστής μπορεί να εκμεταλλευτεί μια δήλωση `const` (με διάφορους τρόπους, ανάλογα με το πόσο έξυπνος είναι)
- Σταθερές και Δείκτες:  
Έστω ότι έχουμε τον ορισμό: `int i = 5;`. Τότε,
  1. Δήλωση σταθεράς για το αντικείμενο στο οποίο δείχνει ο δείκτης:
 

```
const int* pc = &i;
```
  2. Δήλωση σταθεράς για τον δείκτη:
 

```
int *const cp = &i;
```
  3. Δήλωση σταθεράς και για τα δύο.

```
const int *const cpc = &i;
```

## Παρατηρήσεις για τη χρήση του `const`

- χρήσιμο για ορίσματα συναρτήσεων: δηλώνοντας ένα όρισμα σαν `const`, η συνάρτηση απαγορεύεται να αλλάξει την τιμή του μέσα στο σώμα της
- ορισμοί `const` προσδιοριστών ισχύουν μόνο μέσα στο αρχείο που εμφανίζονται –έχουν εσωτερική συνδεσιμότητα (`internal linkage`).

## Συναρτήσεις

- Ο τρόπος για να “κάνουμε κάτι”.
- Μια συνάρτηση δεν μπορεί να κληθεί αν δεν έχει δηλωθεί.
- Μια δήλωση συνάρτησης περιλαμβάνει:
  - το όνομα της συνάρτησης
  - τον τύπο της τιμής που επιστρέφει
  - το πλήθος και τους τύπους των ορισμάτων της
  - π.χ.

```
int sum(int i, int j);
void swap(int*, int*);
```

- Μια συνάρτηση πρέπει να ορίζεται μόνο μια φορά για μία *εμβέλεια (scope)*.
- Κατά την κλήση μιας συνάρτησης, η αντιμετώπιση είναι ίδια με την *αρχικοποίηση* των ορισμάτων (κι όχι με την ανάθεση τιμών). Δηλαδή, όταν καλείται μια συνάρτηση, οι χώροι που έχουν φυλαχθεί για τις τυπικές παραμέτρους της *αρχικοποιούνται* με τα αντίστοιχα ορίσματα.
- Επιστροφή τιμών: ίδια διαδικασία με την αρχικοποίηση μεταβλητών (μια εντολή `return` λειτουργεί σαν να *αρχικοποιεί* μια μεταβλητή του τύπου που επιστρέφει η συνάρτηση, στην οποία δίνεται η τιμή της έκφρασης της `return`, πραγματοποιώντας τις ανάλογες μετατροπές τύπων).

## Συναρτήσεις inline

- Μια συνάρτηση μπορεί να δηλωθεί ως `inline`, δίνοντας μια συμβουλή στον μεταγλωττιστή να αντικαταστήσει με τον κώδικα της συνάρτησης τις κλήσεις της συνάρτησης, αντί να χρησιμοποιήσει τον κλασικό μηχανισμό κλήσης συναρτήσεων και επιστροφής αποτελεσμάτων. Για παράδειγμα, έστω ο ορισμός:

```
inline int max(int i, int j)
    { return (i > j ? i : j); }
```

τότε, η κλήση, π.χ. `max(5, 10)`, ανάλογα με την “εξυπνάδα” του μεταγλωττιστή, μπορεί να αντιμετωπιστεί σαν να ήταν η έκφραση `(5 > 10 ? 5 : 10)` ή `10` ή μπορεί και να μείνει ανέπαφη, όμως.

- Δεν είναι σίγουρο ότι κάθε `inline` συνάρτηση θα αντιμετωπιστεί `inline` (π.χ. αμοιβαία αναδρομικές συναρτήσεις, συναρτήσεις που καλούν τους εαυτούς τους ανάλογα με την είσοδό τους κλπ.)
- Η υπόδειξη `inline` δεν παραβιάζει την κλασική σημασία της κλήσης συνάρτησης. Άρα, δεν έχουμε τις παρενέργειες που προκύπτουν από την χρήση `#define` της C.
- Οι ορισμοί συναρτήσεων `inline` πρέπει να υπάρχουν σε κάθε αρχείο (compilation unit) στο οποίο χρησιμοποιούνται –έχουν εσωτερική συνδεσιμότητα.
- Οι συναρτήσεις `inline` και οι δηλώσεις `const` περιορίζουν τη χρήση `#define` στη C++.



## Υπερφόρτωση Ονομάτων Συναρτήσεων

- Χρήση του ίδιου ονόματος για διαφορετικές συναρτήσεις.
- Διάκριση από το πλήθος/τύπο των ορισμάτων τους.
- Ο τύπος που επιστρέφει η συνάρτηση δεν παίζει ρόλο για τη διάκριση.
- Υπάρχουν κανόνες που υποδεικνύουν ποιο είναι το καλύτερο ταίριασμα τύπων κατά την κλήση μιας συνάρτησης.
- π.χ.

```
void print(int);           // for int
void print(const char*);   // for strings
void print(double); ...
```

## Default Τιμές Ορισμάτων

- Για την περίπτωση που η γενική μορφή της κλήσης μιας συνάρτησης εμπλέκει περισσότερα ορίσματα από τη μορφή των συχνότερων κλήσεών της. Π.χ.

```
void print(int value, int base=10);
```

τότε

```
print(31)          ----->    31
print(31,10)       ----->    31
print(31,16)       ----->    1f
print(31,2)        ----->   11111
```

- Με χρήση εναλλακτικών ορισμών - επιτρεπτό, λόγω υπερφόρτωσης - πετυχαίνουμε το ίδιο αποτέλεσμα. Π.χ.

```
void print(int value, int base);
```

```
void print(int value);
```

```
// με ορισμό για εκτύπωση στο 10δικό
```

- Ο έλεγχος τύπων των default ορισμάτων γίνεται στο επίπεδο της δήλωσης ενώ η αποτίμηση γίνεται κατά την κλήση της συνάρτησης.
- Δεν επιτρέπονται default τιμές σε ενδιάμεσα ορίσματα. Π.χ.

```
OXI int g(int, int=0, int);
```

```
OYTE int g(int=0, int, int);
```

## Οι Τελεστές new και delete

Δέσμευση χώρου στη μνήμη:

**static μεταβλητές** Ο χώρος τους ανατίθεται κατά τη μεταγλώττιση του προγράμματος και διατηρείται καθ' όλη την εκτέλεσή του.

**automatic μεταβλητές** Ο χώρος τους ανατίθεται κάθε φορά που ο έλεγχος της εκτέλεσης φτάνει στον ορισμό τους και διατηρείται έως ότου ο έλεγχος βγει από το block μέσα στο οποίο υπάρχει ο ορισμός.

**δυναμική δέσμευση** Κατά την εκτέλεση ενός προγράμματος, για να δημιουργήσουμε αντικείμενα ρητά τα οποία καταλαμβάνουν κάποιο χώρο έως ότου ρητά να τον αποδεσμεύσουν, χρησιμοποιούμε τους τελεστές new και delete.

- με τον τελεστή new δημιουργούμε τέτοια αντικείμενα
- με τον τελεστή delete τα καταστρέφουμε
- δεν υπάρχει garbage collection για τέτοια αντικείμενα κι έτσι πρέπει να προσέχουμε να αποδεσμεύουμε το χώρο

## Παρατηρήσεις πάνω στη δυναμική δέσμευση και αποδέσμευση χώρου

- Ο τελεστής `new` επιστρέφει δείκτη σε αντικείμενο του τύπου που προσδιορίζουμε. Π.χ.

```
node* pn = new node;
```

Αν το `node` έχει οριστεί ως κλάση, κατά τη δέσμευση του χώρου στον σωρό, καλείται η συνάρτηση αρχικοποίησης της κλάσης.

- Ο τελεστής `delete` πρέπει να εφαρμόζεται σε δείκτες που “δείχνουν” σε αντικείμενα που δημιουργήθηκαν από εφαρμογή του τελεστή `new`, π.χ.,

```
delete pn;
```

Αν το `node` έχει οριστεί ως κλάση, καλείται η συνάρτηση καταστροφής της κλάσης.

- Μπορούμε να δημιουργήσουμε/καταστρέψουμε πίνακες από αντικείμενα χρησιμοποιώντας τους τελεστές `new` και `delete`, π.χ.,

```
char* s = new char[10];  
delete[] s;
```

Αν τα στοιχεία του πίνακα είναι αντικείμενα κλάσης, για καθένα από αυτά, καλείται η default συνάρτηση αρχικοποίησης της κλάσης.

- Για απλές μεταβλητές, δίνεται η δυνατότητα επιλογής της αρχικοποίησης κατά τη δυναμική δημιουργία, π.χ.,

```
node* pn = new node(10);
```

### Προσοχή:

`node* pn = new node(10);` η μεταβλητή `pn` είναι δείκτης σε μεταβλητή τύπου `node`, η οποία μεταβλητή αρχικοποιείται με χρήση της τιμής 10. Για να αποδεσμεύσουμε τον χώρο, αρκεί να κάνουμε: `delete pn;`

`node* pna = new node[10];` δημιουργία ενός πίνακα μεταβλητών τύπου `node`, αρχικοποιημένων by default, ο οποίος έχει μέγεθος 10. Για να αποδεσμεύσουμε τον χώρο, πρέπει να κάνουμε: `delete[] pna;`

## Τύπος Αναφοράς

- Γενική μορφή ορισμού μιας μεταβλητής τύπου αναφοράς:

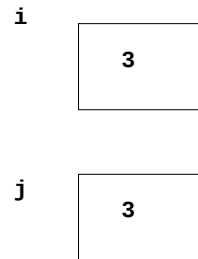
$\langle type \rangle \ \& \ \langle identifier2 \rangle = \langle identifier1 \rangle$

όπου η μεταβλητή  $\langle identifier1 \rangle$  πρέπει να έχει ήδη δηλωθεί νωρίτερα και να είναι τύπου  $\langle type \rangle$ .

- – Ο κώδικας:

```
int i; i = 3; int j = i;
```

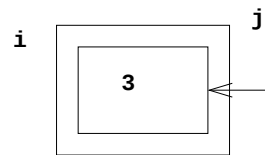
οδηγεί στην (αφαιρετική) εικόνα στη μνήμη:



- Ο κώδικας:

```
int i; i = 3; int& j = i;
```

ορίζει τη μεταβλητή  $j$  να είναι μια αναφορά στη μεταβλητή  $i$  (ένα “συνώνυμο” του χώρου που καταλαμβάνει). Η εικόνα στη μνήμη είναι:



- Οι αναφορές συμπεριφέρονται σαν σταθεροί δείκτες προς το αντικείμενο με το οποίο αρχικοποιούνται (άρα πάντα πρέπει να αρχικοποιούνται) και δεν χρειάζονται dereferencing.
- Προσοχή, ο τελεστής  $\&$  εξακολουθεί να χρησιμοποιείται και για να εκφράσουμε τη διεύθυνση μιας μεταβλητής.

## Τύπος Αναφοράς στα Ορίσματα Συνάρτησης

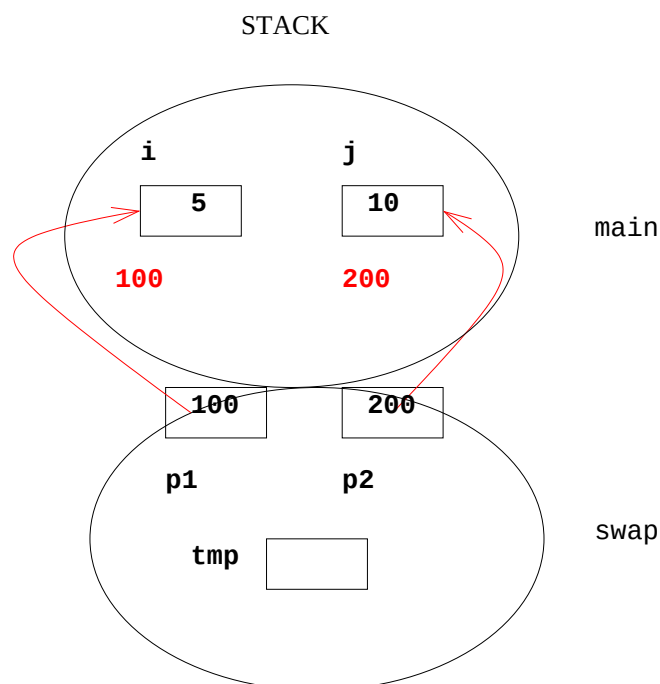
- Χρησιμοποιούμε τον τύπο αναφοράς για το πέρασμα παραμέτρων *κατά αναφορά*, δηλαδή, σε συναρτήσεις που αλλάζουν τις “εξωτερικές” μεταβλητές των ορισμάτων.
- Πέρασμα παραμέτρων à la C:

```
void swap(int* p1, int* p2)
{
    int tmp = *p1;
    *p1 = *p2;
    *p2 = tmp;
}
```

Κλήση:

```
int i, j; i = 5; j = 10;
swap(&i, &j);
```

Η εικόνα των μεταβλητών (αφαιρετικά) στη μνήμη:



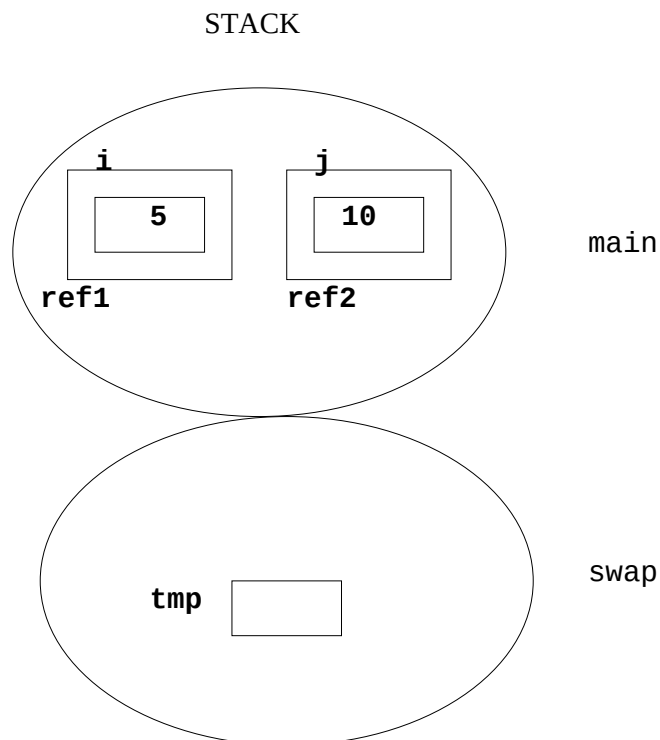
- Πέρασμα παραμέτρων à la C++

```
void swap(int& ref1, int& ref2)
{
    int tmp = ref1;
    ref1 = ref2;
    ref2 = tmp;
}
```

Κλήση:

```
int i, j; i = 5; j = 10;
swap(i, j);
```

Η εικόνα των μεταβλητών (αφαιρετικά) στη μνήμη:





- Μεγάλα αντικείμενα καλό είναι να τα περνάμε μέσω αναφοράς, για λόγους αποδοτικότητας.
- Ακόμα κι αν δεν έχουμε πρόθεση να αλλάξουμε την τιμή τους, το παραπάνω ισχύει. Σε αυτήν την περίπτωση όμως, καλό είναι να περιορίζουμε την αναφορά με `const`.

Π.χ.

```
void f(const tree& t)
{...}
```

οπότε, στο σώμα της `f` απαγορεύεται να αλλάξει η τιμή του `t`.

- Τα arrays περνάνε αυτόματα μέσω αναφορών.

## ΕΜΒΕΛΕΙΑ (scope)

### Ονόματα

- για τις μεταβλητές
- για τις συναρτήσεις
- για τους τύπους
- ...

## Δηλώσεις - Ορισμοί - Αναθέσεις αρχικών τιμών μεταβλητών

**Δηλώσεις** Μια δήλωση αποτελεί μια περιγραφή που συνδέει ένα όνομα μεταβλητής με έναν τύπο, π.χ.

```
extern int error_number;
```

**Ορισμοί** Με έναν ορισμό γίνεται δέσμευση ενός ονόματος με μια συγκεκριμένη οντότητα, π.χ.

```
char ch;
```

Ένας ορισμός αποτελεί ταυτόχρονα και δήλωση.

**Ανάθεση αρχικών τιμών** Με τον τρόπο αυτόν, γίνεται ανάθεση αρχικών τιμών στην οντότητα στην οποία αναφέρεται ένα όνομα, π.χ.

```
int count = 1;
```

```
const double pi = 3.141592;
```

Η ανάθεση αρχικών τιμών γίνεται σε μια έκφραση ορισμού μιας μεταβλητής, άρα αποτελεί και δήλωσή της.

## Παραδείγματα Δηλώσεων και Ορισμών

```
char ch; <---- DEFINITION
```

```
string s; <---- DEFINITION
```

```
int count = 1;
```

```
const double pi = 3.1415;
```

```
extern int ext_number; <---- DECLARATION ONLY !!!!!
```

```
const char* Name = "TheName";
```

```
const char* season[] =  
    {"anixi", "kalokeri", "fthinoporo", "ximonas"};
```

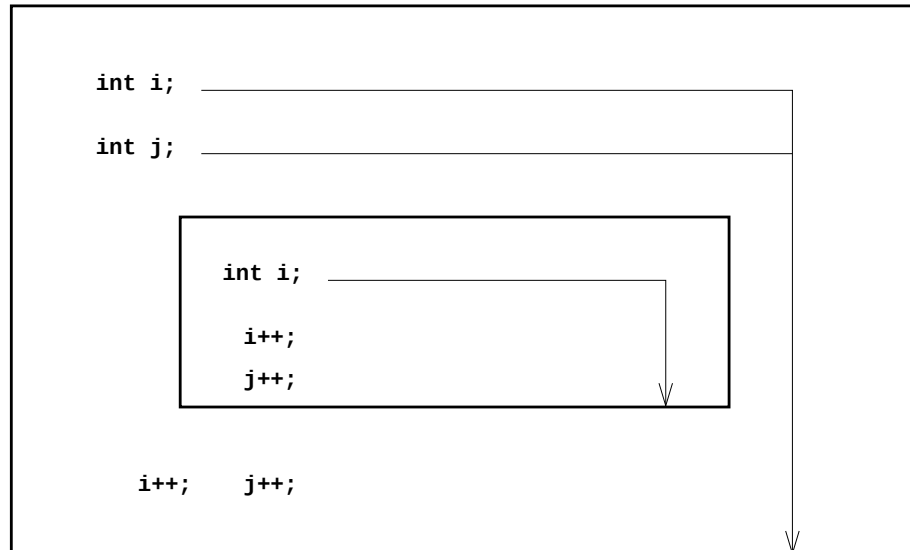
```
int day(Date* pd) { return pd -> d; } <---- DEFINITION
```

```
double sqrt(double); <---- DECLARATION ONLY !!!!!
```

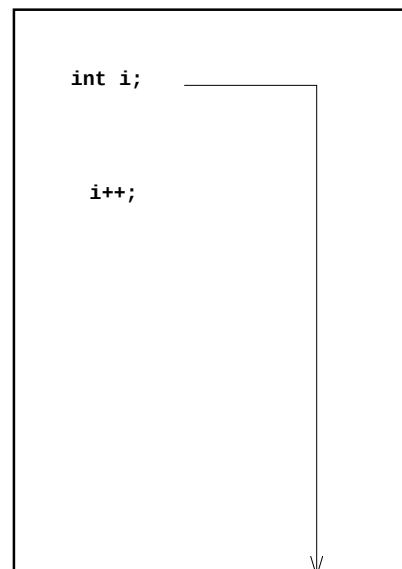
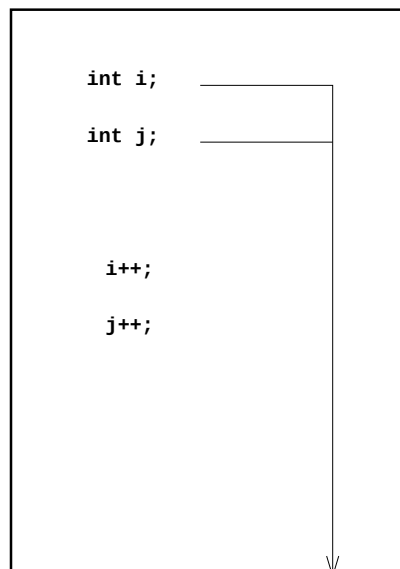
## Εμβέλεια (Δηλώσεων) Ονομάτων

- Μια δήλωση εισάγει κάποιο όνομα μέσα σε ένα περιβάλλον, μια εμβέλεια.
- Η εμβέλεια ενός ονόματος ξεκινά από τη δήλωση του ονόματος.
- Ο γενικός τρόπος να ορίσουμε μια εμβέλεια είναι με τη χρήση αγκύλων,  $\{ \}$  —δηλώσεις ονομάτων μέσα σε αγκύλες ισχύουν μόνο μέσα σε αυτές.
- Μπορούμε να έχουμε εμφωλευμένες αγκύλες, οπότε στις εσωτερικές ισχύουν οι δηλώσεις των εξωτερικών, εκτός εάν τα ονόματα έχουν αποκτήσει άλλο νόημα μέσω νέων δηλώσεων.

- Εμφωλευμένα περιβάλλοντα:



- Διακριτά περιβάλλοντα:



- Όσο πιο μεγάλο το πρόγραμμα που γράφουμε, τόσο πιο πολλές οντότητες θα ορίσουμε, άρα τόσο πιο πολλά είναι και τα ονόματα που θα χρειαστούμε.
- Με τη χρήση εμβελειών βάζουμε σε τάξη και οργάνωση τα ονόματα του προγράμματός μας –διευκολύνει την ανάπτυξη κώδικα ακόμα περισσότερο.
- Οι “αγκύλες” είναι το πιο “κρίσιμο εργαλείο” που προσφέρει η C++ κατά τον ίδιο τον Stroustrup.
- Χώροι ονομάτων (namespaces).
- Κλάσεις κ.ά.
- Τελεστής :: (scope resolution operator): για επίλυση εμβέλειας.

## Γενικότερες Παρατηρήσεις

### Δηλώσεις και Ορισμοί

- Κάθε όνομα προτού να χρησιμοποιηθεί πρέπει να έχει δηλωθεί.
- Σε μια δήλωση, συνδέουμε ένα όνομα με το είδος της οντότητας την οποία ονομάζει το όνομα.
- Σε έναν ορισμό δημιουργούμε την οντότητα –στην περίπτωση μεταβλητών και συναρτήσεων δεσμεύουμε το χώρο στη μνήμη.
- Ένας ορισμός αποτελεί ταυτόχρονα και δήλωση.
- Σε μία εμβέλεια πρέπει, για κάθε όνομα να έχουμε έναν ακριβώς ορισμό –εκτός αν το όνομα αναφέρεται σε οντότητες διαφορετικού είδους, π.χ. χρήση του ίδιου ονόματος για έναν τύπο και μια μεταβλητή, ή εμπίπτει στην περίπτωση όπου επιτρέπεται από τη γλώσσα ρητά η πολλαπλή χρήση του, όπως κατά την υπερφόρτωση ονομάτων συναρτήσεων.



## Μεταβλητές (Εμβέλεια VS Lifetime)

### Καθολικές μεταβλητές:

- Ορίζονται εκτός σωμάτων συναρτήσεων.
- Είναι, εν δυνάμει, ορατές και χρησιμοποιήσιμες από όλο το πρόγραμμα.
- Έχουν χρόνο ζωής τη διάρκεια εκτέλεσης του προγράμματος.
- Έχουν εξωτερική συνδεσιμότητα.
- Κατά την εκτέλεση του προγράμματος φυλάσσονται στον στατικό χώρο της μνήμης.

### Στατικές εξωτερικές:

- Είναι καθολικές μεταβλητές οι οποίες προσδιορίζονται ως `static`.
- Η εμβέλειά μιας τέτοιας μεταβλητής περιορίζεται έως το τέλος του αρχείου μέσα στο οποίο ορίζεται –άρα και δηλώνεται.
- Ο χρόνος ζωής της είναι όλη η διάρκεια εκτέλεσης του προγράμματος.
- Οι μεταβλητές αυτές έχουν εσωτερική συνδεσιμότητα.
- Κατά την εκτέλεση του προγράμματος φυλάσσονται στον στατικό χώρο της μνήμης.

## Στατικές τοπικές σε συνάρτηση:

- Ορίζονται (με ταυτόχρονη αρχικοποίηση) μέσα στο σώμα μιας συνάρτησης και έχουν προσδιοριστεί ως `static`.
- Η εμβέλεια της δήλωσης του ονόματός μιας τέτοιας μεταβλητής περιορίζεται μέσα στο σώμα της συνάρτησης και μόνο.
- Ο χρόνος ζωής τους είναι όλη η διάρκεια εκτέλεσης του προγράμματος.
- Φυλάσσονται στον στατικό χώρο, οπότε η ύπαρξή τους είναι ανεξάρτητη από το εάν το πλαίσιο (frame) της συνάρτησης βρίσκεται στη στοίβα εκτέλεσης (μεταβαλλόμενος χώρος) ή όχι — εξ ου και πολλές φορές τις περιγράφουμε ως μεταβλητές “που διατηρούν την τιμή τους μεταξύ των κλήσεων της συνάρτησης”.

## Τοπικές σε συνάρτηση:

- Είναι οι κλασικές αυτόματες μεταβλητές που ορίζονται στο σώμα μιας συνάρτησης.
- Η εμβέλεια της δήλωσης του ονόματός τους περιορίζεται μέσα στο σώμα της συνάρτησης.
- Και ο χρόνος ζωής τους σταματά με τη λήξη της εκτέλεσης του κώδικα του σώματος της συνάρτησης.

### Τοπικές σε block { }:

- Είναι μεταβλητές που ορίζονται μέσα σε ένα block.
- Η εμβέλεια της δήλωσής τους περιορίζεται από το block.
- Ο χρόνος ζωής τους περιορίζεται από την εκτέλεση του block.

### Στατικές τοπικές σε block:

- Είναι μια μεταβλητή που ορίζεται μέσα σε ένα block και έχει προσδιοριστεί ως `static`.
- Ο χρόνος ζωής της είναι όλη η διάρκεια της εκτέλεσης του προγράμματος.
- Ισχύει ό,τι και για την περίπτωση των μεταβλητών που είναι στατικές τοπικές σε συνάρτηση.

## Χώροι Ονομάτων (Namespaces)

- Ένας χώρος ονομάτων αποτελεί μια εμβέλεια με όνομα.
- Σε έναν χώρο ονομάτων “πακετάρουμε” δηλώσεις σε μια εμβέλεια και στο “πακέτο” δίνουμε ένα όνομα.
- Παράδειγμα:

```
namespace Geometry {
    const double PI = 3.1415;
    double L = 1.0; // Don't include definitions,
                    // in the general case
    double square_surface(double);
    double rectang_surface(double, double);
    double circle_surface(double);
}
```

- Χρήση του τελεστή επίλυσης εμβέλειας, `::` για να επιλέξουμε ένα από τα ονόματα ενός χώρου ονομάτων εκτός του χώρου, π.χ. η έκφραση:  
`exPI = Geometry::PI;`  
 κάνει ανάθεση στη μεταβλητή `exPI` της τρέχουσας εμβέλειας την τιμή που έχει η σταθερά `PI` που έχει οριστεί στον χώρο ονομάτων `Geometry`.
- Παρατήρηση: στα σώματα των συναρτήσεων που δηλώνονται σε έναν χώρο ονομάτων βρισκόμαστε στην εμβέλεια του χώρου αυτού.
- Προσοχή: Μέσα στους χώρους ονομάτων, ασφαλές είναι να γράφουμε δηλώσεις (και ορισμούς μεταβλητών και συναρτήσεων με εσωτερική συνδεσιμότητα).
- Πολλές δυνατότητες σχετικά με τους χώρους ονομάτων.

## Απαλλαγή από τη χρήση του ονόματος του χώρου ονομάτων

- Τα ονόματα ενός χώρου ονομάτων γίνονται άμεσα διαθέσιμα έξω από τον χώρο με τη χρήση της έκφρασης `using namespace` –πλέον δεν χρειάζεται να χρησιμοποιηθεί ο τελεστής επίλυσης εμβέλειας (`::`) για να έχουμε πρόσβαση στα ονόματα αυτά.

```
double Var1 = Geometry::PI;
```

```
using namespace Geometry;
```

```
double Var2 = PI;
```

- Η επίδραση του `using` ισχύει έως το τέλος της τρέχουσας εμβέλειας (δηλαδή, τέλος του τρέχοντος block ή τέλος του αρχείου αν το `using` εμφανίζεται έξω από block).
- Δεν πρέπει να χρησιμοποιούμε `using` μέσα σε αρχεία-επικεφαλίδες διότι είναι πολύ επικίνδυνο.
- Το `using` μπορεί να αναφέρεται σε ολόκληρο τον χώρο ονομάτων ή μόνο σε κάποια από τα ονόματα που δηλώνει.

## Ανώνυμοι Χώροι Ονομάτων

- Χώροι ονομάτων χωρίς όνομα, για τη δημιουργία “τοπικών” “κλειστών” χώρων ονομάτων.
- Ο ορισμός ενός ανώνυμου χώρου ονομάτων κάνει όλα τα ονόματα που δηλώνει άμεσα ορατά έως το τέλος του αρχείου μέσα στο οποίο ορίζεται ο χώρος και μόνο.
- Απαλλάσσει από το να προσδιορίσουμε ονόματα ως static για να περιορίσουμε τη χρήση τους σε ένα αρχείο –απαλλασσόμαστε από τη χρήση του προσδιορισμού static για θέματα που αφορούν εμβέλεια, οπότε μπορούμε να περιορίσουμε τη χρήση του σε περιπτώσεις που αφορούν χρόνο ζωής μεταβλητών.

## Επιπλέον Παρατηρήσεις σχετικά με τους Χώρους Ονομάτων

### Οργάνωση Προγράμματος που περιέχει Χώρους Ονομάτων

- Ορίζουμε τον χώρο ονομάτων σε ένα αρχείο επικεφαλίδας.
- Γράφουμε τους ορισμούς για τα ονόματα που δηλώνει σε ένα αρχείο πηγαίου κώδικα –στο αρχείο αυτό συμπεριλαμβάνουμε το αρχείο επικεφαλίδας που περιέχει τον ορισμό του χώρου ονομάτων.
- Το αρχείο επικεφαλίδας πρέπει να συμπεριληφθεί σε κάθε αρχείο στο οποίο θέλουμε να χρησιμοποιηθούν ονόματα από τον χώρο ονομάτων –γίνεται σαφές, πλέον, γιατί ο ορισμός ενός χώρου ονομάτων πρέπει να περιέχει μόνο δηλώσεις και ορισμούς με εσωτερική συνδεσιμότητα.

## Ο χώρος ονομάτων της καθιερωμένης βιβλιοθήκης

- Τα ονόματά της καθιερωμένης βιβλιοθήκης της C++ είναι δηλωμένα σε έναν χώρο ονομάτων, τον `std`.
- Όποτε πρόκειται να χρησιμοποιήσουμε ονόματα από την καθιερωμένη βιβλιοθήκη είτε τα κάνουμε (άμεσα) ορατά μέσω της έκφρασης `using namespace std`; είτε προσδιορίζουμε την εμβέλεια με χρήση του τελεστή `::`, π.χ. `std::cout`.
- Παράδειγμα:

```
#include <iostream>
```

```
int main()
{ std::cout << "Printing!\n";}
```

Η αλλιώς:

```
#include <iostream>
using namespace std;
```

```
int main()
{cout << "Printing!\n";}
```



## ΚΛΑΣΕΙΣ

- Ορισμός από το χρήστη νέων τύπων, η πρόσβαση στα στοιχεία των οποίων γίνεται μέσω συγκεκριμένων συναρτήσεων.
- Ένας τύπος είναι η φυσική αναπαράσταση μιας ιδεατής οντότητας. Π.χ. ο τύπος `float` με `+`, `-`, `*` ... είναι η αναπαράσταση του πραγματικού αριθμού των μαθηματικών.
- Ένα πρόγραμμα που χρησιμοποιεί τύπους που προσομοιώνουν τις οντότητες της εφαρμογής γίνεται ευκολότερα κατανοητό.
- Διαχωρισμός των λεπτομερειών υλοποίησης του τύπου από τις ιδιότητες/λειτουργίες του τύπου που είναι απαραίτητες για τη χρήση του.
- Οι νέοι τύποι που ορίζει ο προγραμματιστής στον αντικειμενοστραφή προγραμματισμό μοντελοποιούνται ως κλάσεις.
- Οι κλάσεις μπορούν να λειτουργήσουν και ως εργαλείο για να πετύχουμε *τμηματοποίηση* του κώδικά μας με *απόκρυψη πληροφορίας*, ώστε να έχουμε καλύτερη οργάνωση του προγράμματός μας.

## Κλάσεις στη C++

- Η θεμελιώδης έννοια του τύπου που ορίζεται από το χρήστη στη C++ είναι αυτή της *κλάσης* (class).
- Η πρόσβαση στα αντικείμενα μιας κλάσης γίνεται μέσω των *συναρτήσεων-μελών* (member functions) που είναι συνδεδεμένες με την κλάση.
- Κάθε αντικείμενο που δημιουργείται “κτίζεται” από όλα τα *μέλη-δεδομένα* (data members) της κλάσης, δηλαδή τις μεταβλητές που περιγράφονται στον ορισμό μιας κλάσης (εκτός των στατικών).
- Κατά τη δημιουργία αντικειμένων μιας κλάσης γίνεται κλήση συγκεκριμένων συναρτήσεων που (ενδεχόμενα δηλώνονται στη κλάση και) λέγονται *συναρτήσεις αρχικοποίησης/κατασκευής* (constructors).
- Κατ’ αναλογία, *συναρτήσεις καταστροφής* (destructors) καλούνται όταν πρόκειται να γίνει καταστροφή των αντικειμένων των κλάσεων (απελευθέρωση του χώρου που τους έχει ανατεθεί).
- Ένα αντικείμενο μιας κλάσης μπορεί να δημιουργηθεί όπως μια οποιαδήποτε άλλη μεταβλητή της C++ (σαν αυτόματη μεταβλητή, σαν στατική, είτε δυναμικά στον σωρό).

## Παραλλαγή του Παραδείγματος των Πτήσεων:

| <i>Αεροπορική Πτήση</i> |                       |
|-------------------------|-----------------------|
| Αριθμός Πτήσης          | <i>ακέραιος</i>       |
| Πόλη Αφετηρίας          | <i>συμβολοσειρά</i>   |
| Πόλη Αφίξης             | <i>συμβολοσειρά</i>   |
| Ώρα Αναχώρησης          | <i>Χρόνος</i>         |
| Διάρκεια                | <i>Χρόνος</i>         |
| Ημερομηνία Πτήσης       | <i>Ημερομηνία (;)</i> |

### *Ημερομηνία:*

- Οντότητα που εκφράζει την ημερομηνία.
- Την αντιλαμβανόμαστε ως τρεις ακεραίους αριθμούς: ημέρα, μήνα και έτος.
- Συμπεριφορά:
  - δυνατότητα ανάθεσης τιμών για την ημέρα, μήνα και έτος μιας ημερομηνίας, συνολικά και
  - δυνατότητα ταυτόχρονης ανάκτησης των παραπάνω τιμών.

## Ο Νέος Τύπος à la C

### Ορισμός του Τύπου:

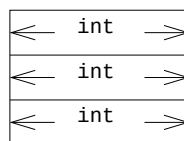
```
struct date { int day; int month; int year; };
```

### Ορισμός Μεταβλητών του Τύπου:

```
...
date today;
date another_day;
..
```

Η εικόνα των μεταβλητών:

today

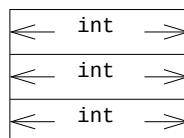


today.day

today.month

today.year

another\_day



another\_day.day

another\_day.month

another\_day.year

### *Ορισμός των Συναρτήσεων της Συμπεριφοράς:*

```
void set_date(date* dt, int d, int m, int y) {
    dt->day = d; dt->month = m; dt->year = y;
}
```

```
void get_date(date* dt, int* d, int* m, int* y) {
    *d = dt->day; *m = dt->month; *y = dt->year;
}
```

### *Χρήση των Μεταβλητών μέσω των Συναρτήσεων:*

```
set_date(&today, 8, 12, 2025);
...
```

Παρατήρηση: Έστω, ότι έχουμε και μια συνάρτηση:

```
void bla(date*, ...);
```

Από συντακτική απόψη, δεν γίνεται διάκριση των συναρτήσεων της συμπεριφοράς του τύπου `date` από τη συνάρτηση `bla`.

## Ο Νέος Τύπος à la C++

### Ένας Ορισμός του Τύπου

```
struct date {
    int day, month, year;
    void set_date(int, int, int);
    void get_date(int*, int*, int*);
};
```

- Οι συναρτήσεις που υλοποιούν τη συμπεριφορά του τύπου δηλώνονται μέσα στον ορισμό του τύπου – δήλωση συναρτήσεων-μελών.
- Υπονοείται ότι εφαρμόζονται σε μια μεταβλητή του τύπου, η οποία δεν αναφέρεται ρητά – είναι απαλλαγμένες από το όρισμα που αντιστοιχεί στη μεταβλητή του τύπου.
- Οι μεταβλητές που δηλώνονται στον ορισμό του τύπου αποτελούν τα *μέλη-δεδομένα* που “χτίζουν” τις μεταβλητές του τύπου.
- Κατά τον ορισμό ενός τύπου ορίζεται και μια εμβέλεια με το όνομά του.

## Συναρτήσεις-μέλη

### Χρήση των Συναρτήσεων-μελών

- Κατά τη χρήση τους, οι συναρτήσεις-μέλη εφαρμόζονται απαραίτητα σε μια μεταβλητή του τύπου.
- Η σύνταξη της κλήσης μιας συνάρτησης-μέλους ακολουθεί τη σύνταξη της χρήσης ενός πεδίου μιας μεταβλητής που είναι τύπου που έχει οριστεί μέσω `struct`: η μεταβλητή γράφεται αριστερά από την κλήση διαχωρίζοντάς την με τελεία, ., π.χ.

### Κλήση στη C:

```
set_date(&today,8,12,2025);
```

### Κλήση στη C++:

```
today.set_date(8,12,2025);
```

## Νέος Ορισμός του Τύπου

```
struct date {  
    int day, month, year;  
    void set(int, int, int);  
    void get(int&, int&, int&);  
};
```

- Ονοματίσαμε τις συναρτήσεις `set_date` και `get_date` με τα ονόματα `set` και `get`, αντίστοιχα, μια που έχουν μοναδικό νόημα στην εμβέλεια `date`.
- (Δευτερεύον) Για πέρασμα παραμέτρων κατά αναφορά, κάνουμε χρήση του τύπου αναφοράς.



## Ορισμός των Συναρτήσεων-μελών

```
void date::set(int i, int j, int k)
{
    day = i;
    month = j;
    year = k;
}
```

```
void date::get(int& i, int& j, int& k)
{
    i = day;
    j = month;
    k = year;
}
```

- Χρησιμοποιήσαμε τον τελεστή επίλυσης εμβέλειας, ::, για να προσδιορίσουμε τον τύπο και να κάνουμε τη σύνδεση με αυτόν.
- Στο σώμα των συναρτήσεων-μελών, συνήθως, αναφερόμαστε στην μεταβλητή που εφαρμόζονται μέσω των πεδίων της. Αν θέλαμε να αναφερθούμε σε ολόκληρη τη μεταβλητή που εφαρμόζεται μια συνάρτηση-μέλος, μπορούμε να χρησιμοποιήσουμε την δεσμευμένη λέξη `this` η οποία ονοματίζει ένα όρισμα τύπου δείκτη του τύπου του οποίου είναι μέλος η συνάρτηση –αυτό που “ξεφορτωθήκαμε” .

## Απόκρυψη Πληροφορίας

- Συνήθως, η αναπαράσταση των μεταβλητών ενός νέου τύπου επιθυμούμε να μείνει “κρυφή” εντός του τύπου και να μην επιτρέπεται να χρησιμοποιηθούν τα ονόματα των μελών-δεδομένων εκτός της εμβέλειας του τύπου.
- Αντίθετα, τα ονόματα των συναρτήσεων-μελών που αντιστοιχούν στην συμπεριφορά του θα πρέπει να είναι “ορατά” και χρησιμοποιήσιμα κι εκτός της εμβέλειας του τύπου.
- Την διάκριση μεταξύ “ιδιωτικών” και “δημόσιων” ονομάτων επιτυγχάνουμε προσδιορίζοντάς τα ως `private` και `public`, αντίστοιχα.
- Τα ονόματα όλων των δηλώσεων που ακολουθούν τον προσδιορισμό πρόσβασης/ορατότητας (`access specifier`) `private` μπορούν να χρησιμοποιηθούν μόνο μέσα στην εμβέλεια του τύπου –έτσι έχουμε απόκρυψη πληροφορίας

## Ολοκληρωμένος ορισμός και χρήση του νέου τύπου date

```
#include <iostream>
using namespace std;

struct date {
    void set(int, int, int);
    void get(int&, int&, int&);
private:
    int day, month, year;
};

void date::set(int i, int j, int k)
{
    day = i;
    month = j;
    year = k;
}

void date::get(int& i, int& j, int& k)
{
    i = day;
    j = month;
    k = year;
}
```

```
int main()
{
    date today;
    int d, m, y;

    today.set(31,12,1999);

    // cout << today.month << endl;
    // If the above is uncommented, an error occurs

    today.get(d,m,y);

    cout << "day " << d << ", month " << m
         << ", year " << y << endl;

    return 0;
}
```

Ο κώδικας αναδεικνύει:

- τον ορισμό νέου τύπου, τον `date`, με χρήση `struct`,
- τη σύνδεση των συναρτήσεων που υλοποιούν τη συμπεριφορά του με τον τύπο, δηλώνοντάς τις ως συναρτήσεις-μέλη του τύπου,
- τη διάκριση ορατότητας ονομάτων των μελών του σε δημόσια και ιδιωτικά,
- τον ορισμό των συναρτήσεων-μελών εκτός του τύπου και χρήση του τελεστή επίλυσης εμβέλειας για συσχέτισή τους με τον τύπο,
- τον ορισμό των συναρτήσεων-μελών και την πρόσβαση στα μέλη της μεταβλητής του τύπου στην οποία εφαρμόζονται,
- τον ορισμό μεταβλητών του τύπου και
- την εφαρμογή συναρτήσεων-μελών σε μεταβλητές του τύπου (και χρήση μελών γενικότερα, στην περίπτωση που τα ονόματά τους είναι ορατά και χρησιμοποιήσιμα εκτός του τύπου).

## Αφαίρεση στα δεδομένα - class αντί για struct

- Χρήση της γλωσσικής δομής `class`, αντί για την `struct`, προκειμένου να ορίσουμε κλάσεις.
- Για τη C++ και οι δύο τρόποι προσφέρουν αντίστοιχη λειτουργικότητα — προτείνουμε τον περιορισμό της χρήσης του `struct` για ορισμό τύπων που αντιστοιχούν σε δομές δεδομένων, αντίστοιχα με τη γλώσσα C.
- Για τον ορισμό τύπων που αντιστοιχούν σε Αφηρημένους Τύπους Δεδομένων και υλοποιούν κλάσεις, προτείνεται η χρήση της γλωσσικής δομής `class`.

Γενική δομή κλάσης:

```
class< όνομα κλάσης >
{
    <δηλώσεις_1>
```

```
public:
    <δηλώσεις_2>
};
```

Ή αλλιώς:

```
class< όνομα κλάσης >
{
public:
    <δηλώσεις_2>
```

```
private:
    <δηλώσεις_1>
};
```

Το παράδειγμα του νέου τύπου `date`, ορισμένο σαν κλάση

```
class date {  
    public:  
        void set(int, int, int);  
        void get(int&, int&, int&) const;  
    private:  
        int day, month, year;  
};
```

```
void date::set(int i, int j, int k)  
{  
    day = i;  
    month = j;  
    year = k;  
}
```

```
void date::get(int& i, int& j, int& k) const  
{  
    i = day;  
    j = month;  
    k = year;  
}
```



## Παρατηρήσεις για τα μέλη μιας κλάσης

- Τα ονόματα του ιδιωτικού μέρους της κλάσης (ονόματα που οι δηλώσεις τους ακολουθούν τον προσδιορισμό `private`), μπορούν να χρησιμοποιηθούν μόνο από τις συναρτήσεις-μέλη της κλάσης –γενικότερα, μόνο από την εμβέλεια της κλάσης. (Σημείωση: υπάρχει κι άλλη μια δυνατότητα, μέσω φιλικών συναρτήσεων και φιλικών κλάσεων, `friends`)
- Το δημόσιο μέρος μιας κλάσης αποτελεί τη διεπαφή (`interface`) της κλάσης μέσω της οποίας χειριζόμαστε τα αντικείμενα της κλάσης έξω από την κλάση. Στο δημόσιο μέρος ανήκουν τα ονόματα που οι δηλώσεις τους ακολουθούν τον προσδιορισμό `public`.
- Τα μέλη μιας `struct` είναι by default δημόσια ενώ τα μέλη μιας `class` είναι by default ιδιωτικά.
- Οι συναρτήσεις-μέλη καθώς και τα μέλη-δεδομένα δηλώνονται, καλούνται και ορίζονται ακριβώς όπως και στη `struct`.
- Η προσπέλαση στα μέλη μιας `class` γίνεται όπως και σε εκείνα μιας `struct` (με τη χρήση των τελεστών `.` και `->`, ανάλογα αν έχουμε μεταβλητή του τύπου ή δείκτη σε μεταβλητή του τύπου)

Ορίζοντας μια κλάση πρέπει να δηλώσουμε, κατ' αρχήν, τα εξής:

- τα δομικά στοιχεία που κτίζουν κάθε αντικείμενο της κλάσης –άτυπα, “τουβλάκια” (building blocks) των αντικειμένων της κλάσης,
- μια σειρά από συναρτήσεις-μέλη που αναθέτουν/μεταβάλλουν τις τιμές στα δομικά στοιχεία των αντικειμένων, οπότε είναι μεταβάλλουσες (mutators),
- μια σειρά από συναρτήσεις-μέλη που αναχτούν τις τιμές αυτές, τις αναγνώστριες (accessors) –στην περίπτωση αυτή καλό είναι να τις προσδιορίζουμε ως σταθερές με χρήση της λέξης-κλειδιού `const` στο τέλος του πρωτοτύπου τους.

Παράδειγμα του νέου τύπου date, ορισμένο ως κλάση και περιπτώσεις χρήσης του

```
#include <iostream>
using namespace std;

class date {
public:
    void set(int, int, int);
    void get(int&, int&, int&) const;
private:
    int day, month, year;
};

void date::set(int i, int j, int k)
{
    day = i;
    month = j;
    year = k;
}

void date::get(int& i, int& j, int& k) const
{
    i = day;
    j = month;
    k = year;
}
```

```

date TODAY;                // Global variable

int main()
{
    date someday;          // Local variable

    int d, m, y;

    someday.set(18,3,2005);
    someday.get(d,m,y);
    cout << "day " << d
         << ", month " << m << ", year " << y << endl;

    date nextday;          // Another instance
    nextday.set(19,3,2005);
    nextday.get(d,m,y);
    cout << "day " << d
         << ", month " << m << ", year " << y << endl;

    //////////////////////////////////////

    TODAY.set(19,11,2019);
    TODAY.get(d,m,y);
    cout << "day " << d
         << ", month " << m << ", year " << y << endl;

```

```
////////////////////////////////////
```

```
date* psomeday = &someday; // Pointer to local date
```

```
cout << "The size of a variable of type int "
```

```
    << sizeof(int) << endl;
```

```
cout << "The size of a variable of type date "
```

```
    << sizeof(someday) << endl;
```

```
cout << "The size of a variable of type pointer to date "
```

```
    << sizeof(psome day) << endl;
```

```
psomeday->get(d,m,y);
```

```
cout << "day " << d
```

```
    << ", month " << m << ", year " << y << endl;
```

```
(*psomeday).get(d,m,y);
```

```
cout << "day " << d
```

```
    << ", month " << m << ", year " << y << endl;
```

```
date* pbreakdate = new date(); // Pointer to date in heap
```

```
                        // (dynamic allocation)
```

```
pbreakdate->set(23,12,2019);
```

```
pbreakdate->get(d,m,y);
```

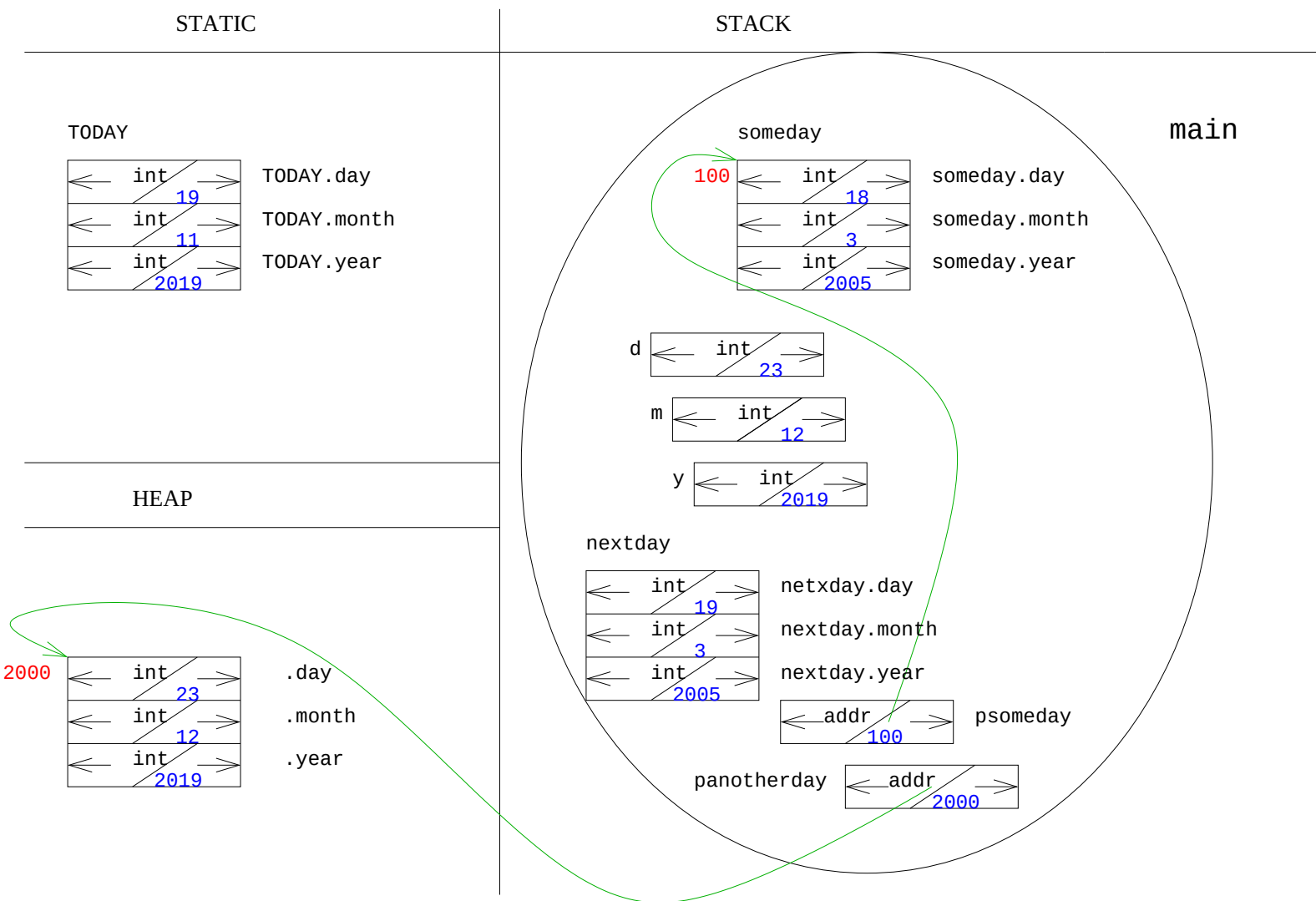
```
cout << "day " << d
```

```
    << ", month " << m << ", year " << y << endl;
```

```
delete pbreakdate;
```

```
}
```

Η εικόνα της μνήμης πριν την εκτέλεση της εντολής delete pbreakdate;



## Ο δείκτης `this`

Τι γίνεται αν η πρόσβαση μέσα από το σώμα μιας συνάρτησης-μέλους στο αντικείμενο πάνω στο οποίο εφαρμόζεται δεν επαρκεί ή δεν είναι κατάλληλο να γίνει μέσω των ονομάτων των μελών-δεδομένων;

Απάντηση: Χρήση του δείκτη `this`

- Οι συναρτήσεις-μέλη δουλεύουν σαν να έχουν κρυμμένο ένα όρισμα που αναφέρεται σε δείκτη στο αντικείμενο για το οποίο καλούνται. Δηλαδή, για μια κλάση `X`, ο δείκτης μπορεί να θεωρηθεί ότι είναι τύπου `X *const this` (το `this` υποδηλώνει τον δείκτη στο αντικείμενο για το οποίο καλείται η συνάρτηση).
- Η χρήση του `this` έχει νόημα:
  - όταν μια συνάρτηση-μέλος παίρνει σαν όρισμα μεταβλητές της ίδιας της κλάσης (π.χ. `void append(list*)`; για συνένωση λιστών), για ευκρίνεια στον κώδικα
  - υπάρχει ανάγκη για πρόσβαση στο δείκτη ή/και σε ολόκληρο το αντικείμενο και χειρισμού του μέσα από το σώμα της συνάρτησης
  - υπάρχει ανάγκη επιστροφής της τιμής του αντικειμένου ή της διεύθυνσής του από τη συνάρτηση
- Σε περίπτωση που μια συνάρτηση-μέλος δεν πρόκειται να τροποποιήσει το αντικείμενο για το οποίο καλείται, τότε στη δήλωσή της συμπεριλαμβάνουμε τη λέξη-κλειδί `const`, π.χ. `char* get_name() const`; . Σ' αυτήν την περίπτωση, ο τύπος του `this` μπορεί να θεωρηθεί ότι είναι για μια κλάση `X`, `const X *const this`

## Το Παράδειγμα του “Φοιτητή”

### Φοιτητής:

- Οντότητα που εκφράζει την έννοια του φοιτητή.
- Ενδιαφερόμαστε για το ονοματεπώνυμό του, τον αριθμό μητρώου του και το έτος σπουδών του.
- Συμπεριφορά:
  - δυνατότητα ανάθεσης τιμών για το ονοματεπώνυμο, τον αριθμό μητρώου και το έτος σπουδών, κατά περίπτωση και
  - δυνατότητα ανάκτησης των παραπάνω τιμών, πάλι κατά περίπτωση.

### Υλοποίηση σαν κλάση Student. Αποφάσεις για:

- την αναπαράσταση των αντικειμένων της κλάσης και
- την υλοποίηση των συναρτήσεων-μελών, με δεδομένη την αναπαράσταση που έχει αποφασιστεί.



## Αναπαράσταση αντικειμένων

**name:** μια συμβολοσειρά που αντιστοιχεί στο ονοματεπώνυμο του φοιτητή,

**no:** ένας αριθμός που αντιστοιχεί στον αριθμό μητρώου του φοιτητή και

**year\_of\_studies:** ένας αριθμός που αντιστοιχεί στο έτος σπουδών του φοιτητή.

## Ο Ορισμός της Κλάσης Student

```
class Student {
private:
    char* name;           // +
    int no;               // | Building blocks
    int year_of_studies; // +
public:
    void set_name(const char*); // +
    void set_no(int);           // | Mutators
    void set_year_of_studies(int); // +
    char* get_name() const;     // +
    int get_no() const;         // | Accessors
    int get_year_of_studies() const; // +
};
```

## Ορισμός των συναρτήσεων-μελών της Κλάσης

```
void Student::set_name(const char* nam)
{
    name = new char[strlen(nam)+1];
    strcpy(name,nam);
}
```

```
void Student::set_no(int n)
{
    no = n;
}
```

```
void Student::set_year_of_studies(int y)
{
    year_of_studies = y;
}
```

```
char* Student::get_name() const
{
    return name;
}
```

```
int Student::get_no() const
{
    return no;
}
```

```
int Student::get_year_of_studies() const
{
    return year_of_studies;
}
```

## Ο νέος τύπος Student: δομή αρχείων προγράμματος

- Ορισμός νέου τύπου:
  - `student.h`: ορισμός του νέου τύπου (αρχείο -επικεφαλίδα)
  - `student.cpp`: ορισμός των συναρτήσεων -μελών του νέου τύπου (αρχείο κώδικα)
- Χρήση του νέου τύπου, για ορισμό μεταβλητών, σε αρχεία πηγαίου κώδικα.

## Χρήση του τύπου Student

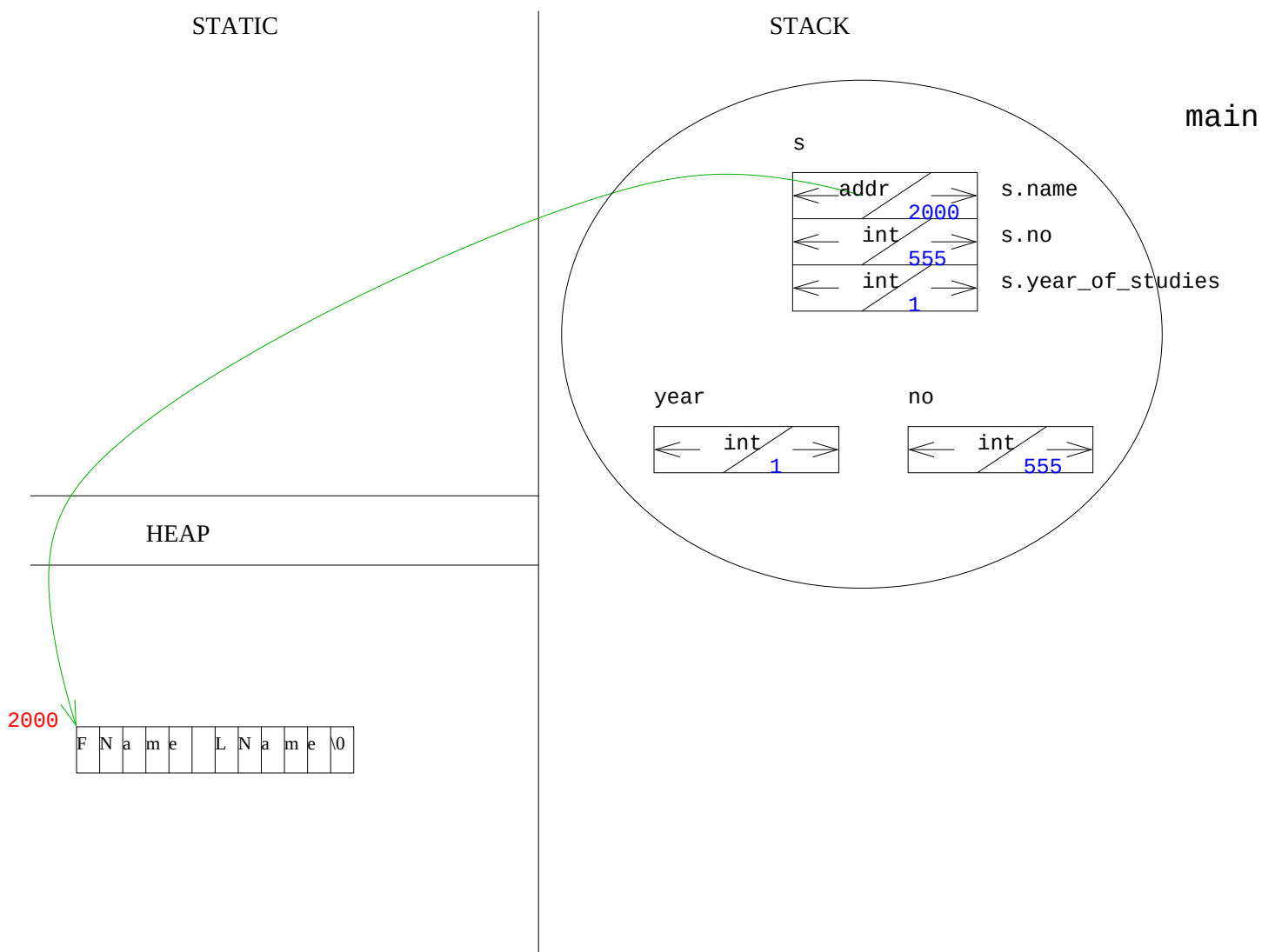
```
int main()
{
    Student s;
    int year, no;

    s.set_name("FName LName");
    s.set_no(555);
    s.set_year_of_studies(1);

    year = s.get_year_of_studies();
    no = s.get_no();

    return 0;
}
```

Η εικόνα των μεταβλητών στην περίπτωση της κλάσης Student (αφαιρετικά) στη μνήμη



## inline συναρτήσεις-μέλη

- Μια συνάρτηση-μέλος που ορίζεται —κι όχι απλώς δηλώνεται— στον ορισμό μιας κλάσης θεωρείται ότι είναι `inline`. Π.χ.

```
class date {
    int day, month, year;
public:
    void set(int d, int m, int y)
        // INLINE DEFINITION
    {   day = d; month = m; year = y; }

    void get(int& d, int& m, int& y) const
        // INLINE DEFINITION
    {   d = day; m = month; y = year; }
};
```

- Υπενθυμίζουμε ότι συναρτήσεις που έχουν προσδιοριστεί ως `inline` έχουν εσωτερική συνδεσιμότητα.

Θα μπορούσαμε να αλλάξουμε τον ορισμό της κλάσης Student ώστε να συμπεριλάβουμε και τους ορισμούς των συναρτήσεων-μελών, οποτε οι συναρτήσεις-μέλη της θα ήταν σαν να είχαν προσδιοριστεί inline.

```
class Student {  
    char* name;  
    // ...  
public:  
    // ...  
    char* get_name() const { return name; }  
                                // INLINE DEFINITION  
    // ...  
};
```



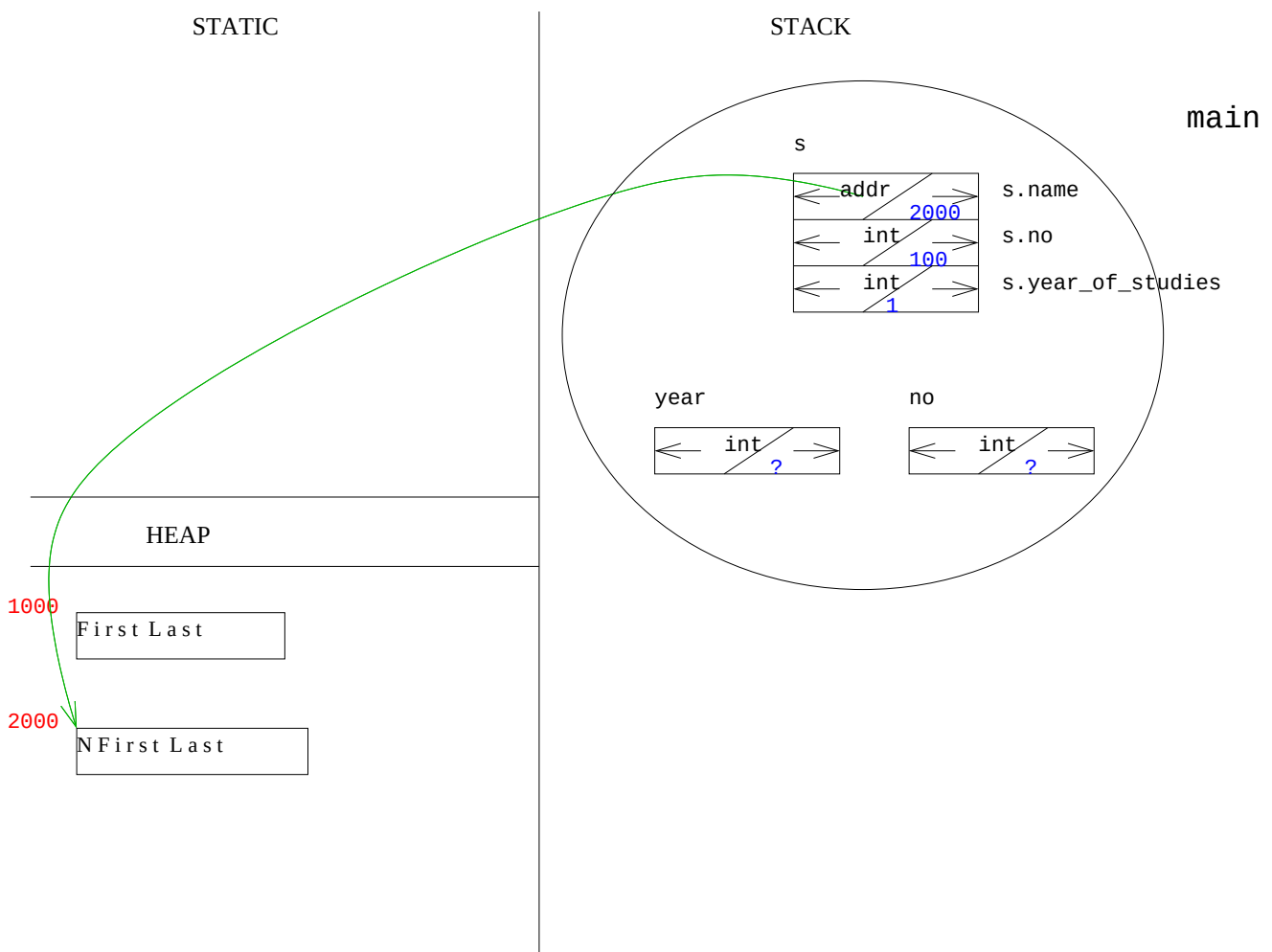
## Αρχικοποίηση Αντικειμένων

- Ανά πάσα στιγμή, ένα αντικείμενο βρίσκεται σε μία κατάσταση (state), η οποία μέσω μιας μεθόδου μπορεί να αλλάξει ή να παρατηρηθεί.
- Η κατάσταση ενός αντικειμένου προκύπτει από την τιμή την οποία έχει το αντικείμενο σε μια συγκεκριμένη χρονική στιγμή.
- Ένα αντικείμενο πρέπει να βρίσκεται πάντα σε καλά ορισμένη κατάσταση καθ' όλη τη διάρκεια του χρόνου ζωής του, είτε πρόκειται για την αρχική κατάσταση, είτε για μεταγενέστερη.
- Η απόδοση αρχικής κατάστασης ενός αντικειμένου και, γενικότερα η διαδικασία αρχικοποίησης, γίνεται μέσω ειδικών συναρτήσεων-μελών, οι οποίες λέγονται *συναρτήσεις αρχικοποίησης* (constructors).
- Οι συναρτήσεις αρχικοποίησης καλούνται αυτόματα κατά τη δημιουργία του αντικειμένου –όταν ορίζεται μια μεταβλητή του τύπου της κλάσης.
- Η διαδικασία αρχικοποίησης έχει σαν στόχο π.χ. να δώσει κάποιες αρχικές τιμές στα μέλη-δεδομένα του αντικειμένου ή/και να γίνουν κάποιες αρχικές διαδικασίες κ.α.

Με τον ορισμό της κλάσης Student ως έχει, αν έχω το παρακάτω:

```
Student s;
int year, no;
s.set_name("First Last");
s.set_no(100);
s.set_year_of_studies(1);
s.set_name("NFirst Last");
```

η αφαιρετική εικόνα της μνήμης μετά την εκτέλεση της τελευταίας εντολής είναι:



**Μη επιθυμητό!**

## Αρχικοποίηση Αντικειμένων στην C++

- Οι συναρτήσεις αρχικοποίησης προσδιορίζονται από το ότι έχουν το ίδιο όνομα με την κλάση.
- Στο πρωτότυπο των συναρτήσεων αρχικοποίησης δεν υπάρχει τύπος επιστροφής.
- Μπορούμε να έχουμε υπερφορτωμένους ορισμούς της συνάρτησης αρχικοποίησης.

```
class date {
// ...
public:
    date(int, int, int);
    date(int, int);    // e.g. this year
    date(int);         // e.g. this year
                        //          and this month
    date();            // e.g. today
// ...
};
```

- Μπορεί να δοθούν default αρχικές τιμές για τα ορίσματά τους, όπως για οποιαδήποτε άλλη συνάρτηση.
- ```
date(int d=1, int m=1, int y=2000);
```

- Η συνάρτηση αρχικοποίησης της οποίας η δήλωση ταιριάζει με τα ορίσματα που ακολουθούν το όνομα της μεταβλητής κατά τον ορισμό της καλείται αυτόματα από τη γλώσσα, μετά τη δέσμευση του χώρου για τη δημιουργία του αντικειμένου.

```
int main()
{
    // ...
    date day(14);
    date day_month(14,3);
    date today;
    // ...
}
```

- Στην περίπτωση δημιουργίας αντικειμένου μέσω δυναμικής δέσμευσης χώρου στον σωρό, τα ορίσματα ακολουθούν τον τύπο της κλάσης στον οποίο εφαρμόζεται ο τελεστής `new`.  
Π.χ.

```
new date(30,5,2000);
```

- Μια συνάρτηση αρχικοποίησης που καλείται χωρίς να έχουν προσδιοριστεί ορίσματα (τιμές τους) αναφέρεται ως *default συνάρτηση αρχικοποίησης (default constructor)*.

## Παράδειγμα αρχικοποίησης με default ορίσματα

```
class date{
    // ...
public:
    date(int d=1, int m=1, int y=2000);
    ...
};
```

```
int main()
{
    // ...
    date day_default;
    // day_default.day = 1
    // day_default.month = 1
    // day_default.year = 2000

    date day_month(14,3);
    // day_month.day = 14
    // day_month.month = 3
    // day_month.year = 2000

    date day_explicit(5,10,2030);
    // day_explicit.day = 5
    // day_explicit.month = 10
    // day_excplicit.year = 2030
    // ...
}
```



Ο ορισμός της κλάσης date προσθέτοντας και συνάρτηση αρχικοποίησης

```
class date {  
private:  
    int day, month, year;  
public:  
    date(int d = 0, int m = 0, int y = 0)  
        : month(m), day(d), year(y) // initializer list  
    { cout << "Creating a date!" << endl; }  
  
    void set(int i, int j, int k)  
    { day = i; month = j; year = k; }  
  
    void get(int& i, int& j, int& k) const  
    { i = day; j = month; k = year; }  
};
```

## Αλλαγές στον ορισμό της κλάσης Student για αρχικοποίηση

Καλό θα ήταν να έχουμε ορίσει τουλάχιστον μία συνάρτηση αρχικοποίησης που να αρχικοποιεί το μέλος-δεδομένο `name` με μια συμβολοσειρά που θα αντιστοιχεί στο όνομά του.

```
class Student {
    char* name;
    // ...
public:
    // ...
    Student(const char*);
    // ...
};
```

Το παραπάνω συνοδεύεται από τις εξής αλλαγές:

```
// CONSTRUCTOR
Student::Student(const char* nam)
{
    name = new char[strlen(nam)+1];
    strcpy(name,nam);
}

// CHANGE name MUTATOR
void Student::set_name(const char* nam)
{
    delete[] name;
    name = new char[strlen(nam)+1];
    strcpy(name,nam);
}
```



## Καταστροφή Αντικειμένων (στην C++)

- Η αντίστροφη διαδικασία της κατασκευής αντικειμένων είναι εκείνη της καταστροφής αντικειμένων.
- Πολλοί τύποι χρειάζονται να οριστεί μια συνάρτηση-μέλος για να διασφαλιστεί η ασφαλής απελευθέρωση του χώρου που καταλαμβάνει ένα στιγμιότυπό τους —ενέργειες που πρέπει να γίνουν— πριν χαθεί η δυνατότητα πρόσβασης στο χώρο που καταλαμβάνει αυτό.
- Η C++ δίνει τη δυνατότητα να ορίσουμε μια συνάρτηση-μέλος, τη *συνάρτηση καταστροφής (destructor)*, η οποία καλείται αυτόματα από τη γλώσσα πριν καταστραφεί ένα αντικείμενο της κλάσης.
- Συντακτικά, η γενική μορφή της συνάρτησης καταστροφής στην C++, για μια κλάση X, είναι:  

$$\sim X()$$
—το σύμβολο  $\sim$  υπονοεί “συμπλήρωμα” (“αντίθετο”) των συναρτήσεων αρχικοποίησης.
- Όταν ένα αντικείμενο πρόκειται να καταστραφεί, καλείται αυτόματα η συνάρτηση καταστροφής της κλάσης του.
- Προσοχή: στη C++, βγαίνοντας από το σώμα της main, δεν σταματά η εκτέλεση: καλούνται και οι συναρτήσεις καταστροφής όλων των αντικειμένων που δημιουργήθηκαν (εκτός εκείνων στο σωρό).

## Προσθήκη συνάρτησης καταστροφής στον ορισμό της κλάσης Student

Για την κλάση Student, θα πρέπει να οριστεί και συνάρτηση καταστροφής για την αποδέσμευση του χώρου που καταλαμβάνει η συμβολοσειρά του ονόματος.

- Ο ορισμός της κλάσης Student επαυξάνεται με τη δήλωση της συνάρτησης καταστροφής:

```
class Student {
    // ...
public:
    // ...
    ~Student();
    // ...
};
```

- Ορισμός της συνάρτησης καταστροφής:

```
// DESTRUCTOR
Student::~~Student()
{
    delete[] name;
}
```

## Αρχικοποίηση Αντικειμένων (συνέχεια):

### Αρχικοποίηση μέσω αντιγραφής

- Κατ' αντιστοιχία με τους τύπους της γλώσσας, μπορούμε να αρχικοποιούμε τα αντικείμενα μιας κλάσης με αντιγραφή της τιμής άλλων αντικειμένων της (π.χ. `date d = today;` αρχικοποίηση της `d` με αντιγραφή της τιμής της `today`).
- Ο χρήστης μπορεί να ορίσει τη διαδικασία που θα ακολουθηθεί σε αυτή την περίπτωση μέσω μιας συνάρτησης αρχικοποίησης, την *συνάρτηση αρχικοποίησης μέσω αντιγραφής* (copy constructor).
- Στη C++, σε περίπτωση που δεν έχει δοθεί κάποιος άλλος ορισμός, γίνεται επιμέρους αντιγραφή τιμών (bitwise copy).
- Ο άλλος ορισμός αναφέρεται στο να οριστεί μια συνάρτηση αρχικοποίησης μέσω αντιγραφής, η οποία για μια κλάση `X`, έχει το πρωτότυπο `X::X(const X&)`.
- “Κανόνας”: Όταν χρειάζεται να οριστεί συνάρτηση καταστροφής, χρειάζεται να οριστεί και συνάρτηση αρχικοποίησης μέσω αντιγραφής, και αντίστροφα –καθώς και να γίνει επαναορισμός του τελεστή ανάθεσης.

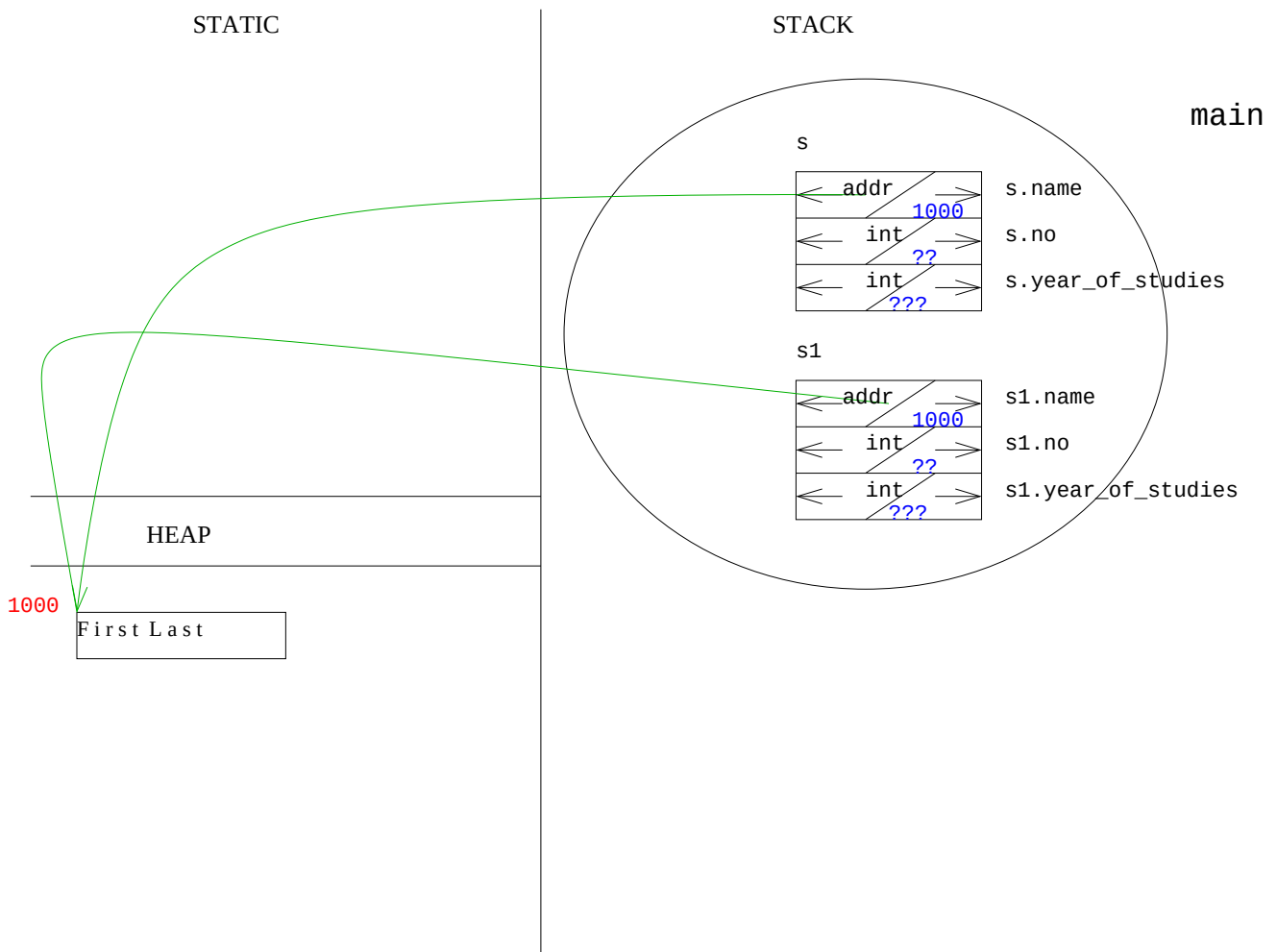
## Παράδειγμα: πρόβλημα της κλάσης Student

Έστω ότι έχουμε τον κώδικα:

```
Student s("First Last");
```

```
Student s1 = s; // identical to: Student s1(s);
```

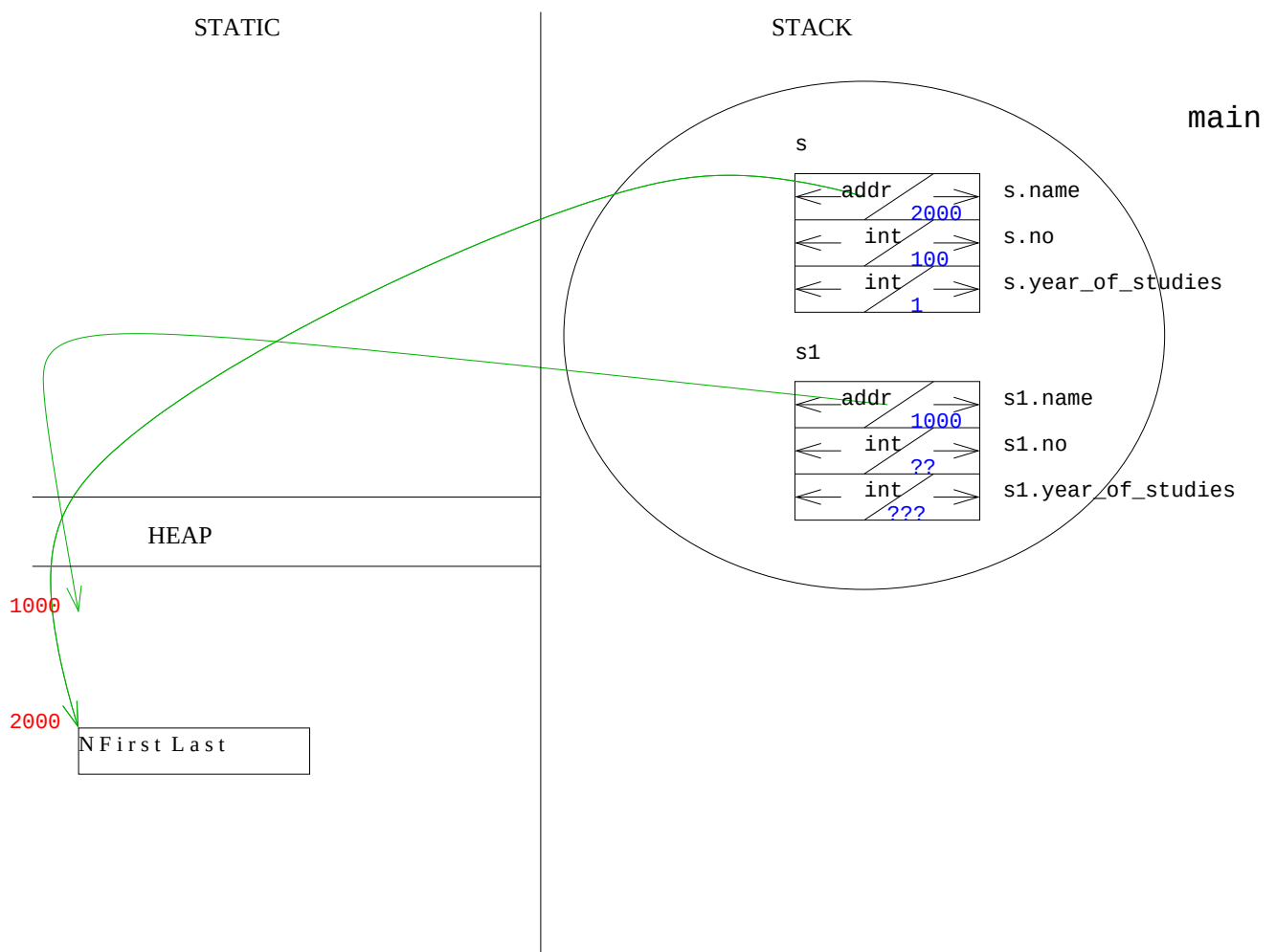
Η αφαιρετική εικόνα της μνήμης μετά την εκτέλεση αυτών των εντολών:



Αν ακολουθεί το:

```
s.set_name("NFirst Last");
```

Η αφαιρετική εικόνα της μνήμης μετά την εκτέλεσή του είναι:



## Λύση

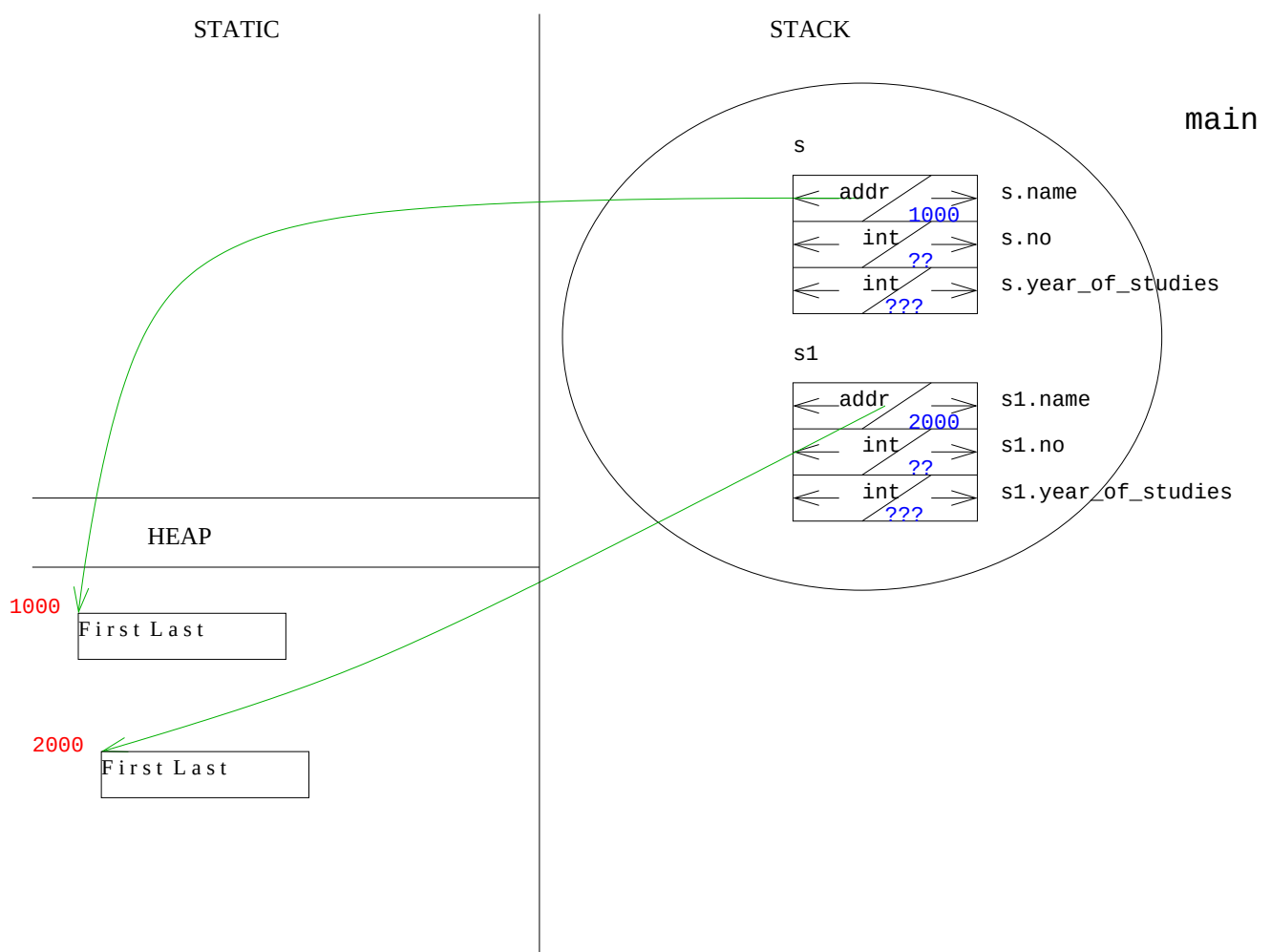
Αλλαγές στον ορισμό της κλάσης Student για αρχικοποίηση μέσω αντιγραφής

```
class Student {
    char* name;
    int no;
    int year_of_studies;
public:
    // ...
    Student(const Student& s);
    // ...
};
```

Το παραπάνω συνοδεύεται από τον εξής ορισμό:

```
// COPY CONSTRUCTOR
Student::Student(const Student& s)
{
    name = new char[strlen(s.name)+1];
    strcpy(name,s.name);
// name = s.name; Careful: this is the result of default copy
    no = s.no;
    year_of_studies = s.year_of_studies;
    cout << "A student was just created by copying ... "
         << endl;
}
```

Θεωρώντας τον προηγούμενο ορισμό συνάρτησης αρχικοποίησης μέσω αντιγραφής, η αφαιρετική εικόνα της μνήμης μετά την δημιουργία των δύο μεταβλητών είναι:



## Επαναορισμός του Τελεστή Ανάθεσης στην C++

- Δυνατότητα αναθέσεων τιμών μεταξύ αντικειμένων (κατ' αντιστοιχία με τους βασικούς τύπους).
- Αν δεν έχει οριστεί κάτι διαφορετικό, είναι “τυφλή” ανάθεση τιμών (bitwise).
- Για παρόμοιους λόγους με την αρχικοποίηση μέσω αντιγραφής, αυτό μπορεί να μην είναι επιθυμητό.
- Δυνατότητα επαναορισμού του τελεστή ανάθεσης για μια κλάση:
  - Σαν συνάρτηση-μέλος της κλάσης (εφαρμογή σε αντικείμενο της κλάσης).
  - Προσοχή στην ταυτοτική ανάθεση (λύση: χρήση του δείκτη `this`).
  - Επιστροφή του αντικειμένου στο οποίο έγινε η ανάθεση (δηλαδή, το `*this`) για να επιτραπεί αλυσιδωτή εφαρμογή του τελεστή.



Επαναορισμός του τελεστή ανάθεσης για την κλάση Student

Αλλαγές στον ορισμό της κλάσης Student

```
class Student {
    char* name;
    int no;
    int year_of_studies;
public:
    // ...
    Student& operator=(const Student& s);
    // ...
};
```

Το παραπάνω συνοδεύεται από τον εξής ορισμό:

```
Student& Student::operator=(const Student& s)
{
    if (this != &s) { // Careful not to assign to itself,
                     // especially if some storage
                     // reclaim was made within the body
        delete[] name;
        name = new char[strlen(s.name)+1];
        strcpy(name,s.name);
        no = s.no;
        year_of_studies = s.year_of_studies;
        cout << "I just performed a student ASSIGNMENT ... "
              << endl;
    }
    return *this;
}
```

## Παρατηρήσεις

Είναι ενδιαφέρον να ξεκαθαριστούν τα παρακάτω:

1. Κατασκευή αντικειμένου κλάσης με αρχικοποίηση με παραμέτρους άλλου τύπου: χρήση ανάλογης συνάρτησης αρχικοποίησης.
2. Κατασκευή αντικειμένου κλάσης με αντιγραφή υπάρχοντος αντικειμένου της κλάσης: χρήση συνάρτησης αρχικοποίησης μέσω αντιγραφής.
3. Ανάθεση τιμών υπάρχοντων αντικειμένων: χρήση του υπερφορτωμένου τελεστή ανάθεσης,  $=$ .

Προσοχή: η χρήση του τελεστή ανάθεσης στον ορισμό αντικειμένου δεν είναι έκφραση ανάθεσης. Υπονοεί χρήση της συνάρτησης αρχικοποίησης μέσω αντιγραφής.

## Κλάσεις σαν τύπος στα ορίσματα συναρτήσεων και τύπος επιστροφής

- Το πέρασμα παραμέτρων κατά την κλήση συνάρτησης και η επιστροφή της τιμής της οδηγούν σε διαδικασίες αρχικοποίησης των εμπλεκόμενων μεταβλητών –χρήση συνάρτησης αρχικοποίησης μέσω αντιγραφής.
- Όταν ένα όρισμα πρόκειται να χρησιμοποιηθεί μόνο για ανάγνωση, καλό είναι να δηλωθεί ως αναφορά και να προσδιοριστεί ως `const`.
- Από την έκδοση C++11 κι ύστερα:
  - συνάρτηση αρχικοποίησης με μεταφορά (move constructor) και
  - τελεστής ανάθεσης με μεταφορά (move assignment operator)
 για την ελεγχόμενη αποφυγή σπατάλης πόρων (ο “κανόνας” επαναορισμού επεκτείνεται).

## Ορίζοντας μια Κλάση

- Καθορισμός δημόσιων ονομάτων:
  1. το όνομα της κλάσης και
  2. οι δηλώσεις των ονομάτων των συναρτήσεων-μελών που αντιστοιχούν στη συμπεριφορά της.
- Απόφαση αναπαράστασης των αντικειμένων της κλάσης.
- Με δεδομένη την αναπαράσταση, ορισμός των συναρτήσεων-μελών –ονόματα που αφορούν λεπτομέρεια αναπαράστασης ή υλοποίησης πρέπει να δηλωθούν στο ιδιωτικό τμήμα της κλάσης.
- Καθορισμός συναρτήσεων αρχικοποίησης –συναρτήσεις αρχικοποίησης που επιτρέπεται να χρησιμοποιηθούν εκτός της εμβέλειας της κλάσης πρέπει να δηλωθούν στο δημόσιο τμήμα της κλάσης.
- Μελέτη της ζωής των αντικειμένων της κλάσης: αν είναι συνυφασμένη με την πρόσβαση σε άλλους πόρους, πρέπει να οριστούν κατάλληλες συναρτήσεις καταστροφής και αρχικοποίησης μέσω αντιγραφής και να γίνει επαναορισμός του τελεστή ανάθεσης. Προσοχή: στην περίπτωση επαναορισμού αυτών των διαδικασιών, πρέπει να συμπεριληφθούν όλες οι ενέργειες που πρέπει να γίνουν –ο έλεγχος πηγαίνει αποκλειστικά στην επαναορισμένη διαδικασία.

## Κλάσεις (συνοπτικά)

- Μια κλάση εισάγεται με τη δεσμευμένη λέξη `class` (ή `struct`).
- Διαχωρισμός `private/public` επιτυγχάνει απόκρυψη πληροφορίας.
- Δημιουργία στιγμιοτύπων (αντικειμένων) που “κτίζονται” από τα μέλη-δεδομένα (data members) της κλάσης στην οποία ανήκουν.
- Αρχικοποίηση στιγμιοτύπων μέσω συναρτήσεων αρχικοποίησης.
- Καταστροφή στιγμιοτύπων εκμεταλλευόμενη συναρτήσεις καταστροφής.
- Χειρισμός αντικειμένων μέσω συναρτήσεων-μελών (member functions).
- Πρόσβαση από το σώμα της συνάρτησης-μέλος στο αντικείμενο για το οποίο καλείται με χρήση του δείκτη `this`.
- Αρχικοποίηση δια αντιγραφής.
- Χρήση τελεστή ανάθεσης.

## Σταθερές και Κλάσεις

**Συναρτήσεις-μέλη const:** Δεν μεταβάλλουν το αντικείμενο πάνω στο οποίο εφαρμόζονται, π.χ.

```
char* Student::get_name() const;
```

Έτσι δηλώνουμε ότι μια συνάρτηση-μέλος είναι αναγνώστρια συνάρτηση.

**Σταθερά Μέλη-Δεδομένα:** Ένα μέλος-δεδομένο που είναι σταθερά. π.χ. ο “αριθμός μητρώου” ενός φοιτητή ανατίθεται εξ αρχής και δεν αλλάζει. Τότε, στον ορισμό της κλάσης Student πρέπει να αλλάξουμε τη δήλωση του μέλους-δεδομένου no, σε:

```
const int no;
```

(οπότε, πρέπει να αφαιρέσουμε τη συνάρτηση-μέλος void Student::set\_no(int)). Επιπλέον, πρέπει να αρχικοποιήσουμε το μέλος Student::no στις συναρτήσεις αρχικοποίησης μέσω λίστας αρχικοποίησης.

**Αντικείμενα const:** Αντικείμενα που δεν επιτρέπεται να μεταβληθούν κατά τη διάρκεια της ζωής τους (χρήση της λέξης-κλειδιού const κατά τον ορισμό τους), π.χ.

```
const Date millennium(1,1,2000);
```

Οι μόνες συναρτήσεις-μέλη που μπορούν να εφαρμοστούν σε αυτά τα αντικείμενα είναι εκείνες που έχουν δηλωθεί ως const –μπορούμε να έχουμε υπερφορτωμένους ορισμούς συναρτήσεων-μελών const με non-const εκδοχές τους που καλούνται ανάλογα με τη φύση του αντικειμένου (const ή όχι).

## Στατικά Μέλη Κλάσεων

- Κάθε αντικείμενο μιας κλάσης έχει το δικό του αντίγραφο για κάθε μέλος-δεδομένο της κλάσης (“χτίζεται” από αυτά).
- Οι συναρτήσεις-μέλη τις οποίες έχουμε συζητήσει έως τώρα εφαρμόζονται υποχρεωτικά σε αντικείμενο της κλάσης.
- Υπάρχουν περιπτώσεις που θέλουμε να “αποσυνδέουμε” τα μέλη κλάσεων από την ύπαρξη αντικειμένων τους. Αυτό επιτυγχάνεται με το να τα δηλώνουμε ως στατικά.
- Στην C++, αυτό επιτυγχάνεται με τη χρήση της λέξης-κλειδιού `static` κατά τη δήλωση της μεταβλητής ή της συνάρτησης στον ορισμό της κλάσης.
- Τα στατικά μέλη κλάσεων αρχικοποιούνται (και μόνο μια φορά) σε επίπεδο εμβέλειας κλάσης.
- Τα στατικά μέλη κλάσεων μπορούν να δηλωθούν και ως `const`.
- Ουσιαστικά, τα στατικά μέλη μιας κλάσης είναι συναρτήσεις και στατικές μεταβλητές που ανήκουν στην εμβέλεια της κλάσης.
- Για την C++ λιγότερο κρίσιμα από ότι για άλλες γλώσσες αντικειμενοστραφούς προγραμματισμού.

## Παράδειγμα

### Δήλωση στατικών μελών:

```

class Student {
public:
    Student(int nov);
    ~Student();

    int get_no() const
    { return no; }

    bool graduate() const
    { return no_courses == TOTAL_COURSES;
      // const static together
      // with instance var
    }

    int evaluate_GPA() const;

    void record_grade(int grade);

    static int get_highest_GPA() // static function definition
    { return highest_GPA; }    // static var access

    static int get_no_students() // static function definition
    { return no_students; }      // static var access

    static const int PASS_GRADE = 5; // static const var definition

    static const int TOTAL_COURSES = 40; // static const var
                                          // definition

private:
    static int no_students; // static var declaration
    static int highest_GPA; // static var declaration

    const int no;
    int no_courses;
    int completed_courses[TOTAL_COURSES];
};

```



## Ορισμός και αρχικοποίηση:

```

int Student::no_students = 0;           // static var definition
int Student::highest_GPA = 0;          // static var definition

Student::Student(int nov) : no(nov), no_courses(0)
{
    no_students++;                      // static var access
    cout << "Student with no " << no << " created!" << endl;
    cout << "There are " << no_students << " Students now!" << endl;
}

Student::~~Student()
{
    no_students--;                      // static var access
    cout << "Student with no " << no << " to be destroyed!" << endl;
    cout << "There are " << no_students // static var access
        << " Students now!" << endl;
}

void Student::record_grade(int grade)
{
    if (grade >= PASS_GRADE) {          // const static together
                                        // with instance var
        completed_courses[no_courses++] = grade;
    };
}

int Student::evaluate_GPA() const
{
    int curr_GPA = 0;
    if (no_courses > 0) {
        for (int i = 0; i < no_courses; i++)
            curr_GPA += completed_courses[i];
        if ((curr_GPA % no_courses) < (no_courses * 0.5))
            curr_GPA = curr_GPA / no_courses;
        else
            curr_GPA = ((curr_GPA / no_courses) + 1);
        if (graduate())
            if (curr_GPA > highest_GPA) // static var access
                highest_GPA = curr_GPA; // static var access
    }
    return curr_GPA;
}

```

## Χρήση στατικών μελών:

```

int main()
{
    cout << Student::get_no_students()    //    static function call
        << endl;                        //    no instances yet

    Student s1(100);
    Student s2(200);

    cout << Student::get_no_students()    //    static function call
        << endl;

    cout << s2.get_no_students() << endl; //    static function call
                                           //    using instance

    s1.record_grade(9);
    s1.record_grade(4);
    s1.record_grade(7);
    s1.record_grade(10);
    cout << "The student with no; " << s1.get_no()
        << " has current GPA: " << s1.evaluate_GPA() << endl;

    cout << "The total courses will be: "
        << s1.TOTAL_COURSES << endl;    //    static const var access
                                           //    using instance

    cout << "The pass grade is: "
        << Student::PASS_GRADE << endl; //    static const var access

    Student* ps3 = new Student(300);
    cout << ps3->get_no_students() << endl; //    static function call
                                           //    using pointer

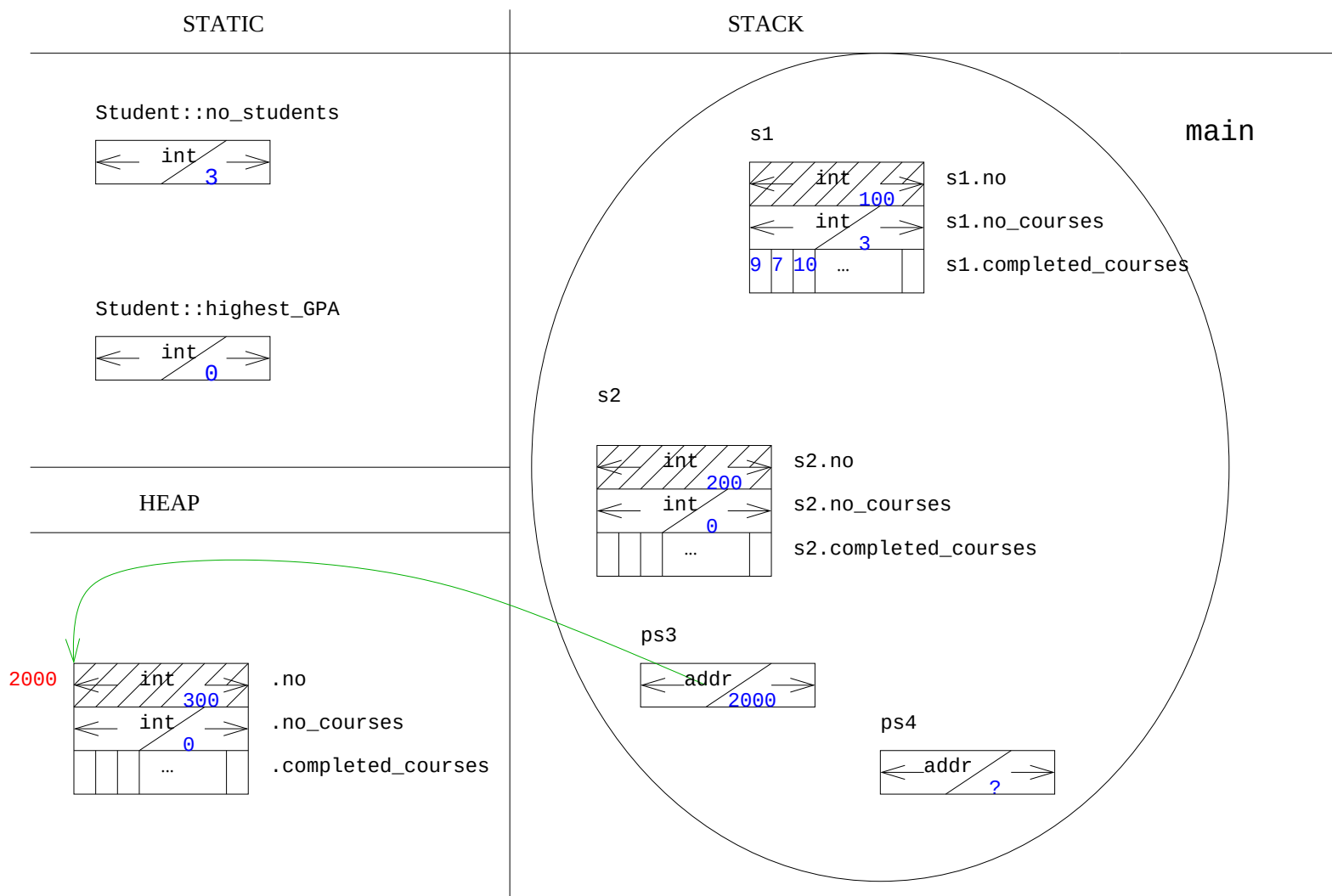
    Student* ps4;
    cout << ps4->get_no_students() << endl; //    static function call
                                           //    using pointer

    delete ps3;

    return 0;
}

```

Η αφαιρετική εικόνα της μνήμης με την αναπαράστασή τους πριν την εκτέλεση της εντολής `delete ps3;`



## Παρατηρήσεις στα στατικά μέλη κλάσεων

- Τα στατικά μέλη-δεδομένα μιας κλάσης υπάρχουν και μπορούν να χρησιμοποιηθούν ακόμα κι αν δεν έχουν δημιουργηθεί αντικείμενα της κλάσης. Επιπλέον, μπορούν να χρησιμοποιηθούν και οι στατικές συναρτήσεις-μέλη.
- Έξω από την κλάση, μπορούμε να έχουμε πρόσβαση στα `public static` μέλη μιας κλάσης
  - είτε μέσω ενός αντικειμένου της κλάσης
  - είτε με χρήση του ονόματος της κλάσης και του τελεστή `::` (υποχρεωτικά αν δεν έχουν δημιουργηθεί αντικείμενα της κλάσης)
- Όταν δεν έχουν δημιουργηθεί αντικείμενα μιας κλάσης, για να έχουμε πρόσβαση σε `private static` μέλη της κλάσης, έξω από την κλάση, πρέπει να έχει οριστεί μια `public static` συνάρτηση-μέλος της κλάσης και να κληθεί με το όνομα της κλάσης και χρήση του τελεστή `::`
- Δε μπορούμε να χρησιμοποιήσουμε το δείκτη `this` σε μια `static` συνάρτηση-μέλος μιας κλάσης.
- Δε μπορούμε να δηλώσουμε μια `static` συνάρτηση-μέλος σαν `const`.

Μερικά θέματα ακόμα σχετικά με την εμβέλεια των κλάσεων

- Φιλικές συναρτήσεις και κλάσεις
- Εμφωλευμένες κλάσεις
- ...

## ΣΥΝΘΕΣΗ ΚΛΑΣΕΩΝ (class composition)

- Μια κλάση μπορεί να έχει σαν μέλη-δεδομένα αντικείμενα άλλης κλάσης.
- Αν μια κλάση έχει σαν μέλος αντικείμενο άλλης κλάσης, κατά τη δημιουργία ενός αντικειμένου της πρώτης, δημιουργείται ένα αντικείμενο της δεύτερης του οποίου η ζωή είναι άρρηκτα συνδεδεμένη με τη ζωή του πρώτου.
- Η νέα κλάση “έχει ένα” (*has a*) αντικείμενο της υπάρχουσας κλάσης.
- Μιλάμε για σύνθεση κλάσεων.
- Η νέα κλάση καλείται σύνθετη.
- Παρατηρήσεις:
  - Δεν είναι απαραίτητο η σύνθεση να αφορά ένα μέλος-δεδομένο μόνο, μπορεί και παραπάνω, μπορεί και όλα τα μέλη.
  - Μπορούμε να έχουμε σύνθεση κλάσεων σε οποιοδήποτε βάθος.

## Σύνθεση Κλάσεων στην C++

- Η σύνθετη κλάση έχει πρόσβαση μόνο στα δημόσια ονόματα των κλάσεων που χρησιμοποιεί ως τύπους μελών της.
- Αρχικοποίηση αντικειμένων:
  - Πρώτα αρχικοποιούνται τα αντικείμενα που είναι μέλη της σύνθετης κλάσης (με τη σειρά που δηλώνονται στον ορισμό της κλάσης) κι έπειτα αρχικοποιείται το αντικείμενο της κλάσης.
  - Χρήση συναρτήσεων αρχικοποίησης που δηλώνονται σε λίστα αρχικοποίησης στις συναρτήσεις αρχικοποίησης της σύνθετης κλάσης –αν δεν δηλωθούν χρησιμοποιούνται οι default συναρτήσεις αρχικοποίησης.
  - Αν δεν δηλωθούν συναρτήσεις αρχικοποίησης στην σύνθετη κλάση, η C++ δημιουργεί μία που χρησιμοποιεί τις default συναρτήσεις αρχικοποίησης των μελών της (“non-trivial synthesized” συνάρτηση αρχικοποίησης).
- Καταστροφή αντικειμένων: Ακολουθείται η αντίστροφη σειρά από εκείνη της δημιουργίας, δηλαδή, πρώτα εκτελείται το σώμα της συνάρτησης καταστροφής της κλάσης και μετά εκείνες των μελών της (με την αντίστροφη σειρά της δήλωσης).
- Οι παραπάνω παρατηρήσεις ισχύουν για οποιοδήποτε βάθος σύνθεσης.

## Παράδειγμα

Τύπος Date (“Ημερομηνία”) :

```
class Date {

    int day;
    int month;
    int year;

public:

    Date(int d = 1, int m = 1, int y = 2006)
        : day(d), month(m), year(y) // non-class types,
            // i.e. initializer list
            // alternative to assignment
            // in the body
    {
//      day = d; month = m; year = y;
        cout << "A Date was just created with value: "
            << day << ' ' << month << ' ' << year << endl; }

    ~Date()
    {   cout << '\n'
        << "A Date is to be destroyed with value: "
        << day << ' ' << month << ' ' << year << endl; }

    void print() const
    {   cout << day << ' ' << month << ' ' << year << endl; }

};
```



Τύπος Time (“Χρονική Στιγμή”):

```
class Time {

    int hours;
    int mins;

public:

    Time(int h = 0, int m = 0) : hours(h), mins(m)
    {   cout << "A Time was just created with value: "
        << hours << ' ' << mins << endl; }

    ~Time()
    {   cout << '\n'
        << "A Time is to be destroyed with value: "
        << hours << ' ' << mins << endl; }

    void print() const
    {   cout << hours << " hours   and "
        << mins << " mins " << endl; }

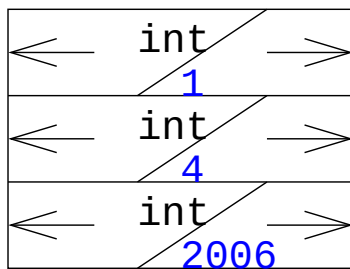
};
```

Έστω ότι ορίζουμε τις μεταβλητές:

`Date d(1,4); Time t;`

Οι αναπαραστάσεις τους (αφαιρετικά) είναι:

d



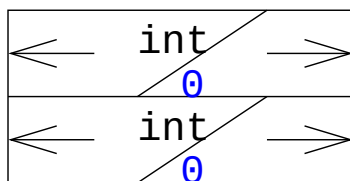
d.day

d.month

d.year

`Date d(1,4);`

t



t.hours

t.mins

`Time t;`

Σύνθετη κλάση DetailedDate (“Χρονική Στιγμή”):  
 “has a” μέλος-δεδομένο τύπου Time και “has a”  
 μέλος-δεδομένο τύπου Date

```
class DetailedDate {

    Date date;
    Time time;

public:

    DetailedDate()
// The default constructors of the data members are used
    { cout << "DetailedDate was just created with value:"
      << endl;

// NO VISIBILITY in private part
//      cout << date.day << ' ' << date.month << ' '
//          << date.year << endl;
//      cout << time.hours << " hours and "
//          << time.mins << " mins " << endl;

        date.print();
        time.print();
    }
```

```

// DetailedDate(int d, int mo, int y, int h, int mi) :
    DetailedDate(int d, int mo = 0, int y = 0,
                  int h = 0, int mi = 0) :
        date(d,mo,y), time(h,mi)
// Initializer list is required to pass the values
// to the constructors of data members.
{   cout << "DetailedDate was just created with value:"
    << '\n' << endl;

    date.print();
    time.print();
}

~DetailedDate()
{   cout << '\n'
    << "A DetailedDate is to be destroyed with value: "
    << endl;
    date.print();
    time.print();
}

void print() const
{   cout << "I am printing a DetailedDate " << endl;
    date.print();
    time.print();
}
};

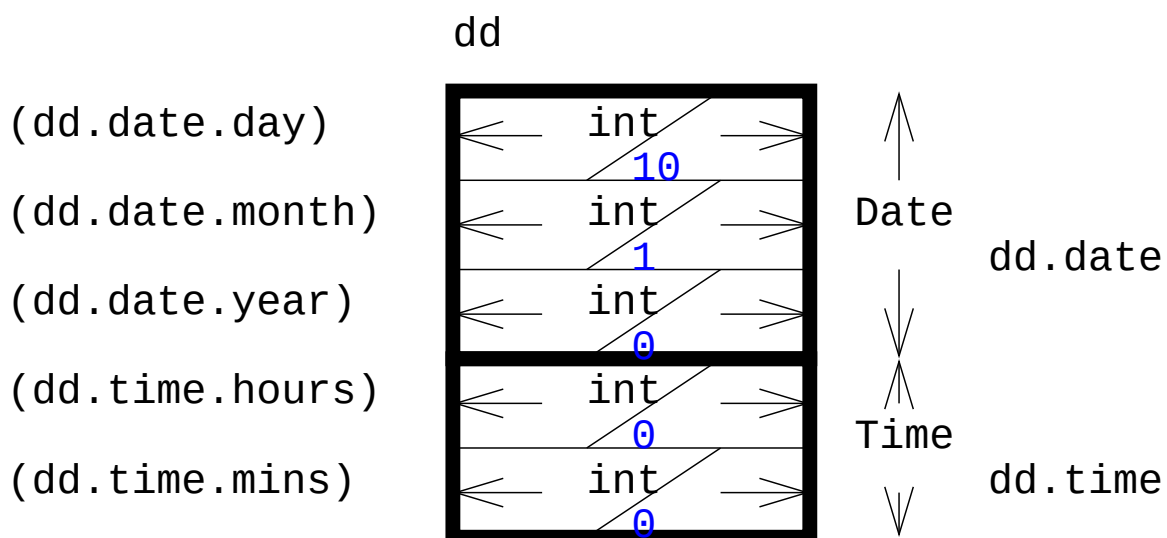
```

Έστω μεταβλητή `dd` που ορίζεται ως

```
DetailedDate dd(10,1);
```

Η αναπαράστασή της (αφαιρετικά) είναι:

```
DetailedDate dd(10,1);
```



## Άλλο Παράδειγμα

Τύπος date:

```
class date {
private:
    int day, month, year;
public:
    date() : month(0), day(0), year(0)
    {   cout << "Creating a date!" << endl; }

    ~date()
    {   cout << "Destroying a date with: " << day
        << "    " << month << "    " << year << endl; }

    void set(int i, int j, int k)
    {   day = i; month = j; year = k; }

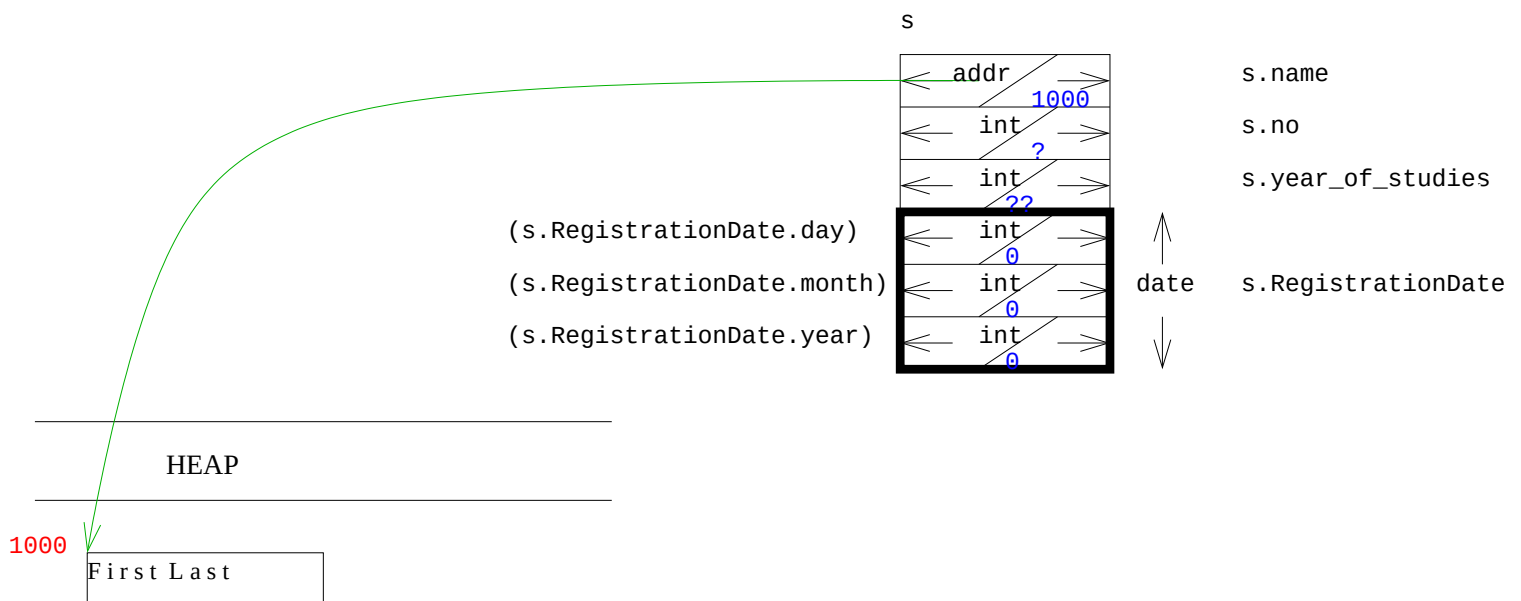
    void get(int& i, int& j, int& k) const
    {   i = day; j = month; k = year; }
};
```

Σύνθετη κλάση Student:

```
class Student {  
private:  
    char* name;  
    int no;  
    int year_of_studies;  
    date RegistrationDate;  
public:  
    Student(const char*);  
    Student(const Student&);  
  
    ~Student();  
  
    void set_name(const char*);  
    void set_no(int);  
    void set_year_of_studies(int);  
    void set_date(int, int, int);  
  
    char* get_name() const;  
    int get_no() const;  
    int get_year_of_studies() const;  
    void get_date(int&, int&, int&) const;  
};
```

Έστω μεταβλητή *s* που ορίζεται ως  
`Student s("First Last");`

Η αναπαράστασή της (αφαιρετικά) είναι:





Αλλαγή ορισμού της συνάρτησης αρχικοποίησης μέσω αντιγραφής για την κλάση Student:

```
Student::Student(const Student& s)
    : RegistrationDate(s.RegistrationDate)
        // Copy constructor to be used (even
        // if not defined by the user).
        // If this is in comments, the default
        // constructor is used for initializing
        // the data member RegistrationDate
{
    name = new char[strlen(s.name)+1];
    strcpy(name,s.name);
    no = s.no;
    year_of_studies = s.year_of_studies;
    cout << "A student was just created by copying ... " << endl;
}
```

## Παραλλαγή (προσθήκη τελεστή ανάθεσης)

Νέος τύπος date

```
class date {
private:
    int day, month, year;
public:
    date(int i = 0, int j = 0, int k = 0)
        : month(j), day(i), year(k)
    { cout << "Creating a date!" << endl; }

    date(const date& d)
        : day(d.day), month(d.month), year(d.year)
    { cout << "Creating a date by copying!" << endl; }

    ~date()
    { cout << "Destroying a date!" << endl; }

    // Experiment with keeping and deleting
    // the assignment redefinition
    date& operator=(const date& d)
    { day = d.day; year = d.year; month = d.month;
      cout << "I just performed a Date Assignment ... "
        << endl;
      return *this; }

    void set(int i, int j, int k)
    { day = i; month = j; year = k; }

    void get(int& i, int& j, int& k) const
    { i = day; j = month; k = year; }
};
```

## Νέος τύπος Student

```

class Student {
private:
    char* name;
    int no;
    int year_of_studies;
    date RegistrationDate;
public:
    Student(const char*);
    ~Student();
    Student(const Student&);

    // Experiment with keeping and deleting
    // the assignment redefinition
    // - in case of deletion, don't forget to comment out
    // the definition of the assignment operator
    // in the source file -
    Student& operator=(const Student& s);

    void set_name(const char*);
    void set_no(int);
    void set_year_of_studies(int);
    void set_date(int, int, int);

    char* get_name() const;
    int get_no() const;
    int get_year_of_studies() const;
    void get_date(int&, int&, int&) const;
};

```

Ορισμός του τελετή ανάθεσης για την σύνθετη κλάση Student

```
Student& Student::operator=(const Student& s)
{
// name = s.name;    Careful: this is the result
//                  of default assignment

    if (this != &s) { // Careful not to assign to itself,

        delete[] name;
        name = new char[strlen(s.name)+1];
        strcpy(name,s.name);
        no = s.no;
        year_of_studies = s.year_of_studies;
        RegistrationDate = s.RegistrationDate;
        // Experiment with
        // commenting out the assignment line
        cout << "I just performed a student ASSIGNMENT ... "
              << endl;
    }
    return *this;
}
```

## Μέλη-δεδομένα τύπου που σχετίζεται με κλάση

Προσοχή στην επιλογή του τύπου του μέλους:

**Τύπου κλάσης** : δημιουργία αντικειμένου –σχέση has a.

Επιλέγεται αν τα αντικείμενα της νέας κλάσης απαρτίζονται υποχρεωτικά από ένα αντίγραφο, έχουν καθένα το δικό τους κι αυτό δεν υφίσταται ανεξάρτητα από αυτά.

**Τύπου αναφοράς σε κλάση** : δεν δημιουργείται νέο αντικείμενο –το μέλος “πατάει” πάνω σε υπάρχον.

Επιλέγεται αν το αντικείμενο του μέλους υφίσταται ανεξάρτητα αλλά το αντικείμενο της νέας κλάσης αναφέρεται υποχρεωτικά σε κάποιο αντικείμενο της κλάσης του τύπου.

**Τύπου δείκτη σε κλάση** : δεν αποτελεί αντικείμενο.

Επιλέγεται αν το μέλος της νέας κλάσης δεν αναφέρεται υποχρεωτικά σε κάποιο αντικείμενο της κλάσης του τύπου.

Προσοχή στην πρόσβαση μια που το μέλος μπορεί να μην δείχνει σε αντικείμενο!

Περιεκτικό παράδειγμα: ο τύπος που αντιστοιχεί στην έννοια της “Οικογένειας”.

## ΚΛΗΡΟΝΟΜΙΚΟΤΗΤΑ (inheritance)

- Στον OOP, πρέπει να έχουμε τη δυνατότητα να προσδιορίζουμε συσχέτιση “γενικότερης-ειδικότερης” μεταξύ κλάσεων – κληρονομικότητα, σχέση *is a*.
- Με τον μηχανισμό που επιτρέπει κληρονομικότητα, ό,τι έχει οριστεί στη γενικότερη κλάση κληροδοτείται στην ειδικότερη και δεν χρειάζεται να επαναλαμβάνεται.
- Στην ειδικότερη κλάση, απλά προσδιορίζεται η “ειδοποιός διαφορά” που τη διακρίνει από τη γενικότερη, είτε από πλευράς χαρακτηριστικών, είτε από πλευράς γενικότερης λειτουργικότητας.



- Τα “οχήματα” (VEHICLE):
  - έχουν “τροχούς” (wheels),
  - ανά πάσα στιγμή βρίσκονται σε μία “θέση” (position) και
  - έχουν δυνατότητα “μετακίνησης” (move).

Διακρίνονται δε, σε:

- “αυτοκίνητα” (CAR)
  - “ποδήλατα” (BICYCLE)
  - “μοτοσυκλέτες” (MOTORBIKE)
- Τα “αυτοκίνητα” είναι “οχήματα” τα οποία έχουν
    - “4 τροχούς” (4\_wheels) και
    - “μηχανή” (engine).

Διακρίνονται σε:

- “κάμπριο” (CABRIO)
  - “χάτσμπακ” (HATCHBACK)
  - “λιμουζίνες” (LIMOUSINE)
  - “4x4” (4\_WHEEL\_DRIVE)
- ...



- Πιο γενικευμένες οντότητες και εξειδικεύσεις τους
- Διαβάζουμε το παραπάνω σχήμα από πάνω προς τα κάτω.
- Σχεδιάζουμε ανάποδα: Βρίσκουμε κοινά χαρακτηριστικά στις έννοιες/οντότητες του κόσμου μας και φτιάχνουμε νέες, πιο γενικευμένες, οντότητες σχετίζοντάς τις με αυτά.
- Ιεραρχική οργάνωση εννοιών/οντοτήτων –ταξινομία (taxonomy).
- Πολλά προβλήματα εμπεριέχουν κόσμους των οποίων οι έννοιες μπορούν να οργανωθούν κατ' αυτόν τον τρόπο.
- *πολλαπλή κληρονομικότητα* (multiple inheritance): μια έννοια/οντότητα αποτελεί άμεση εξειδίκευση παραπάνω από μίας έννοιας/οντότητας.

## Η Κληρονομικότητα στον Αντικειμενοστραφή Προγραμματισμό

- Μπορούμε να ορίσουμε μια κλάση ως εξειδίκευση μιας άλλης.
- Στον ορισμό της εξειδίκευσης δεν χρειάζεται να επαναλάβουμε ό,τι ορίζεται στην πιο γενική γιατί κληρονομείται από αυτήν, απλά καθορίζουμε την επιπλέον λειτουργικότητα, καθώς και λειτουργικότητα που μπορεί να επαναορίζεται.
- Ο μηχανισμός της κληρονομικότητας διασφαλίζει ότι ό,τι έχει οριστεί στη γενικότερη κλάση, θα κληροδοτηθεί στην ειδικότερη.
- Αν μια κλάση είναι εξειδίκευση μιας άλλης κλάσης, τα αντικείμενα της πρώτης είναι αντικείμενα και της δεύτερης –κατά συνέπεια, όπου απαιτείται αντικείμενο τύπου της πιο γενικής κλάσης, μπορεί να χρησιμοποιηθεί αντικείμενο της πιο εξειδικευμένης κλάσης. Προσοχή, το ανάποδο δεν ισχύει!.
- Ορολογία:
 

**Γενικεύσεις:** Βασική κλάση (base class), υπερκλάση (superclass)

**Εξειδικεύσεις:** Παραγόμενη κλάση (derived class), υποκλάση (subclass)

## Κληρονομικότητα vs Σύνθεση Κλάσεων

- Τόσο με την κληρονομικότητα μεταξύ κλάσεων, όσο και με τη σύνθεση κλάσεων, επιτυγχάνουμε επαναχρησιμοποίηση κλάσεων –σε ορισμούς νέων κλάσεων χρησιμοποιούμε ήδη υπάρχουσες κλάσεις.
- Όμως, αναπαριστούν τελείως διαφορετικές συσχετίσεις:
  - Η κληρονομικότητα σχετίζεται με τη συσχέτιση **isa**: ένα “αυτοκίνητο” **είναι** “όχημα” . Ένα αντικείμενο της εξειδικευμένης κλάσης **είναι** και αντικείμενο της πιο γενικής.
  - Η σύνθεση σχετίζεται με τη συσχέτιση **has a**: ένα “αυτοκίνητο” **έχει** “μηχανή”. Ένα αντικείμενο της νέας κλάσης (εμπερι)**έχει** ένα αντικείμενο της υπάρχουσας.
  - Κρίσιμο η διάκριση αυτή να γίνει κατανοητή, για χρήση κατάλληλης συσχέτισης μεταξύ των κλάσεων και πιο καθαρά οργανωμένο κώδικα.

## Κληρονομικότητα στη C++

- Νέες κλάσεις μπορούν να οριστούν από ήδη υπάρχουσες ενσωματώνοντας τα χαρακτηριστικά και τη συμπεριφορά τους ή επαναορίζοντάς τα όπου είναι απαραίτητο.
- Δηλώνουμε ότι μια κλάση, B, είναι υποκλάση μιας ήδη υπάρχουσας κλάσης, A, χρησιμοποιώντας την έκφραση:  

```
class B : public A
```

στον ορισμό της κλάσης B (σχέση *isa*).
- Μια παραγόμενη κλάση μπορεί να προσθέσει μέλη-δεδομένα και συναρτήσεις-μέλη επαυξάνοντας τον ορισμό της (εξειδικεύοντας όμως τα στοιχεία της).
- Ο τελεστής επίλυσης αμβέλειας μπορεί να χρησιμοποιηθεί για να χρησιμοποιηθεί η εκδοχή που ορίζεται σε μια βασική κλάση αντί για την επαναορισθείσα στην παραγόμενη.
- Δήλωση για πολλαπλή κληρονομικότητα, π.χ.  

```
class WorkingStudent : public Student,  
                      public Employee
```
- ...κι άλλα ζητήματα που θα συζητηθούν στη συνέχεια.

## Παράδειγμα

Έστω η έννοια του “Φοιτητή” υλοποιημένη σαν κλάση Student:

```
class Student {  
private:  
    char* name;  
    int no;  
public:  
    Student(const char*);  
    ~Student();  
    void set_no(int);  
    char* get_name() const;  
    int get_no() const;  
    void print() const;  
};
```

Ένας “Εργαζόμενος Φοιτητής” σαν εξειδίκευση του “Φοιτητή” υλοποιημένος σαν κλάση `WorkingStudent`, υποκλάση της κλάσης `Student`

```
class WorkingStudent : public Student {  
    int salary;  
    int months;  
public:  
    WorkingStudent(const char*, int);  
    void set_months(int);  
    int get_income() const;  
    void print() const;  
};
```

## Ορισμός των συναρτήσεων-μελών της WorkingStudent

```
WorkingStudent::WorkingStudent(const char* name, int sal)
    : Student(name), salary(sal)
{
    cout << "A Working Student has been created with salary: "
          << salary << endl;
}

void WorkingStudent::set_months(int mons)
{
    months = mons;
}

int WorkingStudent::get_income() const
{
    return months * salary;
}

void WorkingStudent::print() const
{
    Student::print();
    cout << "The Income is: " << get_income() << endl;
}
```

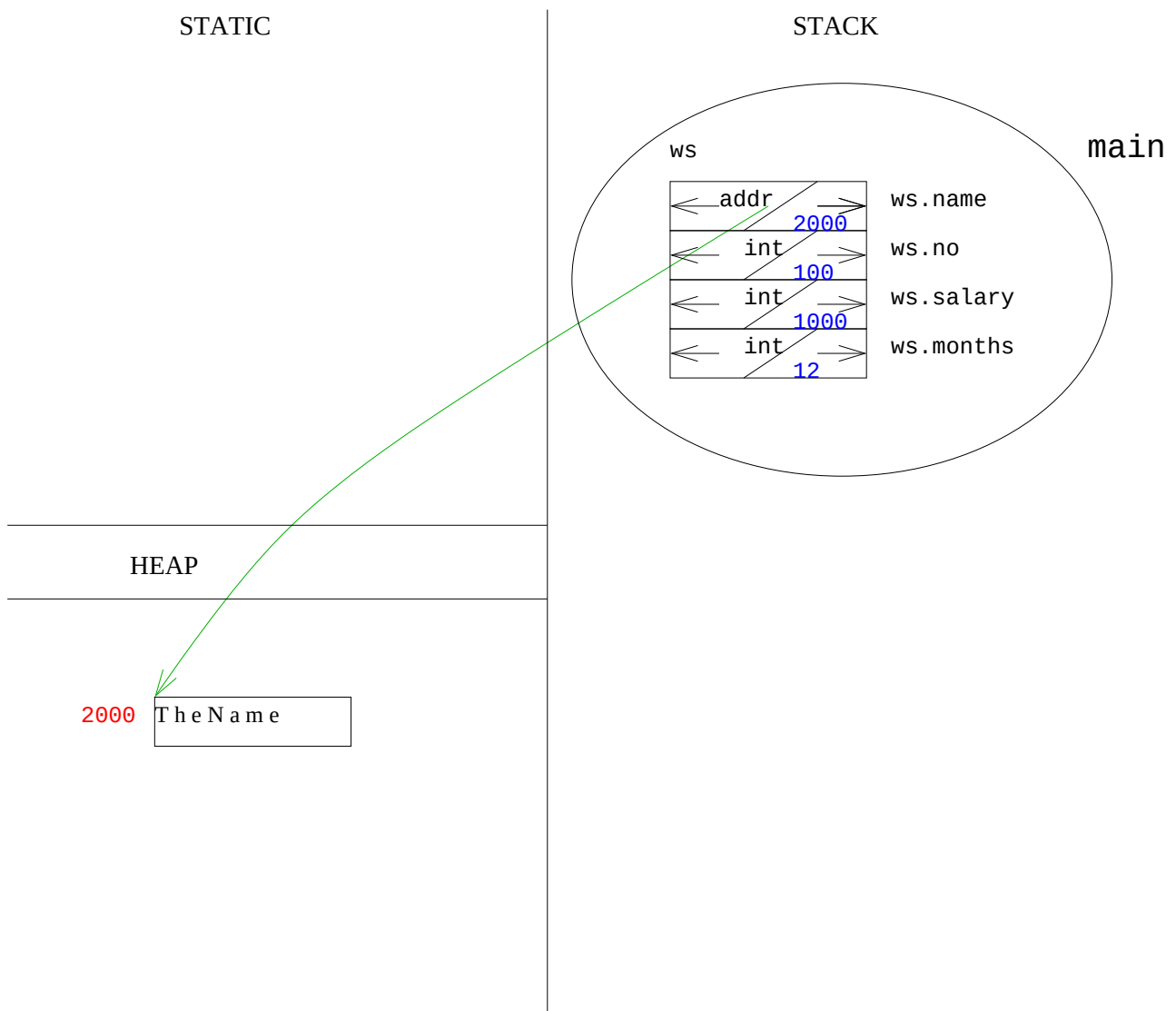
## Χρήση της WorkingStudent

```
int main()
{
    WorkingStudent ws("TheName", 1000); // Object
                                        // creation/initialization
    ws.set_no(100);                    // Inherited function
    ws.set_months(12);                 // Added function
    ws.print();                        // Redefined function

    return 0;
}
```



Η αφαιρετική εικόνα στη μνήμη πριν την ολοκλήρωση της εκτέλεσης



## Διαβαθμίσεις στον έλεγχο της πρόσβασης στα μέλη μιας κλάσης

Ένα μέλος μιας κλάσης μπορεί να είναι:

**private:** Το όνομά του μπορεί να χρησιμοποιηθεί μόνο από την εμβέλεια της κλάσης (και από φιλικές συναρτήσεις και φιλικές κλάσεις της κλάσης).

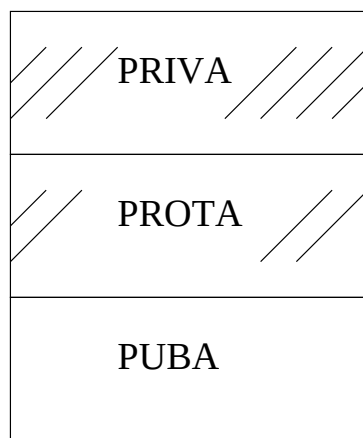
**protected:** Το όνομά του μπορεί να χρησιμοποιηθεί από την εμβέλεια της κλάσης (και από φιλικές συναρτήσεις και φιλικές κλάσεις της κλάσης), καθώς και από την εμβέλεια παραγόμενων κλάσεων (και από φιλικές συναρτήσεις και φιλικές κλάσεις παραγόμενων κλάσεων)  
για τα αντικείμενα των υποκλάσεων και μόνο.

**public:** Το όνομά του μπορεί να χρησιμοποιηθεί από οποιαδήποτε εμβέλεια του προγράμματος.

Κληρονομικότητα μεταξύ δύο κλάσεων A και B που δηλώνεται ως:

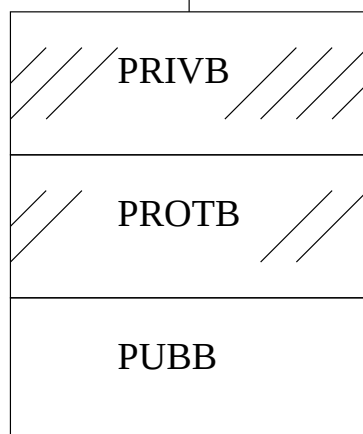
```
class B : public A
```

```
class B : public A {...};
```



**class A**

**public**



**class B**

**PUBB = PUBB + PUBA**

**PROTB = PROTB + PROTA**

**PRIVB = PRIVB**

## Ζητήματα προς προβληματισμό στην κληρονομικότητα

### Αναπαράσταση

Τα αντικείμενα της παραγόμενης κλάσης “κτίζονται” από τη συνένωση των μελών-δεδομένων της βασικής κλάσης και των δικών της.

### Θέματα Εμβέλειας

- Πώς (μέσω ποιων ονομάτων) χειριζόμαστε τα αντικείμενα της παραγόμενης κλάσης από τον υπόλοιπο κώδικα;  
Με τα ονόματα που είναι δηλωμένα σε δημόσιο τμήμα της παραγόμενης κλάσης και κάθε όνομα που είναι σε δημόσιο τμήμα της βασικής κλάσης. Στην περίπτωση που μια συνάρτηση-μέλος έχει επαναοριστεί στην παραγόμενη, ο ορισμός που υπάρχει στη βασική δεν είναι (άμεσα) ορατός.
- Πώς (μέσω ποιων ονομάτων) χειριζόμαστε τα αντικείμενα της παραγόμενης κλάσης από το περιβάλλον της κλάσης αυτής (π.χ. στα σώματα των ορισμών των συναρτήσεων-μελών της παραγόμενης κλάσης);  
Όσον αφορά κληρονομούμενα ονόματα από τη βασική κλάση, μπορούμε να χρησιμοποιήσουμε τα ονόματα που είναι δηλωμένα είτε στο `public` είτε στο `protected` τμήμα της.
- Πώς (μέσω ποιων ονομάτων) χειριζόμαστε τα αντικείμενα της βασικής κλάσης από το περιβάλλον της παραγόμενης;  
Όπως ακριβώς θα τα χειριζόμασταν από οποιοδήποτε άλλο μέρος του κώδικα, –μέσω του δημόσιου τμήματός τους.

## Αρχικοποίηση και Καταστροφή

- Όταν δημιουργείται ένα αντικείμενο μιας παραγόμενης κλάσης, μια συνάρτηση αρχικοποίησης της βασικής κλάσης πρέπει να κληθεί για να αρχικοποιήσει τη “φύση” του ως αντικείμενο της βασικής, π.χ. να αρχικοποιήσει εκείνα τα μέλη του στιγμιοτύπου που προέρχονται από τη βασική κλάση.
- Στις συναρτήσεις αρχικοποίησης της παραγόμενης κλάσης πρέπει να προσδιορίζεται η συνάρτηση αρχικοποίησης της βασικής που είναι επιθυμητό να χρησιμοποιηθεί και να περαστούν οι τιμές στα ορίσματά της (αυτό γίνεται με τη χρήση λίστας αρχικοποίησης). Διαφορετικά, καλείται η default συνάρτηση αρχικοποίησης της βασικής κλάσης.
- Οι συναρτήσεις αρχικοποίησης δεν κληρονομούνται.
- Κατά την κατασκευή ενός αντικειμένου, πρώτα καλείται η συνάρτηση αρχικοποίησης της βασικής κλάσης και μετά της παραγόμενης της.
- Στην καταστροφή, καλούνται οι συναρτήσεις καταστροφής και των δύο κλάσεων, αντίστροφα.

## Αρχικοποίηση μέσω Αντιγραφής

Αντίστοιχα με την παραπάνω διαδικασία αρχικοποίησης.

Προσοχή: αν οριστεί η συνάρτηση αρχικοποίησης μέσω αντιγραφής στην παραγόμενη κλάση, πρέπει μέσω λίστας αρχικοποίησης, να δηλωθεί ρητά η χρήση της συνάρτησης αρχικοποίησης μέσω αντιγραφής της βασικής.

## **Ανάθεση**

Σε περιπτώσεις που δεν έχει γίνει επαναορισμός του τελεστή ανάθεσης, γίνεται bitwise copy. Προσοχή: εάν επαναοριστεί ο τελεστής ανάθεσης για την παραγόμενη κλάση, πρέπει στο σώμα του να κληθεί ρητά και ο τελεστής ανάθεσης για τη βασική για να γίνει η ανάθεση μεταξύ των “υποαντικειμένων” της βασικής κλάσης.

**Κληροδότηση Κληρονομιάς** Τι κληροδοτείται στους απογόνους μιας παραγόμενης κλάσης από αυτά που κληρονόμησε από τη βασική της. Στην περίπτωση της καθαρής “isa” σχέσης κληρονομικότητας:

- τα δημόσια ονόματα μιας παραγόμενης κλάσης επαυξάνονται με τα δημόσια της βασικής της και
- τα προστατευόμενα ονόματα μιας παραγόμενης κλάσης επαυξάνονται με τα προστατευόμενα ονόματα της βασικής της.
- Τα ιδιωτικά ονόματα τόσο της παραγόμενης, όσο και της βασικής υπόκεινται στους περιορισμούς που έχουμε αναφέρει.

Τα παραπάνω ισχύουν αναδρομικά για κάθε βάθος ιεραρχίας.

## Σειρά κατασκευής - καταστροφής: γενική μορφή

Κατασκευή:

bmcs

bc

dmcs

dc

## Καταστροφή:

dd

dmds

bd

bmds

## Όπου:

**bmcs:** συναρτήσεις αρχικοποίησης των μελών-δεδομένων της βασικής κλάσης, με τη σειρά που εμφανίζονται στον ορισμό της κλάσης

**bc:** συνάρτηση αρχικοποίησης της βασικής κλάσης

**dmcs:** συναρτήσεις αρχικοποίησης των μελών-δεδομένων της παραγόμενης κλάσης, με τη σειρά που εμφανίζονται στον ορισμό της κλάσης

**dc:** συνάρτηση αρχικοποίησης της παραγόμενης κλάσης

**dd:** συνάρτηση καταστροφής της παραγόμενης κλάσης

**dmds:** συναρτήσεις καταστροφής των μελών-δεδομένων της παραγόμενης κλάσης, με την αντίστροφη σειρά που εμφανίζονται στον ορισμό της κλάσης

**bd:** συνάρτηση καταστροφής της βασικής κλάσης

**bmds:** συναρτήσεις καταστροφής των μελών-δεδομένων της βασικής κλάσης, με την αντίστροφη σειρά που εμφανίζονται στον ορισμό της κλάσης

## Δείκτες και αναφορές σε αντικείμενα ιεραρχίας κλάσεων

- Υπενθύμιση: Σε ένα μονοπάτι ιεραρχίας κλάσεων, ένα αντικείμενο υποκλάσης “είναι” και αντικείμενο όλων των υπερκλάσεών της στο μονοπάτι της ιεραρχίας.
- Μπορεί να χρησιμοποιηθεί οποτεδήποτε αναμένεται μεταβλητή των τύπων των υπερκλάσεων αυτών. Η δυνατότητα αυτή αφορά και τις μεταβλητές τύπου δείκτη και, κατ’ επέκταση, τις μεταβλητές αναφοράς.
- Δεν επιτρέπεται η ανάθεση αντικειμένου (κλπ...) της υπερκλάσης σε αντικείμενο (κλπ...) της υποκλάσης. –Αυτό μπορεί να παρακαμφθεί είτε επαναορίζοντας τον τελεστή ανάθεσης είτε/και ορίζοντας συνάρτηση μετατροπής.
- Με ρητό cast μπορούμε να μετατρέψουμε έναν δείκτη σε υπερκλάση σε δείκτη σε υποκλάση. Τότε μπορούμε να κάνουμε dereference μόνο όταν δείχνει πράγματι σε ένα αντικείμενο της υποκλάσης.



## Εικονικές συναρτήσεις

- Η χρήση δείκτη (ή αναφοράς) σε υπερκλάση, κατά την κλήση μιας συνάρτησης-μέλους, παραπέμπει στη πρόσβαση της συνάρτησης (στην εμβέλεια) της υπερκλάσης.
- Στην περίπτωση, όμως, που ο δείκτης αυτός (ή αναφορά) “δείχνει” σε αντικείμενο της υποκλάσης, πολλές φορές αυτό δεν είναι επιθυμητό.
- Οι συναρτήσεις για τις οποίες η συμπεριφορά αυτή δεν είναι η κατάλληλη πρέπει να έχουν προσδιοριστεί ως `virtual`, οπότε να είναι *εικονικές συναρτήσεις* (`virtual functions`).
- Η εφαρμογή μιας εικονικής συνάρτησης σε ένα αντικείμενο μέσω δείκτη (ή αναφοράς) οδηγεί στην χρήση του ορισμού της συνάρτησης που είναι ορατός στην εμβέλεια της κλάσης του αντικειμένου και όχι της κλάσης του δείκτη (ή αναφοράς).
- Σημείωση: Στην περίπτωση που μια συνάρτηση-μέλος εφαρμόζεται απευθείας πάνω σε αντικείμενο, ο προσδιορισμός `virtual` δεν παίζει κανένα ρόλο: η συνάρτηση-μέλος που χρησιμοποιείται είναι εκείνη που είναι απευθείας ορατή στην εμβέλεια της κλάσης του αντικειμένου.

- Ο πολυμορφισμός στη C++ γίνεται εμφανής μέσω της δυνατότητας να χειριζόμαστε αντικείμενα υποκλάσεων μέσω δεικτών (ή αναφορών) σε υπερκλάσεις.
- Η δήλωση μιας συνάρτησης ως εικονική σε μια κλάση επιτρέπει να επαναορίζουμε τη συνάρτηση στις παραγόμενες κλάσεις και να επιτυγχάνουμε την πρόσβαση στη “σωστή” συνάρτηση (ορισμό) στην κάθε περίπτωση.
- Οι εικονικές συναρτήσεις μας βοηθούν να σχεδιάζουμε και να υλοποιούμε συστήματα που είναι εύκολα επεκτάσιμα –διαφορετικά, λύσεις μη (εύκολα) επεκτάσιμες.
- Όταν μια κλάση έχει εικονικές συναρτήσεις, αποκτά, από τον μεταγλωττιστή, έναν “πίνακα συναρτήσεων virtual” και τα αντικείμενά της αποκτούν έναν δείκτη στον πίνακα αυτό.
- Απαξ και μια συνάρτηση-μέλος δηλωθεί ως virtual σε μια κλάση, παραμένει εικονική για όλη την ιεραρχία των κλάσεων που βρίσκονται κάτω από την κλάση αυτή.
- Μπορεί και να μην επαναορίζεται σε κάποια παραγόμενη κλάση.
- Δεν μπορούμε να έχουμε εικονικές συναρτήσεις κατασκευής αντικειμένων. Μπορούμε, όμως, να έχουμε εικονικές συναρτήσεις καταστροφής.
- Προσοχή: οι default τιμές εικονικών συναρτήσεων αντικαθίστανται σε φάση μεταγλώττισης, οπότε μπορεί να μην είναι οι επιθυμητές.

## Παράδειγμα

```
class I {
public:
// virtual void play() const = 0; // pure virtual
// Makes the class to be abstract:
// see later
// virtual void play() const
void play() const
{ cout << "I::play" << endl; }
};

class W : public I {
public:
void play() const
{ cout << "W::play" << endl; }
};
```

```
int main()
{
    I instr;
    instr.play();

    W flute;
    flute.play();

    instr = flute;
    // flute = instr; // error

    I* pflute = &instr;
    pflute->play();

    pflute = &flute;
    pflute->play();

    return 0;
}
```

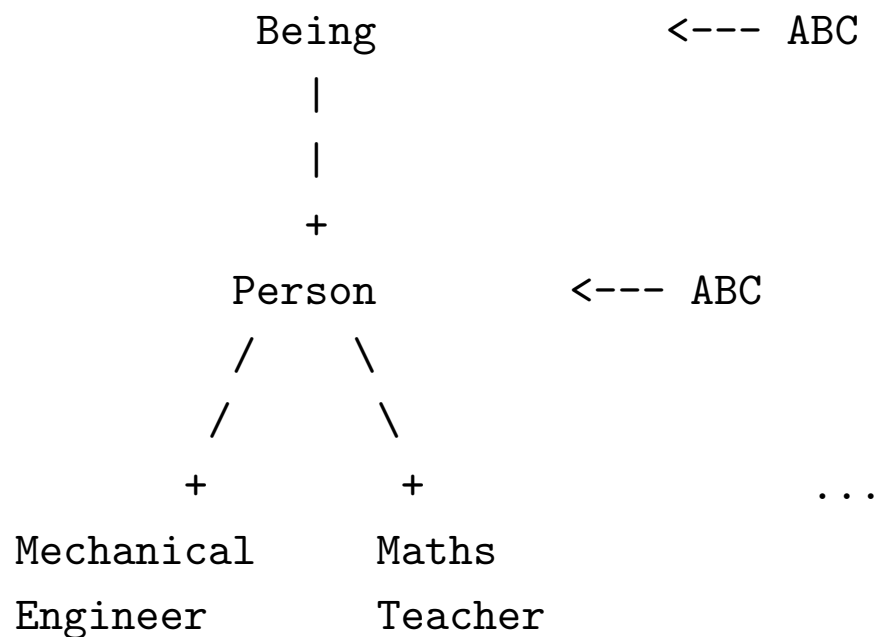
## Αφηρημένες κλάσεις (Abstract base classes)

- Μια συνάρτηση-μέλος μπορεί να δηλωθεί ως πλήρως εικονική (pure virtual).

- Παράδειγμα:

```
virtual void play() const = 0;
```

- Μια κλάση που έχει μια (ή περισσότερες) πλήρως εικονικές συναρτήσεις προσδιορίζεται ως *Αφηρημένη (Βασική) Κλάση* (Abstract Base Class) και δεν μπορούμε να ορίσουμε αντικείμενά της.
- Παράδειγμα



Στο σχήμα αυτό, με ABC συμβολίζουμε το ότι η κλάση είναι αφηρημένη.

- Αν μια παραγόμενη κλάση μιας αφηρημένης δεν ορίζει όλες τις πλήρως εικονικές συναρτήσεις της αφηρημένης κλάσης, τότε και η κλάση αυτή είναι αφηρημένη.

- Οι αφηρημένες κλάσεις χρησιμεύουν μόνο για κληρονομικότητα υπογραφών (signatures) και υλοποίησης –προσφέρουν μια γενική συμπεριφορά την οποία οι παραγόμενες κλάσεις καλούνται να υλοποιήσουν ή/και, ενδεχόμενα, να εξειδικεύσουν.
- Παρόλο που δεν μπορούμε να ορίσουμε αντικείμενα μιας αφηρημένης κλάσης, μπορούμε να ορίσουμε δείκτες σε αφηρημένες κλάσεις και αναφορές σε αφηρημένες κλάσεις, για να πετύχουμε πολυμορφικό χειρισμό αντικειμένων παραγόμενων κλάσεων.
- Αυτός ο χειρισμός μπορεί να αφορά σύνολα αντικειμένων και αποτελεί πολύ συνηθισμένη χρήση των αφηρημένων κλάσεων.

## Παράδειγμα

```
class I {
public:
    virtual void play() const = 0;
};

class W : public I {
public:
    void play() const
    { cout << "W::play" << endl; }
};

int main()
{
    // I instr; instr.play();

    W flute; flute.play();

    I* pflute = &flute;
    pflute->play();
    // pflute->W::play();

    return 0;
}
```

### Αναπαράσταση: πίνακας συναρτήσεων virtual

- Όταν μια κλάση έχει συναρτήσεις virtual αποκτά, από το μεταγλωττιστή, έναν “πίνακα συναρτήσεων virtual” και τα αντικείμενά της αποκτούν ένα δείκτη στον πίνακα αυτό.
- Κάθε κλάση της ιεραρχίας από την πρώτη εμφάνιση συνάρτησης virtual και κάτω αποκτά το δικό της πίνακα συναρτήσεων virtual.
- Για την κάθε συνάρτηση virtual μιας κλάσης, έχει καταχωρηθεί ένας δείκτης πίνακα που αντιστοιχεί στη θέση του πίνακα συναρτήσεων virtual που οδηγεί στον κατάλληλο ορισμό της συνάρτησης για την κλάση αυτή.
- Ο δείκτης πίνακα που αντιστοιχεί σε μια συνάρτηση virtual παραμένει ο ίδιος για όλη την ιεραρχία.



## Αναπαράσταση: παράδειγμα (προσαρμογή από [Lippman96])

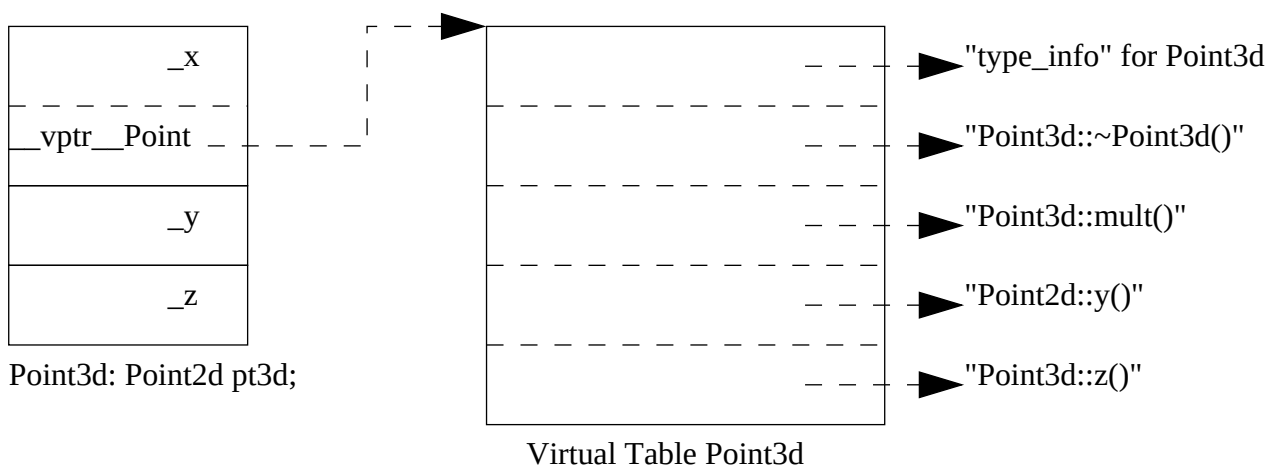
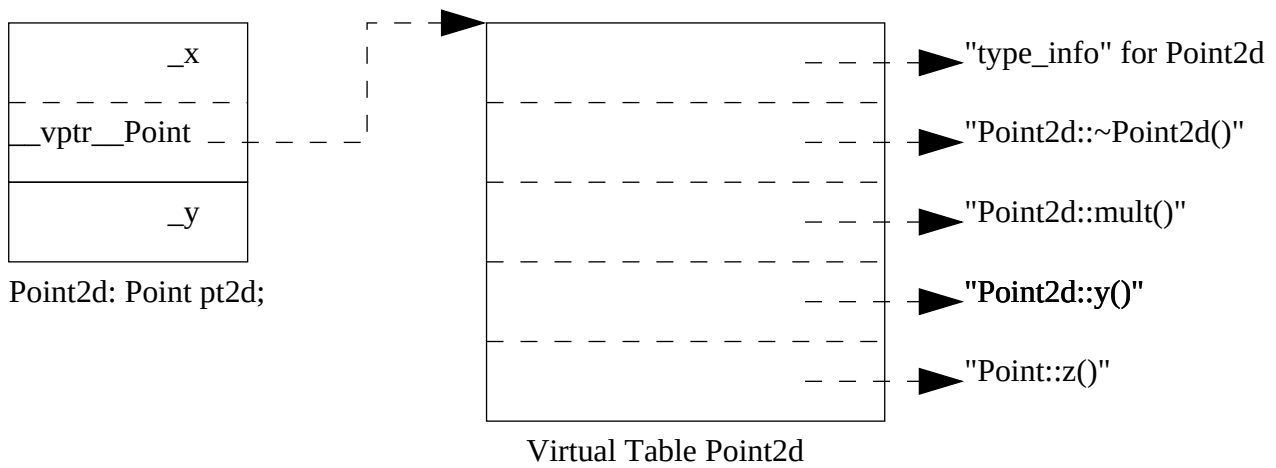
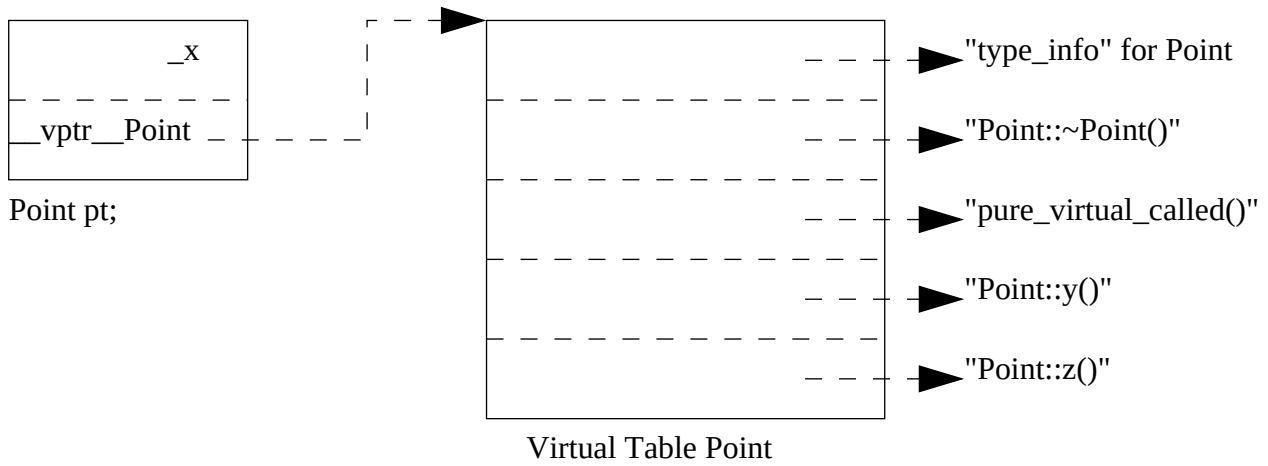
```

class Point {
public:
    virtual ~Point();
    virtual Point& mult( float ) = 0;
// ...other operations...
    float x() const { return _x; }
    virtual float y() const { return 0; }
    virtual float z() const { return 0; }
// ...
protected:
    Point( float x = 0.0 );
    float _x;
};

class Point2d : public Point {
public:
    Point2d( float x = 0.0, float y = 0.0 ) :
        Point( x ), _y( y ) {}
    ~Point2d();
// overridden base class virtual functions
    Point2d& mult( float ) ;
    float y() const { return _y; }
// ...other operations...
protected:
    float _y;
};

class Point3d : public Point2d {
public:
    Point3d( float x = 0.0, float y = 0.0,
             float z = 0.0 ) :
        Point2d( x, y ), _z( z ) {}
    ~Point3d();
// overridden base class virtual functions
    Point3d& mult( float ) ;
    float z() const { return _z; }
// ...other operations...
protected:
    float _z;
};

```



```
Point *pp;
```

```
.....
```

```
pp->z(); => (*pp->__vptr__Point[4])(pp);
```

## ΜΕΤΑΒΑΣΗ ΣΤΗ ΓΛΩΣΣΑ Java

### Ιστορία και Στόχος της Java

- Αρχές δεκαετίας '90, James Gosling, SUN: έργο για ευφυείς συσκευές (πειραματικό), –από ανάγκη υποστήριξης του χώρου– → 1992, καλωδιακή τηλεόραση (εμπορικό) → 1994, έργο για το διαδίκτυο και τον παγκόσμιο ιστό (εκτέλεση από πλοηγούς).
- Αρχικά ονομάστηκε Oak.
- 1995: η πρώτη έκδοση της Java.
- Από το 2009, διατίθεται από την Oracle.
- Νέα τεχνολογία προγραμματισμού για έξυπνες συσκευές (κατανεμημένα περιβάλλοντα - ετερογενείς πλατφόρμες - επικοινωνία μεταξύ προγραμμάτων - επικοινωνία με χρήστη).
- Επιρροές από
  - Σύνταξη: C και C++
  - Όμως λιγότερες δυνατότητες για διαχείριση χαμηλού επιπέδου
  - Σχεδιασμός γλώσσας και αρχιτεκτονικής επηρεασμένος από γλώσσες όπως Eiffel και SmallTalk
  - Περιβάλλον εκτέλεσης: *εικονική μηχανή* (virtual machine) - James Gosling για Pascal στα μεταπτυχιακά του χρόνια.

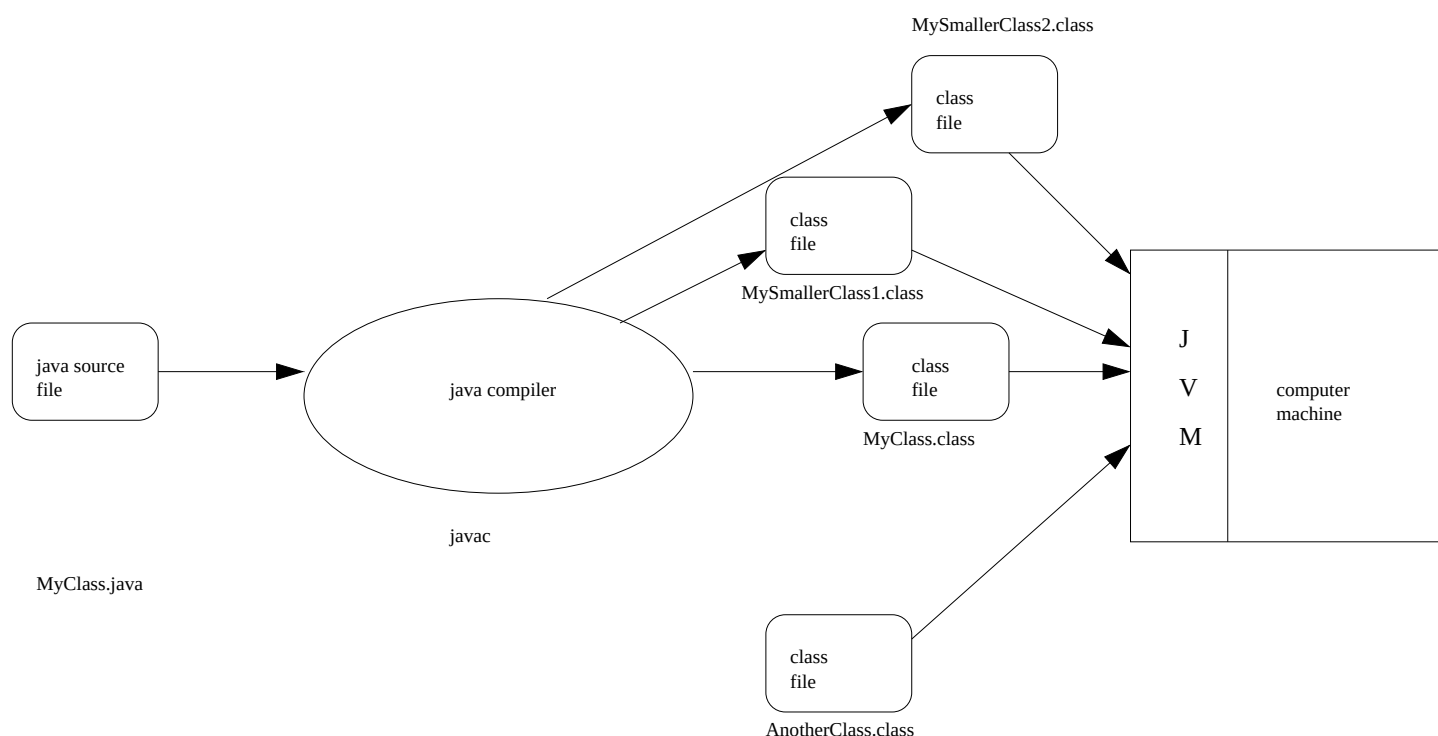
## Αρχές σχεδιασμού

- μεταφερισιμότητα του κώδικα (*write once, run anywhere*)
- αποφυγή σφαλμάτων,
- διευκόλυνση στην ανάπτυξη κατανεμημένων εφαρμογών (ακόμα και) σε ετερογενείς πλατφόρμες,
- διευκόλυνση στην ανάπτυξη φιλικών (γραφικών) διεπαφών ανθρώπου-υπολογιστή. και ασφάλεια.

## Βασικά χαρακτηριστικά της γλώσσας

- Αντικειμενοστραφής προγραμματισμός - τα πάντα οργανωμένα σε κλάσεις.
- Οι κλάσεις αυτές δομούν μια ιεραρχία, η οποία έχει ως βασική κλάση την κλάση `Object`, η οποία προσφέρεται από τη γλώσσα.
- Όλες οι κλάσεις, είτε οι προσφερόμενες από τη γλώσσα είτε εκείνες που ορίζει ο προγραμματιστής, είναι υποκλάσεις της κλάσης `Object`.

## Το Περιβάλλον Εκτέλεσης



- Αρχεία Java πηγαίου κώδικα (.java) –ένα εκ των οποίων πρέπει να περιέχει την *κύρια κλάση*.
- Τα αρχεία αυτά μεταγλωττίζονται από το μεταγλωττιστή (javac) σε *αρχεία κλάσεων* (class file) (.class).
- Κάθε κλάση του πηγαίου κώδικα μεταγλωττίζεται σε ένα αρχείο κλάσης το οποίο περιέχει την αναπαράσταση/πληροφορία για την κλάση αυτή σε *bytecodes* (one-byte opcodes) που είναι εντολές για την *Java Virtual Machine (JVM)*. Το περιεχόμενο ενός αρχείου κλάσης είναι μια ακολουθία από 8-bit bytes.
- Τα bytecodes της Java είναι μια ειδική περίπτωση εντολών *p-code* (“p” or table- “code” ) για αφηρημένες μηχανές.

- Η JVM είναι μια αφηρημένη υπολογιστική μηχανή (abstract computing machine).
- Η JVM δεν σχετίζεται αποκλειστικά με την Java απλά αναγνωρίζει το συγκεκριμένο δυαδικό format, το *class file format*.
- Υπάρχουν κι άλλες γλώσσες που μεταγλωττίζονται σε bytecode για να μπορούν να εκτελεστούν στη JVM, π.χ. η Scala.
- Η πλατφόρμα της Java έχει δύο συστατικά: την JVM και το Application Programming Interface (API)
- Το API είναι μια μεγάλη συλλογή έτοιμου λογισμικού που παρέχει πλήθος λειτουργιών. Αποτελεί την βασική βιβλιοθήκη της γλώσσας. Λογισμικό συναφούς λειτουργικότητας είναι ομαδοποιημένο σε *πακέτα* (packages).
- Integrated Development Environments (IDEs) για Java: π.χ. ECLIPSE, Net Beans.

## Δομή και Μεταγλώττιση Προγραμμάτων Java

- Κάθε πρόγραμμα πρέπει να έχει μια κύρια κλάση.
- Κύρια κλάση είναι μια οποιαδήποτε κλάση που περιέχει μια μέθοδο `main` με την εξής επικεφαλίδα:

```
public static void main(String[] args)
```

Η παράμετρος `args` είναι τύπου `String[]` και περιέχει τα ορίσματα που δίνονται από τη γραμμή εντολής όταν εκτελούμε το πρόγραμμα.

- Η κλήση της παραπάνω μεθόδου αποτελεί το αρχικό σημείο της εκτέλεσης του προγράμματος αυτού.



Το ελάχιστο πρόγραμμα Java είναι για παράδειγμα το:

```
// file: java.java
class NameDoesNotMatter {
    public static void main(String[] args) {}
}
```

Για να έχουμε και κάποια έξοδο μπορούμε να γράψουμε το εξής πρόγραμμα:

```
class FirstApp {
    public static void main(String[] args) {
        System.out.println("Hello World!");
    }
}
```

- Οι κλάσεις (τύποι) μπορούν να ομαδοποιηθούν σε πακέτα.
- Η ορατότητα των κλάσεων μπορεί να περιοριστεί μέσα στο πακέτο ή να είναι `public`.
- Εάν ένα αρχείο περιέχει πολλούς ορισμούς κλάσεων, μόνο μια από αυτές μπορεί να δηλωθεί `public` (και στην περίπτωση που το αρχείο αποτελεί `compilation unit` ενός πακέτου πρέπει να έχει το ίδιο όνομα με το αρχείο).
- Αν δεν δηλωθεί το περιεχόμενο του αρχείου ως μέρος ενός πακέτου (ύπαρξη δήλωσης `package` στη πρώτη γραμμή μη σχολίου), τότε θεωρείται ότι αυτό είναι μέρος ενός `unnamed` πακέτου.
- Θεωρούνται ορατά και μπορούν να χρησιμοποιηθούν ονόματα από διάφορα πακέτα του `API`.

## Γενικές Παρατηρήσεις για τα Προγράμματα Java

### Πακέτα και Κλάσεις:

- Τα πακέτα ορίζουν εμβέλειες με όνομα και περιέχουν ορισμούς κλάσεων.
- Τα δεδομένα των κλάσεων και τα δομικά στοιχεία των αντικειμένων τους αναφέρονται ως πεδία.
- Η συμπεριφορά των κλάσεων και των αντικειμένων τους υλοποιούνται από συναρτήσεις που αποκαλούνται μέθοδοι.
- Μπορούμε να ορίσουμε αντικείμενα των κλάσεων, είτε ως τιμή των μεταβλητών των πεδίων είτε ως τιμή μεταβλητών στις μεθόδους.
- Διάφορες κατηγορίες εμφωλευμένων κλάσεων.

### Μέλη Στιγμιοτύπων και Μέλη Κλάσεων:

- Τα πεδία και οι μέθοδοι μιας κλάσης, μπορούν να σχετίζονται με κάποιο αντικείμενο της κλάσης -μέλη στιγμιοτύπου (instance members)-, ή να είναι ανεξάρτητα από αντικείμενο και να αφορούν την κλάση και μόνο -μέλη κλάσης (class members)-.
- Αν πρόκειται για πεδία, τα μέλη στιγμιοτύπου συμμετέχουν στο “κτίσιμο” ενός αντικειμένου της κλάσης
- Αν πρόκειται για μεθόδους, εφαρμόζονται απαραίτητα σε αντικείμενο.
- Τα μέλη κλάσης συμπεριφέρονται όπως τα στατικά μέλη κλάσεων της C++ και δηλώνονται με τη λέξη-κλειδί `static`.

## Ως προς τι Δηλώνουμε Ορατότητα;

- Δηλώνουμε ορατότητα τόσο για τα ονόματα των μελών των κλάσεων, όσο και για τα ονόματα των ίδιων των κλάσεων.
- Οι διαβαθμίσεις ορατότητας περιορίζουν τη χρήση των ονομάτων,
  - είτε ως προς τις άλλες κλάσεις
  - είτε ως προς τα άλλα πακέτα (έχουμε τη δυνατότητα να περιορίσουμε τη χρήση κλάσεων ή/και μελών τους, μόνο στην εμβέλεια του πακέτου μέσα στο οποίο έχουν οριστεί).

## Κληρονομικότητα:

- Όλες οι κλάσεις είναι οργανωμένες ιεραρχικά, μέσω κληρονομικότητας.
- Η κλάση που βρίσκεται στη ρίζα της ιεραρχίας είναι η κλάση `Object`.
- Κάθε κλάση ή τύπος πίνακα είναι υποκλάση της κλάσης `Object`.
- Κάθε κλάση μπορεί να δηλωθεί ως άμεση υποκλάση το πολύ μιας κλάσης (χρησιμοποιώντας τη λέξη-κλειδί `extends`) –αν δεν δηλωθεί ρητά υπερκλάση, υπονοείται ότι η κλάση είναι άμεση υποκλάση της κλάσης `Object`.
- Για να υποστηριχθεί κάποιος μορφής πολλαπλή κληρονομικότητα με “καθαρό” τρόπο, η Java προσφέρει έναν τύπο, εναλλακτικό των κλάσεων, τους τύπους διεπαφής (`interfaces`).
- Στη γενική περίπτωση, οι τύποι διεπαφής περιέχουν μόνο δηλώσεις μεθόδων. Κάθε κλάση που μπορεί να θεωρηθεί ότι “αντιπροσωπεύεται” από τις προδιαγραφές που εκφράζουν οι δηλώσεις αυτές, μπορεί να δηλώσει ότι υλοποιεί τον τύπο (η λέξη-κλειδί που χρησιμοποιείται είναι `implements`) και οφείλει να ορίσει τις μεθόδους του.
- Μια κλάση μπορεί να υλοποιήσει πάνω από έναν τύπο διεπαφής.

## Η Χρήση της Λέξης-κλειδιού final: “τελικό” / “τελεία και παύλα” !

- Ως προσδιοριστής σε μια κλάση, απαγορεύει να οριστούν υποκλάσεις της κλάσης αυτής.
- Ως προσδιοριστής σε μια μέθοδο κλάσης, προσδιορίζει ότι η μέθοδος αυτή απαγορεύεται να επαναοριστεί στις υποκλάσεις της κλάσης.
- Ως προσδιοριστής σε ένα πεδίο κλάσης, καθορίζει ότι η τιμή του πεδίου είναι σταθερή –αν το πεδίο είναι *αναφορά* (reference) σε αντικείμενο, το αντικείμενο μπορεί να αλλάξει τιμή (δεν γίνεται άμεσα δήλωση σταθερών αντικειμένων: πρέπει να οριστεί κατάλληλη κλάση με μη μεταβαλλόμενα δεδομένα).
- Ως προσδιοριστής μεταβλητών γενικότερα, δηλώνει πάλι σταθερά.

...

## Η γενική μορφή του περιεχομένου μιας απλής κλάσης

Σε μια απλή μορφή, μια κλάση Java περιμένουμε να έχει:

- κάποια πεδία
- κάποιες μεθόδους
- κάποιες μεθόδους (συναρτήσεις) αρχικοποίησης

Π.χ.

```
class MySimpleClass {
    private int data1;        // field
    private int data2;        // field

    void myMethod() {         // method
        data1++; data2++;
        System.out.println("The data1 value is: " +
                            data1 +
                            " The data2 value is: " +
                            data2);
    }

    MySimpleClass() {}        // constructor (default)

    MySimpleClass(int d1, int d2) { // constructor
        data1 = d1; data2 = d2;
    }
}
```

Μπορούμε να δηλώσουμε για μια κλάση ότι απαγορεύεται να δημιουργηθούν αντικείμενά της, είναι δηλαδή *αφηρημένη κλάση*. Αυτό γίνεται προσδιορίζοντάς την ως `abstract`.

## Διαβαθμίσεις Ορατότητας - modifiers για τα μέλη κλάσεων

- Τα πεδία, οι μέθοδοι και τα άλλα μέλη μιας κλάσης, σε σχέση με την ορατότητα έξω από την εμβέλεια της κλάσης, μπορούν να δηλωθούν ως:
  - **public**: μπορούν να προσπελαστούν και έξω από το πακέτο που ορίζεται η κλάση
  - **private**: μπορούν να χρησιμοποιηθούν μόνο από την κλάση μέσα στην οποία ορίζονται
  - **protected**: μπορούν να προσπελαστούν μέσα από το πακέτο στο οποίο ορίζεται η κλάση και από τις υποκλάσεις της κλάσης μέσα στην οποία ορίζονται -ακόμα και αν οι υποκλάσεις αυτές είναι ορισμένες σε άλλο πακέτο
  - χωρίς προσδιορισμό: μπορούν να προσπελαστούν μόνο μέσα από το πακέτο στο οποίο ορίζεται η κλάση
- modifiers στις δηλώσεις πεδίων μιας κλάσης:
  - **static**: το πεδίο είναι μια μεταβλητή εμβέλειας κλάσης
  - **final**: το πεδίο δεν μπορεί να αλλάξει τιμή (άμεσα) μετά την αρχικοποίηση
- modifiers στις δηλώσεις μεθόδων μιας κλάσης:
  - **static**: η μέθοδος δεν χρειάζεται να εφαρμοστεί σε αντικείμενο
  - **final**: η μέθοδος δεν μπορεί να επαναοριστεί σε υποκλάση
  - **abstract**: η μέθοδος δηλώνεται μόνο (δεν έχει υλοποίηση) -και η κλάση πρέπει να δηλωθεί ως **abstract**.



## Μεταβλητές στην Java - Τύποι

Οι τύποι χωρίζονται σε δύο κατηγορίες:

- *πρωτογενείς τύπους* (primitive data types) (boolean και numeric) και
- *τύπους αναφοράς* (reference types):
  1. *τύπος κλάσης* (class type)
  2. *τύπος πίνακα* (array type)
  3. *τύπος διεπαφής* (interface type)

Οι μεταβλητές μπορεί να είναι:

- απλές μεταβλητές πρωτογενών τύπων
- *μεταβλητές αναφοράς* (reference variables), των οποίων ο τύπος είναι ένας τύπος αναφοράς.

Παρατηρήσεις:

- Τα πεδία αποκτούν αυτόματα αρχικές τιμές.
  - Στην περίπτωση που ο τύπος τους είναι πρωτογενής, η τιμή είναι η μηδενική τιμή του τύπου τους.
  - Στην περίπτωση μεταβλητών αναφοράς, η τιμή είναι `null`.
- Αν οι μεταβλητές δεν είναι μεταβλητές-πεδία, τότε δεν αποκτούν αρχικές τιμές.

**Κλάση String:** για την αναπαράσταση συμβολοσειρών –τα αντικείμενα της κλάσης δεν πρέπει να εκλαμβάνονται ως πίνακες χαρακτήρων.

## Απλές μεταβλητές και Μεταβλητές Αναφοράς

- Δήλωση μεταβλητής:

type t

Δηλώνει στο μεταγλωττιστή ότι το t πρόκειται να χρησιμοποιηθεί σαν όνομα δεδομένων τύπου type.

- Αν πρόκειται για πρωτογενή τύπο τότε κατά τη δήλωση αυτή δεσμεύεται ο απαιτούμενος χώρος για την μεταβλητή του τύπου.
- Αν το type είναι κλάση, με την παραπάνω δήλωση δεν δημιουργείται αντικείμενο.
- Δημιουργία στιγμιοτύπου/αντικειμένου μιας κλάσης γίνεται με χρήση του τελεστή new. Η κλήση του δεσμεύει στη μνήμη τον κατάλληλο χώρο για το αντικείμενο και επιστρέφει μια αναφορά σε αυτό. Επίσης, καλεί την (κατάλληλη) συνάρτηση αρχικοποίησης.

- Δήλωση μεταβλητής αναφοράς και δημιουργίας αντικειμένου ανατεθειμένου σε αυτή:

```
type t = new type(5,10,20);
```

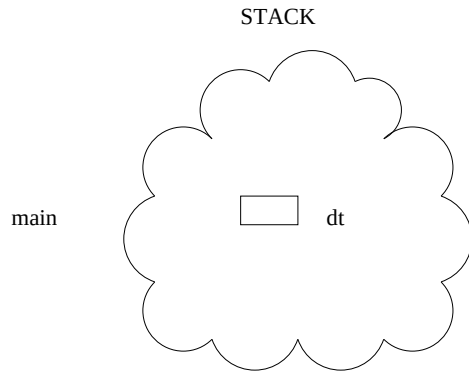
- Μια μεταβλητή τύπου `type` μπορεί να πάρει τιμή μια αναφορά σε ένα στιγμιότυπο του τύπου `type` ή υποκλάσης του ή να έχει την τιμή `null`.
- Οι μεταβλητές αναφοράς:
  - Είναι δείκτες σε αντικείμενο του τύπου τους (ή υποκλάσης του) που πάντα γίνονται dereference (όπως οι αναφορές της C++)
  - δεν είναι σταθεροί, μπορεί να “δείξουν” σε διαφορετικό αντικείμενο (αντίθετα με τις αναφορές της C++)
  - και μπορεί να μην αναφέρονται σε κανένα αντικείμενο (οπότε ενδέχεται να έχουν τη τιμή `null`).
- Αν μια μεταβλητή που έχει δηλωθεί `final` περιέχει αναφορά σε αντικείμενο, η τιμή (κατάσταση) του αντικειμένου μπορεί να αλλάξει αλλά η μεταβλητή θα αναφέρεται πάντα στο ίδιο αντικείμενο.

```
class Date{ int day; int month; int year}
```

---

```
main:
```

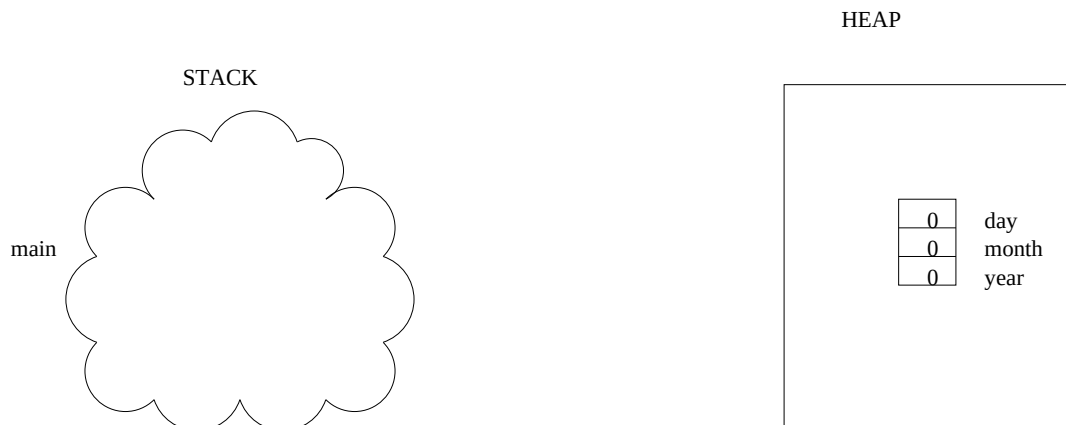
```
Date dt ;
```



---

```
main:
```

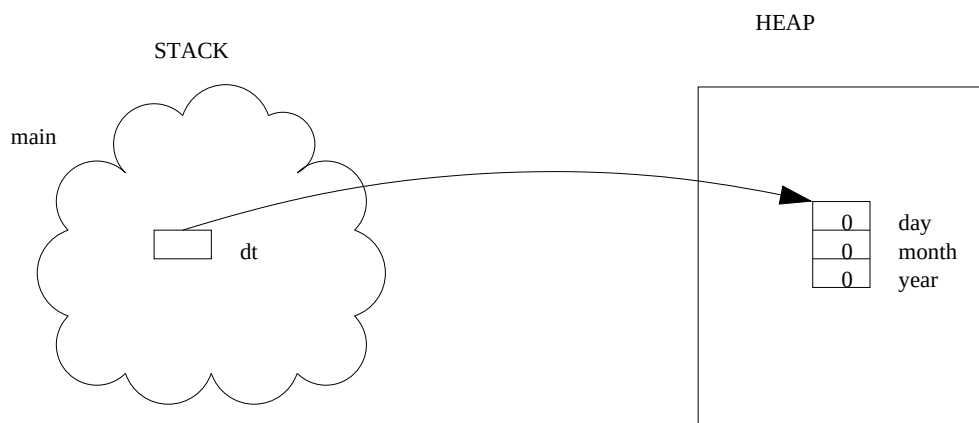
```
new Date();
```



---

```
main:
```

```
Date dt = new Date();
```



## Πώς γίνεται κάποια “δουλειά”

- Χρήση τελεστών - κατασκευή εκφράσεων
- Χρήση εντολών
- Χρήση blocks
- Συνοπτικά: à la C
- Προσοχή: όχι δυνατότητα υπερφόρτωσης των τελεστών

## Αρχικοποίηση Αντικειμένων στην Java

### Μέθοδοι Αρχικοποίησης:

- Οι μέθοδοι αρχικοποίησης καλούνται αυτόματα με τη χρήση του τελεστή `new`.
- Αν δεν έχει οριστεί κάποια μέθοδος αρχικοποίησης σε μια κλάση, η Java προσφέρει μια default μέθοδο αρχικοποίησης –αυτή καλεί τη μέθοδο αρχικοποίησης της γονεϊκής κλάσης.
- Μέσα από το σώμα μιας μεθόδου αρχικοποίησης, μπορούμε να καλέσουμε μια άλλη με χρήση της λέξης-κλειδιού `this`, προσδιορίζοντας μέσω παραμέτρων ποια εννοούμε, π.χ.

```
Dish() { this(600,"Variety"); }
```

### Αρχικοποίηση Πεδίων κατά τη Δήλωσή τους:

- Αρχικοποίηση των πεδίων μιας κλάσης μπορεί να γίνει και κατά τη δήλωσή τους.
- Αυτό μπορεί να γίνει είτε με απευθείας ανάθεση αρχικών τιμών, π.χ.

```
private int calories = 1000;
```

```
private String name = "Junk food";
```

είτε μέσω χρήσης `final` μεθόδου (*final method initialization*) –χρήσιμο αν οι υποκλάσεις θέλουν να χρησιμοποιήσουν τη μέθοδο.

- Αν δεν έχουν δοθεί αρχικές τιμές, τα πεδία μιας κλάσης αρχικοποιούνται με τις μηδενικές τιμές του τύπου τους.

## Παράδειγμα

```
// Multiple constructors
```

```
class App {
    public static void main(String[] args) {
        Dish d = new Dish();
        Dish d1 = new Dish("hot dog");
        Dish d2 = new Dish(100, "giaourti");
    }
}
```

```
class Dish {
    private int calories = 1000;
    private String name = "Junk food";
```

```
/*
    Dish() {
        System.out.println("The new dish named: " + name +
                           " contains " + calories + " calories!");
    }
*/
/**
// If default construction with default values needed
    Dish() {
        this(400,"LightFood");
        System.out.println("Again: The new dish named: " + name +
                           " contains " + calories + " calories!");
    }
//*/
```

```
Dish(String nam) {  
    name = nam;  
    System.out.println("The new dish named: " + name +  
        " contains " + calories + " calories!");  
}
```

```
Dish(int cal, String nam) {  
    calories = cal;  
    name = nam;  
    System.out.println("The new dish named: " + name +  
        " contains " + calories + " calories!");  
}
```

```
void eat() {  
    System.out.println("Eating " + name );  
}  
int getCalories() {  
    return calories;  
}  
}
```



### ***Block Αρχικοποίησης:***

- Κώδικας μέσα σε ένα block που ορίζεται μέσω αγκύλων, { }, γραμμένο απευθείας στον ορισμό της κλάσης.
- Προκειμένου να μην επαναλαμβάνουμε κώδικα κοινό για κάθε μέθοδο αρχικοποίησης στο σώμα της καθεμίας –ο μεταγλωττιστής αντιγράφει τον κώδικα του block στην αρχή οποιασδήποτε μεθόδου αρχικοποίησης κατά τη δημιουργία ενός αντικειμένου και εκτελείται πριν από τον κώδικα της μεθόδου.

## Παράδειγμα

```

class App {
    public static void main(String[] args) {
        Dish d = new Dish(100,"giaourti");
        d.eat();
        System.out.println("Actually you got: " +
                           d.getCalories() + " calories!");
    }
}

class Dish {
    int calories = 2000;
    String name = "What you have to eat";

    // Block initialization
    { System.out.println("The new dish named: " + name +
                        " contains " + calories + " calories!");
      // NOT WHAT WE WANTED - it is copied in the beginning

    Dish(int cal, String nam) {
        calories = cal;
        name = nam;
        System.out.println("The new dish named: " + name +
                           " contains " + calories + " calories!");
    }

    //...
}

```

## Βήματα κατά τη Διαδικασία Αρχικοποίησης Στιγμιοτύπων Κλάσεων

Κατά τη δημιουργία ενός αντικειμένου/στιγμιοτύπου μιας κλάσης, η οποία δεν είναι δηλωμένη ρητά ως υποκλάση, τα βήματα κατά την αρχικοποίησή του έχουν ως εξής:

1. Αρχικά, δίνονται στα πεδία του οι default μηδενικές τιμές των τύπων τους.
2. Στη συνέχεια, γίνεται υπολογισμός των ορισμάτων της μεθόδου αρχικοποίησης που προσδιορίζεται μέσω του τελεστή `new`).
3. Κατόπιν, στην περίπτωση που υπάρχει εμφάνιση της λέξης-κλειδιού `this`, οδηγούμαστε στο Βήμα 2 για τη μέθοδο αρχικοποίησης που προσδιορίζεται από τη χρήση του `this`.
4. Όταν δεν υπάρχει εμφάνιση της λέξης-κλειδιού `this`, γίνεται αρχικοποίηση των πεδίων σύμφωνα με τις δηλώσεις τους, αν περιέχουν ρητές αρχικοποιήσεις και, στη συνέχεια, εκτελείται ο κώδικας από τα block αρχικοποίησης με τη σειρά που εμφανίζονται στον ορισμό της κλάσης.
5. Ακολουθεί η εκτέλεση του κώδικα των μεθόδων αρχικοποίησης με αντίστροφη σειρά. Δηλαδή, η εκτέλεση ξεκινά από το σώμα εκείνης που προσδιορίστηκε πιο πρόσφατα και καταλήγει στο σώμα εκείνης που χρησιμοποιήθηκε στην έκφραση που εμφανίστηκε ο τελεστής `new` για τη δημιουργία του αντικειμένου.

## Κληρονομικότητα στην Java

- Η Java προσφέρει μια έτοιμη ιεραρχία κλάσεων μέσω του API.
- Ο προγραμματιστής ορίζει τις δικές του κλάσεις ως υποκλάσεις κλάσεων –όλες οι κλάσεις είναι υποκλάσεις της κλάσης `Object`.
- Κλάσεις - Υποκλάσεις (`isa` → `extends`) –αν δεν δηλώνεται υπερκλάση, η κλάση θεωρείται άμεση υποκλάση της κλάσης `Object`.

`class B extends A`

ορίζουμε μια κλάση B ως υποκλάση της κλάσης A,  
εννοώντας ότι

`class B isa class A`

- Όταν καλείται μια μη στατική μέθοδος, πάντα γίνεται `method dispatch` και εκτελείται η μέθοδος που είναι ορατή στην κλάση του αντικειμένου –εικονικές συναρτήσεις, έχουμε αυτόματα πολυμορφισμό.
- Δεν μπορούμε να χρησιμοποιούμε ονόματα κλάσεων για να έχουμε άμεση πρόσβαση σε μεθόδους τους –εκτός αν πρόκειται για στατικές μεθόδους.
- Μέσα στο σώμα μιας μεθόδου της υποκλάσης, μπορούμε να χρησιμοποιήσουμε τη λέξη-κλειδί `super` ώστε να κληθεί η επαναορισμένη μέθοδος της άμεσης υπερκλάσης της –αυτή είναι και η μόνη περίπτωση πρόσβασης σε επαναορισμένη μη στατική μέθοδο σε μια κλάση.

## Παραδείγματα

```
class App {  
    public static void main(String[] args) {  
        A var = new B();  
        var.bla();  
    }  
}
```

```
class A {  
    void bla() { System.out.println("Executing the A bla()"); }  
}
```

```
class B extends A {  
    void bla() { System.out.println("Executing the B bla()"); }  
}
```

Αποτέλεσμα:

Executing the B bla()

```
class App {  
    public static void main(String[] args) {  
        A var = new B();  
        var.bla();  
    }  
}  
  
class A {  
    void bla() { System.out.println("Executing the A bla()"); }  
}  
  
class B extends A {  
    void bla() {  
        super.bla();    // Calling superclass bla()  
        System.out.println("Executing the B bla()");  
        super.bla();    // Calling superclass bla()  
    }  
}
```

Αποτέλεσμα:

```
Executing the A bla()  
Executing the B bla()  
Executing the A bla()
```

Όροι *overriding* και *hiding*: αλλαγή ορατότητας λόγω επαναορισμού μεθόδων

***overriding***: μεταξύ μη στατικών μεθόδων που επαναορίζονται σε υποκλάση: το αν καλείται η μέθοδος της υποκλάσης ή της υπερκλάσης εξαρτάται από τον τύπο του αντικειμένου πάνω στο οποίο εφαρμόζεται η μέθοδος

***hiding***: μεταξύ στατικών μεθόδων που επαναορίζονται σε υποκλάση –σε περίπτωση που η κλήση γίνεται με χρήση μεταβλητής αναφοράς, το αν καλείται η μέθοδος της υποκλάσης ή της υπερκλάσης εξαρτάται από τον τύπο της μεταβλητής (και όχι του αντικειμένου).

Όρος *shadowing*: αφορά την αλλαγή κατάστασης ορατότητας μεταξύ μεταβλητών στενότερης και ευρύτερης εμβέλειας.

## Αφηρημένες Κλάσεις

- Χρήση κυρίως ως κοινές προδιαγραφές μιας γενικής κατηγορίας κλάσεων και αποφυγή επανάληψης κώδικα.
- Χρήση της λέξης-κλειδιού `abstract` για προσδιορισμό μιας κλάσης ως αφηρημένη. Π.χ. αν έχουμε:
 

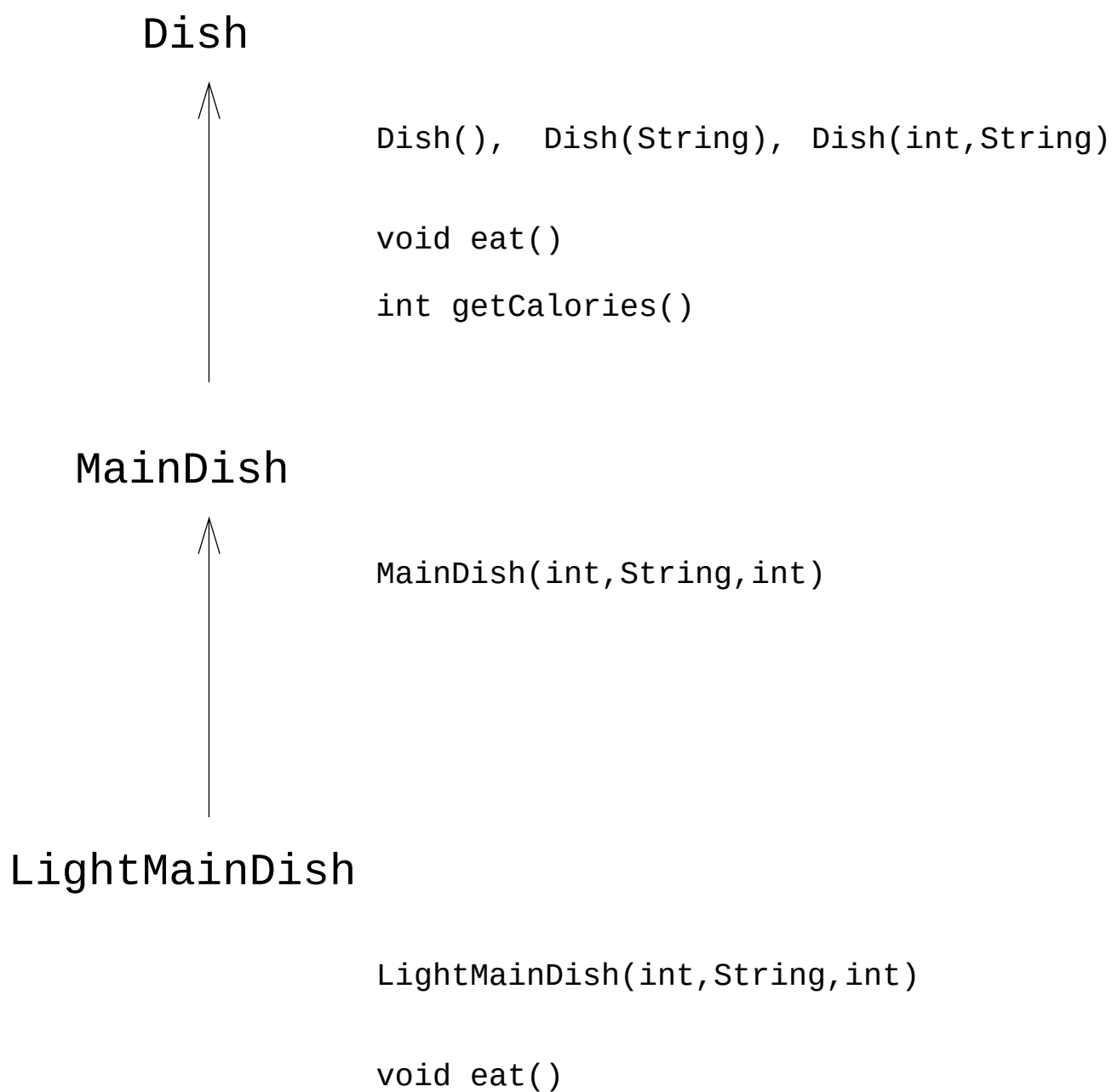
```
abstract class AbsClas{... }
```

 απαγορεύεται να δημιουργήσουμε αντικείμενά της `AbsClas` –η εντολή `new AbsClas()`; θα προκαλέσει σφάλμα μεταγλώττισης.
- Στο σώμα μιας αφηρημένης κλάσης έχουμε κώδικα που θα περιείχε μια κοινή κλάση.
- Τα μέλη μπορεί να είναι οποιασδήποτε κατηγορίας (στατικά, μη-στατικά κλπ.) και ορατότητας.
- Επιπλέον, μπορούμε να έχουμε μεθόδους των οποίων δίνουμε μόνο το πρωτότυπό τους και όχι το σώμα του ορισμού τους, *αφηρημένες μεθόδους* –πρέπει να προσημάνουμε τη δήλωση της μεθόδου με τη λέξη-κλειδί `abstract`.
- Μια κλάση με αφηρημένες μεθόδους υποχρεωτικά πρέπει να δηλωθεί ως `abstract`. Π.χ.
 

```
abstract class AbsClas {
    abstract void commonMethod(); }
```
- Υποκλάσεις αφηρημένων κλάσεων με αφηρημένες μεθόδους πρέπει είτε να τις ορίζουν είτε να δηλώνονται ως αφηρημένες –μια αφηρημένη μέθοδος δεν μπορεί να δηλωθεί ως `final`.
- Μια κλάση είναι άμεση υποκλάση ακριβώς μίας κλάσης, ανεξάρτητα αν αυτή είναι αφηρημένη ή όχι.



Ένα παράδειγμα ιεραρχίας κλάσεων



## Διαδικασία Αρχικοποίησης Στιγμιότυπων Κλάσεων: η Γενική Περίπτωση

- Όταν δημιουργείται ένα στιγμιότυπο μιας κλάσης, δεσμεύεται χώρος ώστε να χωράνε όλα τα μη στατικά πεδία του –οι μεταβλητές στιγμιότυπου της κλάσης του και των υπερκλάσεων της.
- Τα πεδία αυτά αρχικοποιούνται με τις default τιμές του τύπου τους.
- Στη συνέχεια, εκτελείται η παρακάτω διαδικασία:
  1. Γίνεται υπολογισμός και ανάθεση ορισμάτων στις παραμέτρους της μεθόδου αρχικοποίησης που χρησιμοποιείται (αρχικά εκείνης που χρησιμοποιείται μέσω του τελεστή new).
  2. Αν το σώμα της μεθόδου αρχικοποίησης ξεκινά με κλήση (άλλης) μεθόδου αρχικοποίησης της κλάσης του στιγμιότυπου (χρησιμοποιώντας τη λέξη-κλειδί `this`), εφαρμόσε τη διαδικασία για τη μέθοδο αυτή από το βήμα 1 και, κατόπιν, μεταπήδησε στο βήμα 5 για την αρχική μέθοδο.
  3. Αν δεν ισχύει το παραπάνω και η κλάση δεν είναι η `Object`, τότε η εκτέλεση του σώματος της μεθόδου αρχικοποίησης ξεκινά με κλήση μεθόδου αρχικοποίησης της υπερκλάσης (είτε έμμεσα είτε άμεσα χρησιμοποιώντας τη λέξη-κλειδί `super`) ξεκινώντας τη διαδικασία αρχικοποίησης γι' αυτήν τη μέθοδο αρχικοποίησης από το βήμα 1.

4. Εκτελούνται οι αρχικοποιήσεις των πεδίων στιγμιοτύπου και οι συναρτήσεις αρχικοποίησης για τα πεδία του στιγμιοτύπου –τηρείται η σειρά με την οποία εμφανίζονται στο κείμενο του ορισμού της κλάσης. Εκτελούνται τα block αρχικοποίησης.
  5. Εκτελείται το υπόλοιπο σώμα της μεθόδου αρχικοποίησης.
- Μετά την παραπάνω διαδικασία αρχικοποίησης, επιστρέφεται η αναφορά στο αντικείμενο αυτό.

## Μεταβλητές Αναφοράς: Αναθέσεις και Χρήση σε Μεθόδους Στιγμιοτύπου

- Μια μεταβλητή αναφοράς
  - μπορεί να αναφερθεί σε αντικείμενο της κλάσης της —και σε αντικείμενο οποιασδήποτε υποκλάσης της—, και
  - μπορεί να πάρει την τιμή μιας άλλης μεταβλητής αναφοράς της κλάσης της —ή υποκλάσης της—, μέσω ανάθεσης.
- Μετά από μια επιτρεπτή ανάθεση, η μεταβλητή χάνει την προηγούμενη τιμή. Αν αναφερόταν σε αντικείμενο και στο αντικείμενο αυτό δεν αναφέρεται καμία άλλη μεταβλητή αναφοράς, αυτό είναι υποψήφιο για “καταστροφή” και ανάκτηση του χώρου του κατά την περισυλλογή σκουπιδιών.
  - Αν το δεξί μέλος της ανάθεσης είναι αντικείμενο, η μεταβλητή πλέον αναφέρεται στο αντικείμενο αυτό.
  - Αν το δεξί μέλος της ανάθεσης είναι μια άλλη μεταβλητή αναφοράς η οποία αναφέρεται σε αντικείμενο, η μεταβλητή πλέον αναφέρεται και εκείνη στο αντικείμενο αυτό.
- Στις μεταβλητές αναφορών μπορούμε να εφαρμόσουμε μόνο μεθόδους με όνομα ορατό μέσα από την κλάση τους —δεν αρκεί η μέθοδος να έχει όνομα ορατό στην κλάση του αντικειμένου, πρέπει το όνομα να είναι ορατό και στην κλάση της μεταβλητής.
- Κατά την εφαρμογή μιας μεθόδου στιγμιοτύπου, θα κληθεί η μέθοδος που είναι ορατή στην κλάση του αντικειμένου στο οποίο αναφέρεται η μεταβλητή αναφοράς.

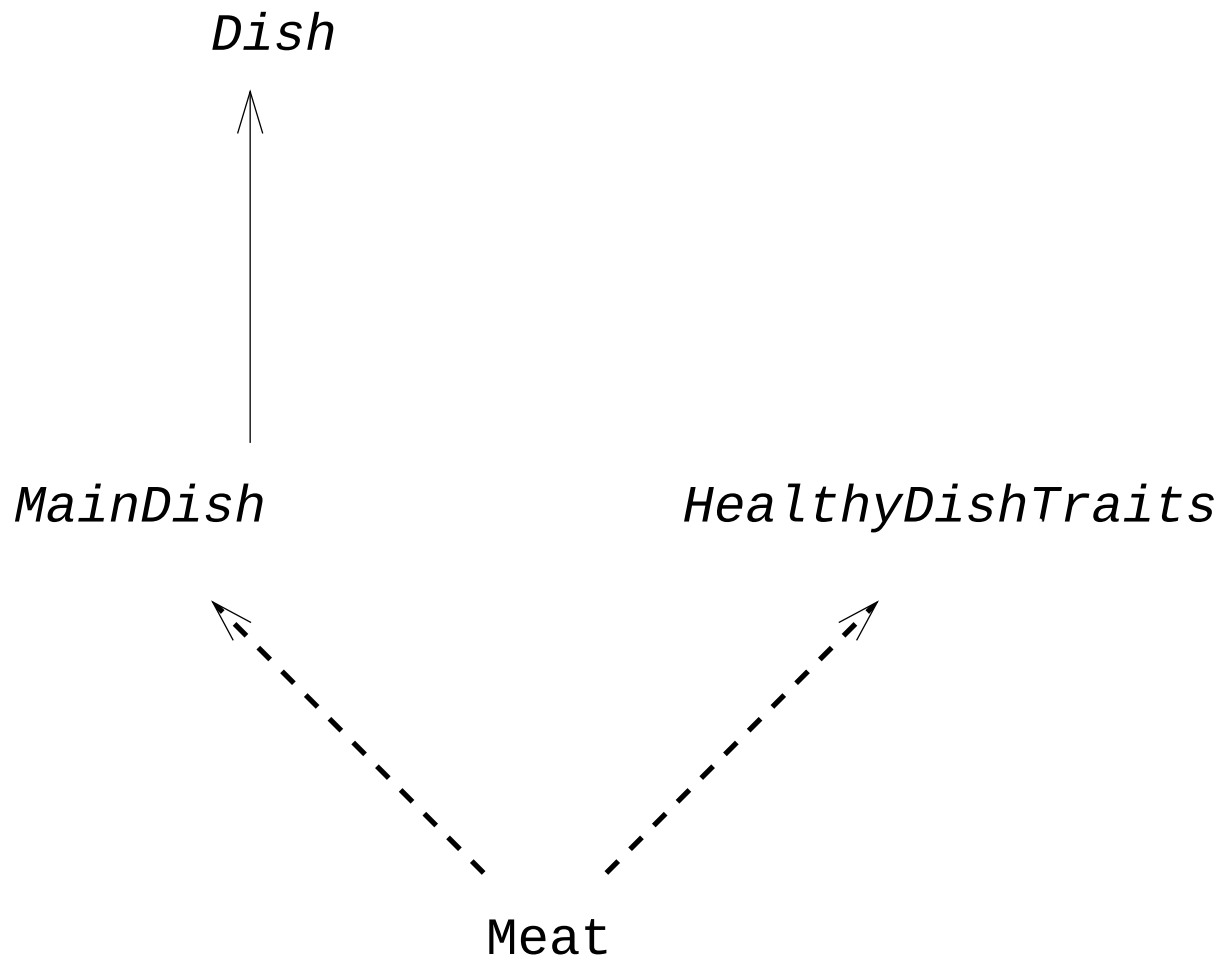
## Πολλαπλή Κληρονομικότητα - Τύπος Διεπαφής

- Ειδική κατηγορία τύπων που απλά αποτελούν προδιαγραφή τύπων.
- Χρήση της λέξης-κλειδί `interface`, αντί για `class`, στη δήλωση.
- Δεν μπορούμε να ορίσουμε αντικείμενα των τύπων διεπαφής.
- Ένας τύπος διεπαφής μπορεί να συνδέεται άμεσα μέσω κληρονομικότητας με παραπάνω από έναν τύπους διεπαφής.
- Στην αρχική τους εκδοχή, οι τύποι διεπαφής αποτελούντο μόνο από πεδία και προδιαγραφές μεθόδων.
- Τα πεδία των τύπων διεπαφής θεωρούνται: `public`, `static` και `final`.
- Οι μέθοδοί τους, στη γενική περίπτωση, θεωρούνται `public` και `abstract` –δίνουμε μόνο την επικεφαλίδα τους και όχι σώμα.
- Οι τύποι διεπαφής υλοποιούνται από τύπους κλάσης, χρησιμοποιώντας τη λέξη-κλειδί `implements`.
- Στην περίπτωση αυτή, η κλάση πρέπει να ορίσει όλες τις μεθόδους του τύπου διεπαφής.
- Μια κλάση μπορεί να υλοποιεί οποιονδήποτε αριθμό από τύπους διεπαφής, δίνοντας ορισμούς στις μεθόδους τους.
- Οι τύποι διεπαφής που υλοποιεί απαγορεύεται να έχουν προδιαγραφές μεθόδων που διαφέρουν μόνο στον τύπο επιστροφής.
- Μετά την έκδοση SE8 της Java, υπάρχουν αρκετές αλλαγές στις δυνατότητες των τύπων διεπαφής.

## Παραδείγματα

```
interface Interf {  
    int CONST = 100;  
    void interfMethod();  
}
```

```
interface SubInterf extends Interf {  
    int ANOTHER_CONST = 200;  
    void subinterfMethod();  
}
```



```
interface MainDish extends Dish
```

```
class Meat implements MainDish, HealthyDishTraits
```

## Στατικά Μέλη στην Java

- Τα στατικά μέλη, μέλη κλάσης, είναι υψίστης σημασίας για την Java, μια που δεν υπάρχει “χώρος” για να γράψουμε κώδικα εκτός κλάσεων.
- Κάθε τι σχετίζεται με την εμβέλεια μιας κλάσης, οπότε, αν κάποια μεταβλητή δεν αποτελεί πεδίο στιγμιοτύπου και μια μέθοδος δεν αποτελεί μέρος της συμπεριφοράς των αντικειμένων της κλάσης, θα οριστεί μέσα σε μια κλάση και θα χαρακτηριστεί ως `static` για την κλάση αυτή.
- Ένα μέλος κλάσης μπορεί να προσπελαστεί, είτε χρησιμοποιώντας το όνομα της κλάσης απευθείας είτε μέσω μιας μεταβλητής αναφοράς –η μεταβλητή πρέπει να έχει πάρει τιμή, έστω και την τιμή `null`.



## Παράδειγμα

```

class Bla {
    public static void main(String[] args) {
        //    Tt t = new Tt();
        //    Tt t = new Ttm();
        //    Tt t = null;  // variable must be initialized

        Tt.bla();
        Ttm.bla();

        //    Ttm t = new Ttm();

        //    t.bla();
        }
}

class Tt {
    static void bla() {
        System.out.println("Tt.bla()");
    }
}

class Ttm extends Tt {
    static void bla()
// void bla()    // compilation error
    { System.out.println("Ttm.bla()"); }
}

```

## Αρχικοποίηση Κλάσεων

- Όταν μια κλάση πρόκειται να χρησιμοποιηθεί —άρα και να φορτωθεί στη JVM—, π.χ. λόγω χρήσης κάποιου πεδίου της, κλήσης μεθόδου της, δημιουργίας αντικειμένου της, πριν να πραγματοποιηθεί η διαδικασία αυτή, η κλάση αρχικοποιείται.
- Η διαδικασία αρχικοποίησης κλάσης θα γίνει άπαξ, την πρώτη φορά που θα χρησιμοποιηθεί η κλάση.
- Κατά τη διαδικασία αυτή, στην περίπτωση που η κλάση περιέχει στατικά πεδία, αυτά αρχικοποιούνται σύμφωνα με τις δηλώσεις τους.
- Η διαδικασία αρχικοποίησης μιας κλάσης δεν αφορά μόνο την κλάση που χρησιμοποιείται άμεσα, αλλά και όλη την ιεραρχία των κλάσεων των οποίων είναι υποκλάση. Δηλαδή, θα αρχικοποιηθούν όλες οι μεταβλητές κλάσης των κλάσεων αυτών.
- Θυμηθείτε: οι ίδιες οι κλάσεις αποτελούν στιγμιότυπα μιας μετακλάσης, οπότε, πριν τη χρήση τους, καλό είναι να αποκτήσουν μια καλά ορισμένη αρχική κατάσταση.

- Κατά τη διαδικασία αρχικοποίησης μιας κλάσης, πέρα από την αρχικοποίηση των μεταβλητών κλάσης, θα εκτελεστούν και τα στατικά *block* αρχικοποίησης της κλάσης.
- Τα στατικά *block* αρχικοποίησης:
  - συντακτικά, είναι ίδια με τα απλά *block* αρχικοποίησης
  - προσημαίνονται με τη λέξη-κλειδί *static*
  - ο κώδικάς τους δεν εκτελείται κάθε φορά που δημιουργείται αντικείμενο της κλάσης, αλλά συμμετέχει στη διαδικασία αρχικοποίησής της
  - στο σώμα τους μπορούμε να περιγράψουμε διαδικασίες που θα θέλαμε να εκτελεστούν κατά την αρχικοποίηση και δεν είναι δυνατόν αυτό να γίνει με επιμέρους κλήσεις μεθόδων κατά την αρχικοποίηση των στατικών μεταβλητών της κλάσης

## Διαδικασία Αρχικοποίησης για (μεταβλητές) Κλάση(ς) και Τύπους Διεπαφής

- Η αρχικοποίηση μιας κλάσης έγκειται στο να εκτελεστούν οι `static block` διαδικασίες αρχικοποίησης και να γίνουν οι αρχικοποιήσεις των στατικών πεδίων της.
- Η αρχικοποίηση ενός τύπου διεπαφής έγκειται στο να εκτελεστούν οι διαδικασίες αρχικοποίησης για τα πεδία του τύπου.
- Οι παραπάνω διαδικασίες εκτελούνται με τη σειρά που εμφανίζονται στον ορισμό της κλάσης (ή του τύπου διεπαφής).
- Πριν γίνει η αρχικοποίηση μιας κλάσης, γίνεται η αρχικοποίηση της άμεσης υπερκλάσης της, αλλά όχι των τύπων διεπαφής που υλοποιεί.
- Όταν γίνεται αρχικοποίηση ενός τύπου διεπαφής, δεν γίνεται αρχικοποίηση των τύπων διεπαφής τους οποίους εξειδικεύει (μέσω `extends`).
- Η αρχικοποίηση κλάσεων ή των τύπων διεπαφής γίνεται αμέσως πριν συμβεί κάποιο από τα εξής (το πρώτο χρονικά εξ αυτών):
  - Αν πρόκειται για κλάση, δημιουργία στιγμιοτύπου της.
  - Αν πρόκειται για κλάση, καλείται μια στατική μέθοδός της.
  - Γίνεται ανάθεση σε στατικό πεδίο της κλάσης.
  - Ένα στατικό πεδίο του τύπου χρησιμοποιείται (εκτός αν πρόκειται για πεδίο με πρωτογενή τύπο ή τύπο `String`, αρχικοποιημένου σε σταθερά).

- Μια αναφορά στατικού μέλους προκαλεί αρχικοποίηση μόνο της κλάσης η οποία το δηλώνει, ακόμα και αν η αναφορά γίνεται με χρήση υποκλάσης ή, αν πρόκειται για τύπο διεπαφής, κλάσης που το υλοποιεί.
- Η διαδικασία αρχικοποίησης για ένα στατικό πεδίο αποτιμάται άπαξ και η ανάθεση γίνεται επίσης άπαξ, όταν γίνεται η αρχικοποίηση της κλάσης του.
- Στη φάση της εκτέλεσης, πρώτα αρχικοποιούνται τα στατικά πεδία που έχουν δηλωθεί `final` και αρχικοποιούνται με απλές εκφράσεις (τυπικά: `constant expressions`).

## Τα Πακέτα της Java

- Τα πακέτα αποτελούν εργαλείο οργάνωσης κώδικα, παρέχοντας δική τους εμβέλεια στην οποία μπορούμε να περιορίσουμε τη χρήση ονομάτων.
- Ένα πακέτο ομαδοποιεί ένα σύνολο κλάσεων και η ορατότητά τους μπορεί να περιοριστεί στο πακέτο.
- Έχουμε τη δυνατότητα να περιορίσουμε τη χρήση των μελών μιας κλάσης μέσα στο πακέτο στο οποίο ορίζεται η κλάση –μη προσδιορίζοντας ορατότητα κατά τη δήλωση του μέλους στην κλάση.

## Γενικές παρατηρήσεις στα πακέτα

- Κάθε πακέτο έχει τον δικό του χώρο ονομάτων τύπων.
- Τα μέλη των πακέτων είναι τύποι, για παράδειγμα κλάσεις και τύποι διεπαφής.
- Οι τύποι που συνιστούν ένα πακέτο αποκαλούνται *μέλη του πακέτου* (package members) και μπορούμε να περιορίσουμε τη χρήση τους μόνο στο πακέτο αυτό.
- Στη γενική περίπτωση, αναφερόμαστε σε ένα όνομα που ορίζεται μέσα σε ένα πακέτο, χρησιμοποιώντας το πλήρες όνομα το οποίο συμπεριλαμβάνει το όνομα του πακέτου, π.χ.  
`p1.p11.BlaClass.bla();`
- Μπορούμε, όμως να κάνουμε δηλώσεις εισαγωγής με χρήση της λέξης-κλειδιού `import`, π.χ.  
`import p1.p11.BlaClass`
- Μια δήλωση εισαγωγής μπορεί να αναφέρεται σε ολόκληρο πακέτο ή σε συγκεκριμένο μέλος του.
- Όταν γίνει μια δήλωση εισαγωγής, επιτρέπεται να χρησιμοποιούνται τύποι από το άλλο πακέτο με το απλό όνομά τους μόνο και όχι το fully qualified, στο οποίο αναφέρεται ρητά το πακέτο, π.χ.  
`BlaClass.bla();`
- Ένα πακέτο περιλαμβάνει μία ή παραπάνω *μονάδες μεταγλώττισης*. Κάθε μονάδα μεταγλώττισης έχει άμεση πρόσβαση σε όλους τους τύπους του πακέτου στο οποίο ανήκει, καθώς και στους `public` τύπους του πακέτου της γλώσσας `java.lang`, που προσφέρει κλάσεις που είναι θεμελιώδεις για τη γλώσσα.

- Μια μονάδα μεταγλώττισης αποτελείται από τρία (προαιρετικά) μέρη:
  1. Μια δήλωση πακέτου, που περιλαμβάνει το πλήρες όνομα (fully qualified name) του πακέτου στο οποίο συμμετέχει η μονάδα, π.χ.
 

```
package p1.p11;
```

 Αν παραλείπεται η δήλωση, η μονάδα είναι μέρος του πακέτου χωρίς όνομα, (*unnamed* πακέτου).
  2. Δηλώσεις εισαγωγής.
  3. Δηλώσεις τύπων.
- Υπάρχει η εκδοχή *static import* για να “εισαγάγουμε” τα στατικά μέλη κλάσεων ενός πακέτου –γενικά, όμως, η χρήση του `import static` δεν προτείνεται.



Η οργάνωση του κώδικα σε πακέτα συνδέεται με την κατανομή του κώδικα του πηγαίου προγράμματος σε αρχεία.

Συγκεκριμένα, ισχύουν τα παρακάτω.

- Η οργάνωση των ονομάτων των πακέτων είναι ιεραρχική.
- Η ιεραρχία στην οργάνωση των πακέτων δεν έχει κάποια σχέση με τα δικαιώματα πρόσβασης.
- Παρόλο που το πρότυπο της Java δεν απαιτεί ιεραρχικό σύστημα αρχείων, συνήθως οργανώνουμε τα αρχεία του κώδικά μας ως εξής:
  - Γράφουμε τον κώδικά κάθε κλάσης (τύπου, γενικότερα) σε ένα αρχείο με όνομα το όνομα της κλάσης και κατάληξη `.java`
  - Τοποθετούμε το αρχείο αυτό σε έναν κατάλογο με όνομα το όνομα του πακέτου που ανήκει ο τύπος.
  - Αντίστοιχη οργάνωση έχουν και τα αρχεία `.class` που προκύπτουν από τη μεταγλώττιση.
  - Το πλήρες μονοπάτι του καταλόγου κάτω από τον οποίο βρίσκεται η ιεραρχία των `.class` αρχείων λέγεται *μονοπάτι κλάσεων* (`class path`) και προσδιορίζεται από τη μεταβλητή περιβάλλοντος/συστήματος (`CLASSPATH`).



Το παρόν αποτελεί σημειώσεις βασισμένες στο σύγγραμμα:

Καράλη, Ι. (2023).

*“Αντικειμενοστραφής Προγραμματισμός: Έννοιες και Τεχνικές”* .

Κάλλιπος, Ανοικτές Ακαδημαϊκές Εκδόσεις.

ISBN: 978-618-228-050-8